

Table of Contents

1	OBS – POPolitics (Organisation Breakdown Structure)	18
1.1	0. Contexte	18
1.2	1. Pilotage & Produit	18
1.2.1	Responsable : Samy (Chef de projet / Product Owner).....	18
1.3	2. Backend & Architecture Applicative	19
1.3.1	Responsable : Arthur – Support technique / Tech Lead : Jaures	19
1.4	3. Frontend & UX/UI	20
1.4.1	Responsables : Evilavy, Raphaël – Contribution UX/UI : Samy	20
1.5	4. Data Engineering & Data Science	20
1.5.1	Responsables : Mehdi (ETL / Data Engineering), Chloé (IA / Data Science) ..	20
1.6	5. Data Lake, Data Warehouse & Datamarts	21
1.6.1	Responsable principal : Mehdi (schémas & flux analytiques)	21
1.7	6. DevOps, Intégration & Déploiement	22
1.7.1	Responsable : Lazare.....	22
1.8	7. Qualité, Tests & Sécurité	22
1.8.1	Responsabilité partagée – Coordination : Samy & Jaures	22
1.9	8. UX/UI & Communication Produit	23
1.9.1	Responsables : Samy, Evilavy, Raphaël.....	23
1.10	9. Synthèse et spécificités.....	24
2	POPolitics – Méthodologie et gestion de projet	24
2.1	1. Objectif du document	24
2.2	2. Méthodologie choisie	24
2.2.1	2.1 Approche globale	24
2.2.2	2.2 Sprints.....	25
2.2.3	2.3 Les 5 cérémonies Scrum.....	25
2.3	3. Outil de gestion de projet.....	27
2.3.1	3.1 Outil utilisé	27
2.3.2	3.2 Bonnes pratiques d'utilisation.....	27
2.3.3	3.3 Suivi de l'avancement.....	27
2.4	4. Lien avec WBS et OBS	27
2.5	5. Jalons clés.....	28

3	POPolitics – Mapping OBS ↔ WBS	28
3.1	1. Principaux lots WBS (niveau 1)	28
3.2	2. Tableau de responsabilités (vue synthétique).....	28
3.3	3. Exemple de mapping détaillé (extrait)	29
3.3.1	3.1 Lot WBS : Plateforme Data & ETL	29
3.3.2	3.2 Lot WBS : Backend Django – Data Service	29
3.3.3	3.3 Lot WBS : Frontend – Portail collectif	30
4	Rapport de recrutement – Projet POPolitics.....	39
4.1	1. Contexte du recrutement	39
4.2	2. Méthodologie d’évaluation	40
4.2.1	Motivation et intérêt pour le projet	40
4.2.2	Compétences techniques	40
4.2.3	Capacité de travail en équipe	40
4.2.4	Polyvalence	40
4.2.5	Engagement dans le projet	40
4.3	3. Analyse des candidatures.....	40
4.4	Lazare.....	40
4.5	Samy	40
4.6	Arthur	41
4.7	Jaures	42
4.8	Alexandre Benoist	42
4.9	Alexandre Vallon	42
4.10	Evilavy	43
4.11	4. Tableau comparatif des profils.....	43
4.12	5. Composition finale de l’équipe	44
4.13	Direction technique	44
4.13.1	Tech Lead	44
4.14	Développement.....	44
4.14.1	Frontend	44
4.14.2	Backend	44
4.15	Infrastructure	44
4.15.1	DevOps.....	44
4.16	Pilotage produit.....	44

4.16.1	Chef de projet / Product Owner	44
4.17	Data	44
4.17.1	IA.....	44
4.17.2	ETL / Data Engineering	44
4.18	6. Justification des choix	45
4.19	Complémentarité des compétences.....	45
4.20	Équilibre technique et organisationnel	45
4.21	Capacité de collaboration	45
4.22	Engagement dans le projet	45
4.23	7. Candidats non retenus	45
5	Team Composition & Skills	46
5.1	1. Justification du choix de l'équipe	46
5.1.1	1.1 Besoins du projet	46
5.1.2	1.2 Arguments professionnels	46
5.2	2. Compétences requises	47
5.2.1	2.1 Gestion de projet.....	47
5.2.2	2.2 Implémentation technique.....	47
5.2.3	2.3 Autres	49
5.3	3. Gap de compétences & plan de montée en compétences	50
5.3.1	Manques identifiés	50
5.3.2	Actions prévues.....	50
6	POPolitics – Plan de communication	50
6.1	1. Introduction	50
6.2	2. Équipe & rôles de communication.....	50
6.3	3. Outils de communication	51
6.4	4. Canaux par type de communication	51
6.5	5. Cérémonies Scrum (rappel)	52
6.6	6. Règles de communication	52
6.6.1	6.1 Règles générales	52
6.6.2	6.2 Revues de code.....	53
6.6.3	6.3 Gestion des désaccords	53
6.7	7. Communication externe	53
6.7.1	7.1 Avec l'école	53

6.7.2	7.2 Comptes rendus de réunion.....	53
6.8	8. Escalade des problèmes	53
7	POPolitics – Personas.....	54
7.1	Introduction	54
7.2	Persona 1 — Le Citoyen engagé.....	54
7.2.1	 Profil	54
7.2.2	Biographie.....	54
7.2.3	Objectifs	55
7.2.4	Frustrations	55
7.2.5	Comportement numérique.....	55
7.2.6	Scénarios d’usage sur POPolitics	55
7.2.7	Citation représentative	55
7.3	Persona 2 — Le Journaliste / Expert politique.....	56
7.3.1	 Profil	56
7.3.2	Biographie.....	56
7.3.3	Objectifs	56
7.3.4	Frustrations	56
7.3.5	Comportement numérique.....	57
7.3.6	Scénarios d’usage sur POPolitics	57
7.3.7	Citation représentative	57
7.4	Persona 3 — L’Administrateur plateforme	57
7.4.1	 Profil	57
7.4.2	Biographie.....	58
7.4.3	Objectifs	58
7.4.4	Frustrations	58
7.4.5	Comportement numérique.....	58
7.4.6	Scénarios d’usage sur POPolitics	59
7.4.7	Citation représentative	59
7.5	Récapitulatif	59
8	POPolitics – Choix techniques.....	59
8.1	1. Vision générale	59
8.2	2. Objectifs fonctionnels de l’application	60
8.2.1	2.1 Objectifs principaux	60

8.2.2	2.2 Contraintes clés.....	60
8.3	3. Vision globale de l'architecture	60
8.3.1	3.1 Approche retenue	60
8.3.2	3.2 Motivations.....	60
8.4	4. Sources de données & ingestion	61
8.4.1	4.1 Sources exploitées	61
8.4.2	4.2 Caractéristiques	61
8.4.3	4.3 Choix.....	61
8.5	5. Landing Zone & Data Lake	61
8.5.1	5.1 Landing Zone – Data Lake	61
8.5.2	5.2 Organisation en 3 niveaux.....	61
8.5.3	5.3 Avantages.....	61
8.6	6. Orchestration des pipelines ETL.....	61
8.6.1	6.1 Outil : Kestra	61
8.6.2	6.2 Pourquoi Kestra ?	62
8.7	7. Stockage analytique : Datamarts	62
8.7.1	7.1 Outil : PostgreSQL.....	62
8.7.2	7.2 Raisons du choix	62
8.8	8. Couche IA & Model Training	62
8.8.1	8.1 Socle IA	62
8.8.2	8.2 Cas d'usage.....	62
8.8.3	8.3 Intégration	62
8.9	9. Backend – Django	63
8.9.1	9.1 Découpage en services.....	63
8.9.2	9.2 Communication	63
8.9.3	9.3 Objectifs.....	63
8.10	10. Authentification & gestion des utilisateurs	63
8.10.1	10.1 Base d'authentification dédiée.....	63
8.10.2	10.2 Flux	63
8.10.3	10.3 Objectifs.....	63
8.11	11. Frontend – Next.js	63
8.11.1	11.1 Architecture	63
8.11.2	11.2 Motivations	64

8.12	12. Synthèse	64
9	POPolitics – Priorisation MoSCoW	64
9.1	Méthode	64
9.2	1. Must have.....	64
9.2.1	1.1 Données & ETL (plateforme minimale)	64
9.2.2	1.2 Backend & services	65
9.2.3	1.3 Frontend (MVP Portail collectif)	65
9.2.4	1.4 Qualité, sécurité & documentation	65
9.3	2. Should have.....	65
9.3.1	2.1 Données & analytique	65
9.3.2	2.2 Couche IA.....	65
9.3.3	2.3 Frontend (tendances & page personnelle v1)	66
9.3.4	2.4 Opérations & outillage	66
9.4	3. Could have	66
9.4.1	3.1 Données & IA	66
9.4.2	3.2 Frontend & UX.....	66
9.4.3	3.3 Plateforme & DevOps	66
9.5	4. Won't have (for now)	66
10	POPolitics – Exigences fonctionnelles et non fonctionnelles liées au WBS	67
10.1	1. Portail collectif (WBS – Fonctionnalité principale)	67
10.1.1	1.1 Description	67
10.1.2	1.2 Exigences fonctionnelles	67
10.1.3	1.3 Exigences non fonctionnelles.....	67
10.1.4	1.4 Critères d'acceptation (exemples).....	68
10.2	2. Page personnelle & notifications (WBS – Fonctionnalité utilisateur)	68
10.2.1	2.1 Description	68
10.2.2	2.2 Exigences fonctionnelles	68
10.2.3	2.3 Exigences non fonctionnelles.....	68
10.2.4	2.4 Critères d'acceptation.....	68
10.3	3. Plateforme Data & ETL (WBS – Infrastructure data)	69
10.3.1	3.1 Description	69
10.3.2	3.2 Exigences fonctionnelles	69
10.3.3	3.3 Exigences non fonctionnelles.....	69

10.3.4	3.4 Critères d'acceptation.....	69
10.4	4. Datamarts & analytique (WBS – Stockage analytique).....	69
10.4.1	4.1 Description.....	69
10.4.2	4.2 Exigences fonctionnelles.....	69
10.4.3	4.3 Exigences non fonctionnelles.....	69
10.4.4	4.4 Critères d'acceptation.....	70
10.5	5. Couche IA & modèles (WBS – IA Service)	70
10.5.1	5.1 Description.....	70
10.5.2	5.2 Exigences fonctionnelles	70
10.5.3	5.3 Exigences non fonctionnelles.....	70
10.5.4	5.4 Critères d'acceptation.....	70
10.6	6. Authentification & gestion des utilisateurs (WBS – Auth Service)	70
10.6.1	6.1 Description.....	70
10.6.2	6.2 Exigences fonctionnelles	70
10.6.3	6.3 Exigences non fonctionnelles.....	70
10.6.4	6.4 Critères d'acceptation.....	70
10.7	7. Qualité, monitoring & opérations (WBS – Support).....	71
10.7.1	7.1 Description.....	71
10.7.2	7.2 Exigences fonctionnelles	71
10.7.3	7.3 Exigences non fonctionnelles.....	71
11	POPolitics – Plan Qualité & Pratiques de développement.....	71
11.1	1. Objectif.....	71
11.2	2. Méthodologie de développement.....	71
11.3	3. Standards de code et conventions	72
11.3.1	3.1 Backend (Python / Django)	72
11.3.2	3.2 Frontend (Next.js / React)	72
11.3.3	3.3 Data & ETL	72
11.4	4. Stratégie de tests	72
11.4.1	4.1 Types de tests	72
11.4.2	4.2 Organisation	72
11.5	5. Processus de déploiement	73
11.5.1	5.1 Environnements.....	73
11.5.2	5.2 Étapes de déploiement (vue générale)	73

11.6	6. Bonnes pratiques & sécurité	73
11.7	7. Onboarding d'un nouveau membre	73
12	POPolitics – Definition of Done (DoD)	74
12.1	1. Objectif	74
12.2	2. Critères généraux de complétion	74
12.2.1	2.1 Développement terminé	74
12.2.2	2.2 Tests réalisés	74
12.2.3	2.3 Revue de code	75
12.2.4	2.4 Intégration dans le repository	75
12.2.5	2.5 Documentation mise à jour	75
12.2.6	2.6 Fonctionnalité testée sur l'environnement de développement	75
12.2.7	2.7 Issue mise à jour dans l'outil de gestion de projet	75
12.3	3. Critères spécifiques selon le type de tâche	76
12.3.1	3.1 Fonctionnalité Backend	76
12.3.2	3.2 Fonctionnalité Frontend	76
12.3.3	3.3 Traitement de données / IA	76
12.4	4. Validation finale	76
12.5	5. Évolution de la Definition of Done	76
13	POPolitics – Fonctionnalités	77
13.1	1. Objectif général	77
13.2	2. Portail collectif	77
13.2.1	2.1 Historique et suivi des votes	77
13.2.2	2.2 Analyse des grandes tendances	78
13.2.3	2.3 Synthèses automatiques des débats	78
13.2.4	2.4 Indicateurs de cohérence des représentants	78
13.2.5	2.5 Visualisations interactives	78
13.3	3. Page personnelle	79
13.3.1	3.1 Notifications personnalisées	79
13.3.2	3.2 Vue simplifiée des actions et décisions des élus suivis	79
13.3.3	3.3 Fiches éclair express sur les lois, projets et amendements	79
14	POPolitics – Comparaison des stacks techniques	79
14.1	1. ETL & Orchestration	80
14.1.1	1.1 Kestra vs Airflow	80

14.2	2. Backend.....	80
14.2.1	2.1 Django vs FastAPI.....	80
14.2.2	2.2 Backend Python vs Backend JavaScript	81
14.3	3. Frontend	82
14.3.1	3.1 Next.js vs SPA React classique.....	82
14.4	4. IA & NLP	82
14.4.1	4.1 Framework de modélisation NLP	82
14.4.2	4.2 Exposition du service IA	83
14.5	5. Synthèse	83
15	POPolitics – Ressources matérielles & coûts	84
15.1	1. MVP local – Docker sur machine de développement.....	84
15.2	2. V1 – Serveur local (on-premise léger).....	84
15.3	3. Version Cloud – Hébergement managé.....	85
15.4	4. Cohérence avec les choix techniques	85
16	POPolitics – User Stories	86
16.1	1. Introduction	86
16.2	2. Personas.....	86
16.3	3. Épics.....	86
16.4	Epic 1 – Authentification & Gestion de compte.....	87
16.4.1	US-101 – Créer un compte	87
16.4.2	US-102 – Se connecter.....	87
16.4.3	US-103 – Se déconnecter	87
16.4.4	US-104 – Réinitialiser son mot de passe	88
16.5	Epic 2 – Portail collectif : Votes	88
16.5.1	US-201 – Consulter le tableau des votes	88
16.5.2	US-202 – Filtrer les votes	88
16.5.3	US-203 – Filtrer les votes par thème, commission et type de texte.....	89
16.5.4	US-204 – Consulter les indicateurs d’un vote.....	89
16.5.5	US-205 – Exporter les résultats de vote	89
16.6	Epic 3 – Portail collectif : Tendances & Analyses	90
16.6.1	US-301 – Visualiser les indicateurs de cohérence d’un élu	90
16.6.2	US-302 – Comparer deux élus ou deux groupes	90
16.6.3	US-303 – Visualiser la cartographie des alliances	90

16.6.4	US-304 – Croiser les données AN / Sénat / UE.....	90
16.7	Epic 4 – Portail collectif : Synthèses IA	91
16.7.1	US-401 – Obtenir un résumé automatique d’un débat	91
16.7.2	US-402 – Consulter une fiche éclair sur un texte de loi	91
16.8	Epic 5 – Page personnelle : Suivi & Notifications	92
16.8.1	US-501 – Suivre des élus	92
16.8.2	US-502 – Consulter la timeline d’un élu suivi	92
16.8.3	US-503 – Configurer ses thèmes d’intérêt.....	92
16.8.4	US-504 – Recevoir des notifications personnalisées	93
16.9	Epic 6 – Plateforme Data & ETL	93
16.9.1	US-601 – Ingérer les données de l’Assemblée nationale	93
16.9.2	US-602 – Ingérer les données du Sénat	93
16.9.3	US-603 – Ingérer les données du Parlement européen.....	94
16.9.4	US-604 – Normaliser les données en zone Silver	94
16.9.5	US-605 – Préparer les données analytiques en zone Gold.....	94
16.10	4. Récapitulatif	95
17	POPolitics - Plan de Sprints	95
17.1	1. Introduction	95
17.1.1	Principes.....	96
17.2	2. Vue d’ensemble.....	96
17.3	3. Detail des sprints	96
17.3.1	Sprint 1 - Fondations (Must).....	96
17.3.2	Sprint 2 - MVP Portail collectif.....	97
17.3.3	Sprint 3 - Enrichissements (Should)	97
17.3.4	Sprint 4 - IA et finitions (Should/Could).....	98
17.4	4. Sprint 5 - Stabilisation & soutenance	98
17.5	5. Récapitulatif.....	98
18	POPolitics – Registre des risques	99
18.1	1. Méthode d’évaluation	99
18.2	2. Registre des risques	99
18.2.1	Catégorie : Données & Sources externes.....	99
18.2.2	Catégorie : Technique & Architecture	102
18.2.3	Catégorie : Organisation & Équipe	105

18.2.4	Catégorie : Périmètre & Livraison	107
18.3	3. Récapitulatif par criticité	108
18.3.1	Critiques (score 6–9)	108
18.3.2	Moyens (score 3–4)	108
18.3.3	Faibles (score 1–2)	108
18.4	4. Suivi	109
19	POPolitics – Exigences non-fonctionnelles (NFR)	109
19.1	1. Introduction	109
19.2	2. Performance	109
19.2.1	2.1 Temps de réponse API	109
19.2.2	2.2 Pipeline ETL	109
19.2.3	2.3 Chargement frontend	110
19.3	3. Scalabilité	110
19.3.1	3.1 Charge utilisateurs	110
19.3.2	3.2 Volume de données	110
19.4	4. Disponibilité & fiabilité	110
19.4.1	4.1 Disponibilité cible	110
19.4.2	4.2 Gestion des pannes	111
19.4.3	4.3 Reprise après incident	111
19.5	5. Sécurité	111
19.5.1	5.1 Authentification & autorisation	111
19.5.2	5.2 Protection des données	111
19.5.3	5.3 Sécurité des échanges	111
19.5.4	5.4 Données personnelles (RGPD)	112
19.6	6. Maintenabilité	112
19.6.1	6.1 Lisibilité du code	112
19.6.2	6.2 Traçabilité	112
19.6.3	6.3 Évolutivité de l’architecture	112
19.7	7. Compatibilité	112
19.7.1	7.1 Navigateurs supportés	112
19.7.2	7.2 Résolutions	113
19.8	8. Contraintes d’infrastructure (MVP)	113
20	POPolitics – Sources de données	113

20.1	1. Vue d'ensemble	113
20.2	2. Assemblée nationale.....	113
20.2.1	2.1 Portail	113
20.2.2	2.2 Données disponibles.....	113
20.2.3	2.3 Fréquence de mise à jour.....	114
20.2.4	2.4 Particularités	114
20.3	3. Sénat	114
20.3.1	3.1 Portail	114
20.3.2	3.2 Données disponibles.....	114
20.3.3	3.3 Fréquence de mise à jour.....	115
20.3.4	3.4 Particularités	115
20.4	4. Parlement européen	115
20.4.1	4.1 Portail	115
20.4.2	4.2 Données disponibles.....	115
20.4.3	4.3 Fréquence de mise à jour.....	115
20.4.4	4.4 Particularités	116
20.5	5. Formats et ingestion	116
20.5.1	5.1 Formats bruts attendus	116
20.5.2	5.2 Stratégie d'ingestion (pipeline ETL)	116
20.5.3	5.3 Identifiants croisés.....	116
20.6	6. Licences et conditions d'utilisation	117
20.7	7. Limitations connues.....	117
20.8	8. Évolutions envisagées	118
21	POPolitics – Sécurité & RGPD	118
21.1	1. Introduction	118
21.2	2. Périmètre des données	118
21.3	3. Authentification & gestion des accès.....	118
21.3.1	3.1 Mécanisme d'authentification.....	118
21.3.2	3.2 Stockage des mots de passe.....	118
21.3.3	3.3 Gestion des rôles	119
21.4	4. Sécurité des échanges	119
21.4.1	4.1 Transport.....	119
21.4.2	4.2 Protection des APIs	119

21.4.3	4.3 Protection contre les injections	119
21.5	5. Protection des secrets	120
21.5.1	5.1 Règles	120
21.5.2	5.2 Gestion en production	120
21.6	6. Obligations RGPD	120
21.6.1	6.1 Base légale du traitement	120
21.6.2	6.2 Droits des utilisateurs	120
21.6.3	6.3 Données collectées sur les utilisateurs	121
21.6.4	6.4 Minimisation des données	121
21.6.5	6.5 Consentement	121
21.7	7. Sécurité de l'infrastructure	121
21.7.1	7.1 Isolation des bases de données	121
21.7.2	7.2 Journalisation	121
21.7.3	7.3 Mises à jour de sécurité	122
21.8	8. Responsabilités	122
22	POPolitics – Stratégie de tests	122
22.1	1. Introduction	122
22.2	2. Principes généraux.....	122
22.3	3. Pyramide de tests	123
22.4	4. Types de tests par pôle	123
22.4.1	4.1 Backend (Django)	123
22.4.2	4.2 Pipeline ETL (Kestra / Python)	124
22.4.3	4.3 Frontend (Next.js)	124
22.5	5. Environnements de test.....	125
22.6	6. Intégration continue (CI)	125
22.7	7. Données de test.....	126
22.8	8. Responsabilités	126
22.9	9. Critères de qualité minimaux (MVP).....	126
23	POPolitics – Pipeline CI/CD	127
23.1	1. Introduction	127
23.2	2. Vue d'ensemble.....	127
23.3	3. Dépôts Git	127
23.4	4. Branches.....	128

23.5	5. Pipeline CI (Intégration continue)	128
23.5.1	5.1 Dépôt Documentation	128
23.5.2	5.2 Dépôt Backend (Django)	128
23.5.3	5.3 Dépôt Frontend (Next.js).....	129
23.5.4	5.4 Dépôt Data (ETL)	129
23.6	6. Règles de merge	129
23.7	7. Pipeline CD (Déploiement continu)	129
23.7.1	7.1 Environnements	129
23.7.2	7.2 Déploiement en démo	130
23.7.3	7.3 Services déployés	130
23.8	8. Variables d'environnement.....	130
23.9	9. Responsabilités	131
24	POPolitics – Modèle de données	131
24.1	1. Introduction	131
24.2	2. Vue d'ensemble des bases de données	131
24.3	3. Datamart Assemblée nationale.....	132
24.3.1	Schéma	132
24.3.2	Tables.....	132
24.4	4. Datamart Sénat	133
24.4.1	Tables principales	134
24.5	5. Datamart Parlement européen.....	135
24.5.1	Tables principales	135
24.6	6. Datamart Cross-match	136
24.6.1	Tables principales	136
24.7	7. Base Auth.....	136
24.7.1	Schéma	137
24.7.2	Tables.....	137
24.8	8. Index et performances	138
25	POPolitics - Diagramme de Gantt.....	138
25.1	1) Vue macro.....	139
25.2	2) Vue detaillee (toutes les US)	139
26	POPolitics – Solutions IA & Model Training	140
26.1	1. Vue d'ensemble.....	140

27	2. Architecture IA	141
27.1	2.1 Vue logique.....	141
27.2	2.2 Intégration applicative.....	141
28	3. Cas d'usage IA	141
28.1	3.1 Résumés automatiques (MVP)	141
28.1.1	Objectif.....	141
28.1.2	Sources	141
28.1.3	Pipeline	142
28.2	3.2 Score de similarité textes (MVP)	142
28.2.1	Objectif.....	142
28.2.2	Approche	142
28.2.3	Technologie	142
28.2.4	Exemple de sortie	142
28.3	3.3 Chatbot parlementaire (V1).....	142
28.3.1	Objectif.....	142
28.3.2	Architecture	142
28.3.3	Sources interrogées.....	142
28.3.4	Capacités	143
29	4. Modèles IA	143
29.1	4.1 Modèles LLM	143
29.2	4.2 Modèles d'embeddings	143
30	5. Stockage IA.....	143
30.1	5.1 Base vectorielle	143
31	6. Déploiement.....	143
31.1	6.1 Environnement	143
31.1.1	Développement :	143
31.1.2	Production :	143
32	7. Limites connues.....	144
33	Benchmarks POPolitics.....	144
33.1	0. Démarrage rapide bench.ps1	144
33.2	1. Backend Django vs FastAPI (overhead du framework).....	144
33.2.1	Prérequis	145
33.2.2	Lancer (automatique) — recommandé.....	145

33.2.3	Lancer (manuel, 2 terminaux)	145
33.2.4	Mesurer les services réels	145
33.3	2. Frontend Next.js (SSR/SSG) vs SPA React	145
33.3.1	Prérequis	146
33.3.2	a) Core Web Vitals (Lighthouse)	146
33.3.3	b) Crawlabilité / SEO (HTML brut)	146
33.4	3. Où reporter les résultats	146
34	POPolitics - Dossier marketing	147
34.1	1. Introduction	147
34.1.1	1.1 Presentation du projet	147
34.1.2	1.2 Contexte et origine de l'idée	147
34.1.3	1.3 Objectifs du projet	147
34.2	2. Presentation de l'offre	147
34.2.1	2.1 Concept de la plateforme	148
34.2.2	2.2 Fonctionnement du service	148
34.2.3	2.3 Description des offres et trajectoire de monetisation	148
34.2.4	2.4 Experience client	149
34.3	3. Etude de marche	149
34.3.1	3.1 Analyse du marche	149
34.3.2	3.2 Analyse de la demande	150
34.3.3	3.3 Personas	150
34.3.4	3.4 Analyse de la concurrence	151
34.3.5	3.5 Tendances du marche	151
34.3.6	3.6 Analyses strategiques	151
34.4	4. Positionnement et strategie marketing	153
34.4.1	4.1 Positionnement	153
34.4.2	4.2 Proposition de valeur	153
34.4.3	4.3 Avantage concurrentiel	153
34.4.4	4.4 Ciblage	154
34.5	5. Marketing mix (4P)	154
34.5.1	5.1 Produit	154
34.5.2	5.2 Prix	154
34.5.3	5.3 Distribution	155

34.5.4	5.4 Communication	155
34.6	6. Approche centree client (4C)	155
34.6.1	6.1 Client	155
34.6.2	6.2 Cout.....	156
34.6.3	6.3 Convenience	156
34.6.4	6.4 Communication	156
34.7	7. Strategie de communication.....	157
34.7.1	7.1 Objectifs.....	157
34.7.2	7.2 Cibles.....	157
34.7.3	7.3 Messages	157
34.7.4	7.4 Canaux.....	157
34.7.5	7.5 Plan media	157
34.8	8. Plan d'actions marketing.....	158
34.8.1	8.1 Actions digitales.....	158
34.8.2	8.2 Actions evenementielles.....	158
34.8.3	8.3 Partenariats	158
34.8.4	8.4 Calendrier editorial	159
34.9	9. Budget previsionnel	159
34.9.1	9.1 Repartition des couts	159
34.9.2	9.2 ROI attendu	160
34.10	10. Indicateurs de performance (KPI).....	160
34.10.1	10.1 Notoriete.....	160
34.10.2	10.2 Acquisition	160
34.10.3	10.3 Engagement	161
34.10.4	10.4 Fidelisation	161
34.11	Conclusion	161
35	POPolitics – Plan de Communication Externe	161
35.1	1. Introduction	161
35.2	2. Objectifs de communication.....	162
35.3	3. Cibles de communication.....	162
35.3.1	3.1 Cibles primaires.....	162
35.3.2	3.2 Cibles secondaires.....	163
35.4	4. Messages clés	164

35.4.1	Message principal	164
35.4.2	Messages secondaires.....	164
35.5	5. Canaux et stratégie par canal	164
35.5.1	5.1 GitHub (dépôt public)	164
35.5.2	5.2 LinkedIn	164
35.5.3	5.3 X (Twitter) / Mastodon.....	165
35.5.4	5.4 Présentation jury / soutenance.....	165
35.5.5	5.5 Email / Relations presse	166
35.6	6. Ligne éditoriale & charte de communication	166
35.6.1	Ton général	166
35.6.2	Ce qu'on ne fait PAS	166
35.6.3	Identité visuelle (à définir en Sprint 4).....	167
35.7	7. Calendrier de communication global.....	167
35.7.1	7.1 Pendant le projet (calé sur les sprints)	167
35.7.2	7.2 Post-projet — Calendrier éditorial (après soutenance).....	167
35.8	8. Responsabilités	169
35.9	9. Indicateurs de succès (KPIs)	170

1 OBS – POPolitics (Organisation Breakdown Structure)

1.1 0. Contexte

Projet étudiant de Master 2 visant à concevoir une plateforme web POPolitics pour l'analyse de données politiques et citoyennes. L'OBS ci-dessous structure l'organisation de l'équipe en unités de responsabilité, en cohérence avec l'organigramme et le document Team composition & Skills.

1.2 1. Pilotage & Produit

1.2.1 Responsable : Samy (Chef de projet / Product Owner)

- **1.1 Gouvernance du projet**
 - 1.1.1 Définition de la vision produit POPolitics
 - 1.1.2 Animation des réunions d'équipe / comités

- 1.1.3 Suivi des jalons académiques (livrables, soutenance)
 - **1.2 Gestion de projet & backlog**
 - 1.2.1 Planification globale (roadmap, sprint planning)
 - 1.2.2 Priorisation du backlog fonctionnel
 - 1.2.3 Suivi des tâches (Trello / Notion / GitHub Projects)
 - **1.3 Coordination inter-pôles**
 - 1.3.1 Synchronisation Frontend / Backend / Data / DevOps
 - 1.3.2 Gestion des dépendances entre lots techniques
 - 1.3.3 Arbitrage des charges et disponibilités de l'équipe
-

1.3 2. Backend & Architecture Applicative

1.3.1 Responsable : Arthur – Support technique / Tech Lead : Jaures

- **2.1 Architecture backend**
 - 2.1.1 Choix du framework (ex. Django / FastAPI)
 - 2.1.2 Structuration des modules backend (IA Service, Data Service, Auth Service, couches d'API)
 - 2.1.3 Standards de développement et bonnes pratiques
- **2.2 Développement des APIs**
 - 2.2.1 Conception des endpoints REST exposés par les services (Data, IA, Auth) vers la couche frontend / BFF Next.js
 - 2.2.2 Gestion des accès aux données exposées par les services Data (APIs internes / connecteurs)
 - 2.2.3 Intégration avec les services Data & IA et le service d'authentification (Auth Service / Auth Database), sans gestion directe de la plate-forme data (data lake / datamarts)
- **2.3 Sécurité & performances backend**
 - 2.3.1 Gestion de l'authentification / autorisation
 - 2.3.2 Validation des entrées et gestion des erreurs
 - 2.3.3 Optimisation des temps de réponse (caching, requêtes)

1.4 3. Frontend & UX/UI

1.4.1 Responsables : Evilavy, Raphaël – Contribution UX/UI : Samy

- **3.1 Architecture frontend**
 - 3.1.1 Choix des technologies frontend (Next.js / React, HTML/CSS/JS)
 - 3.1.2 Organisation des vues et composants, avec séparation BFF (Next Server / Server Actions) et Next Client
 - 3.1.3 Intégration du design system POPolitics
 - **3.2 Implémentation de l'interface**
 - 3.2.1 Intégration des maquettes UX/UI
 - 3.2.2 Consommation des APIs backend
 - 3.2.3 Gestion des états de chargement / erreurs / feedbacks utilisateurs
 - **3.3 Visualisation de données**
 - 3.3.1 Choix des bibliothèques de visualisation (graphiques, cartes, etc.)
 - 3.3.2 Conception des dashboards et écrans d'analyse
 - 3.3.3 Optimisation de l'ergonomie et de la lisibilité des données
-

1.5 4. Data Engineering & Data Science

1.5.1 Responsables : Mehdi (ETL / Data Engineering), Chloé (IA / Data Science)

- **4.1 Ingestion & préparation des données**
 - 4.1.1 Conception des pipelines ETL depuis les sources (AN, Sénat, UE) vers la Landing Zone
 - 4.1.2 Gestion des couches DATA BRONZE (raw), DATA SILVER (soft process) et DATA GOLD (ready to ingest)
 - 4.1.3 Choix des formats et emplacements de stockage (Parquet / filesystem / cloud scalable)
- **4.2 Modélisation & analyse**
 - 4.2.1 Définition des indicateurs et KPI politiques / citoyens

- 4.2.2 Développement des modèles d'analyse ou de machine learning
 - 4.2.3 Validation des modèles et interprétation des résultats
 - **4.3 Intégration au produit**
 - 4.3.1 Exposition des fonctionnalités data via services / APIs, y compris l'API FastAPI du socle IA vers le backend Django
 - 4.3.2 Collaboration avec le frontend pour la restitution visuelle
 - 4.3.3 Documentation des jeux de données et des modèles
 - **4.4 Orchestration des pipelines**
 - 4.4.1 Conception et configuration des workflows d'orchestration dans Kestra
 - 4.4.2 Supervision, logging et relance des jobs ETL
 - 4.4.3 Coordination avec le pôle DevOps pour l'exécution en environnement contrôlé
-

1.6 5. Data Lake, Data Warehouse & Datamarts

1.6.1 Responsable principal : Mehdi (schémas & flux analytiques)

- **5.1 Conception des couches de stockage**
 - 5.1.1 Modélisation logique des données pour le data lake (Landing Zone BRONZE / SILVER / GOLD) et le data warehouse
 - 5.1.2 Choix de l'architecture de stockage et des technologies (filesystem / cloud, formats Parquet, etc.)
 - 5.1.3 Définition des clés, index et contraintes d'intégrité pour les usages analytiques
- **5.2 Datamarts analytiques PostgreSQL**
 - 5.2.1 Conception et implémentation des datamarts AN, Sénat, UE et Cross-match dans PostgreSQL
 - 5.2.2 Optimisation des schémas tabulaires pour l'interrogation par les APIs backend
 - 5.2.3 Gestion des évolutions de schémas en fonction des besoins métier
- **5.3 Administration & accès**

- 5.3.1 Création et maintenance des environnements de stockage (data lake / data warehouse / datamarts)
 - 5.3.2 Gestion des connexions et performances des requêtes analytiques
 - 5.3.3 Stratégie de sauvegarde minimale (contexte projet étudiant)
-

1.7 6. DevOps, Intégration & Déploiement

1.7.1 Responsable : Lazare

- **6.1 Environnements & outils**
 - 6.1.1 Définition des environnements (dev / démo / éventuellement prod)
 - 6.1.2 Mise en place de la gestion de versions (Git / GitHub)
 - 6.1.3 Standardisation de la configuration locale de l'équipe
 - **6.2 Intégration continue (CI)**
 - 6.2.1 Configuration des pipelines CI de base (lint, tests)
 - 6.2.2 Vérification automatique des pull requests
 - 6.2.3 Reporting minimal vers l'équipe (statut build/tests)
 - **6.3 Déploiement & hébergement**
 - 6.3.1 Choix de la plateforme d'hébergement (cloud / PaaS) adaptée au projet étudiant
 - 6.3.2 Scripts de déploiement ou fichiers de configuration (Docker, etc.)
 - 6.3.3 Supervision légère et gestion des incidents
 - 6.3.4 Mise en production et supervision des workflows Kestra, des services Django (Data, IA, Auth) et de la stack Next.js
-

1.8 7. Qualité, Tests & Sécurité

1.8.1 Responsabilité partagée – Coordination : Samy & Jaures

- **7.1 Stratégie de test**
 - 7.1.1 Définition des types de tests (unitaires, intégration, tests manuels)

- 7.1.2 Répartition des responsabilités de test par pôle
 - 7.1.3 Plan de recette avant démonstrations / soutenance
 - **7.2 Mise en œuvre des tests**
 - 7.2.1 Écriture des tests techniques essentiels (backend / data)
 - 7.2.2 Scénarios de tests fonctionnels côté frontend
 - 7.2.3 Suivi des anomalies et corrections
 - **7.3 Sécurité & conformité minimale**
 - 7.3.1 Revue des accès API et des droits utilisateurs
 - 7.3.2 Application des bonnes pratiques de développement sécurisé
 - 7.3.3 Revue de code croisée entre membres de l'équipe
-

1.9 8. UX/UI & Communication Produit

1.9.1 Responsables : Samy, Evilavy, Raphaël

- **8.1 Recherche utilisateur & cadrage UX**
 - 8.1.1 Identification des profils d'utilisateurs cibles
 - 8.1.2 Clarification des besoins d'analyse et de visualisation
 - 8.1.3 Définition des parcours utilisateurs principaux
 - **8.2 Design d'interface**
 - 8.2.1 Création des maquettes et prototypes
 - 8.2.2 Définition des chartes graphiques et composants
 - 8.2.3 Itérations sur la base des retours équipe / encadrants
 - **8.3 Documentation & communication**
 - 8.3.1 Contribution aux supports de présentation du projet
 - 8.3.2 Participation à la rédaction de la documentation fonctionnelle
 - 8.3.3 Mise en cohérence des messages clés (vision POPolitics)
-

1.10 9. Synthèse et spécificités

- Organisation légère et pragmatique : chaque bloc OBS est dimensionné pour être gérable par 1–2 personnes au sein d’une équipe étudiante.
- Polyvalence assumée : certains membres interviennent sur plusieurs blocs (ex. Samy sur pilotage + UX, Mehdi sur data + base de données).
- Priorisation forte : les parties critiques (APIs, frontend, data principale, déploiement minimal) sont couvertes clairement par des responsables identifiés.
- Alignement avec l’organigramme : la hiérarchie de cet OBS reprend les pôles affichés dans l’organigramme (pilotage, backend, frontend, data, DevOps, qualité).
- Séparation claire des couches : plate-forme data (Landing Zone, datamarts PostgreSQL, Kestra), couche backend Django (services Data/IA/Auth) et couche frontend Next.js (BFF, Next Server, Next Client).

2 POPolitics – Méthodologie et gestion de projet

2.1 1. Objectif du document

Ce document décrit la **méthodologie de gestion de projet** retenue pour POPolitics et la manière dont l’équipe organise le suivi des tâches, des livrables et des responsabilités.

2.2 2. Méthodologie choisie

2.2.1 2.1 Approche globale

Le projet POPolitics s’appuie sur un cadre **Scrum adapté** au contexte étudiant :

- Des **sprints à durée adaptée** au rythme d’alternance structurent l’avancement.
- **GitHub Projects** est utilisé comme tableau **Kanban** pour visualiser le flux de travail au quotidien — les deux pratiques coexistent : Scrum cadre les itérations, Kanban visualise l’état des tâches en temps réel.
- La priorisation est guidée par :
 - la valeur fonctionnelle (MoSCoW),
 - la complexité technique (story points),
 - les jalons académiques (livrables, soutenances).

2.2.2 2.2 Sprints

La durée des sprints est **fixe** et alignée sur le rythme d'alternance M2 :

Rythme M2	Durée d'un sprint
Cycle de référence	9 semaines

- Les sprints ont une durée cible de **9 semaines**.
- Les semaines de présentiel (follow-up) sont intégrées dans le sprint en cours ; elles ne créent pas de mini-sprint séparé.
- En fin d'année, le dernier sprint peut être plus court pour s'aligner sur la date de fin du projet.
- Chaque sprint démarre avec un **Sprint Planning** et se clôt avec une **Sprint Review** et une **Sprint Retrospective**.
- Le contenu d'un sprint est sélectionné depuis le **Product Backlog** (liste priorisée de toutes les user stories et tâches du projet).

2.2.2.1 Story points et estimation

L'effort de chaque tâche est estimé en **story points** selon les tailles de T-Shirt et l'échelle de Fibonacci :

Points	Complexité indicative
XXS - 1	Trivial, moins d'1 h de travail
XS - 2	Simple, quelques heures
S - 3	Modéré, une demi-journée
M - 5	Complexe, 1-2 jours
L - 8	Très complexe, nécessite découpage
XL - 13+	Trop large, à découper obligatoirement

2.2.2.2 Vitesse

- La **vitesse** correspond au nombre total de story points **complétés** (Definition of Done respectée) sur un sprint.
- Elle est mesurée à chaque fin de sprint et sert de référence pour calibrer le Sprint Planning suivant.
- Elle n'est pas une contrainte rigide mais un indicateur de capacité réelle de l'équipe.

2.2.3 2.3 Les 5 cérémonies Scrum

2.2.3.1 Sprint Planning

- **Quand** : début de chaque sprint (jour 1).
- **Durée** : 1 h maximum.
- **Participants** : toute l'équipe ou certains représentants indispensables pour ne pas mobiliser trop de temps d'équipe.

- **Objectif** : sélectionner les items du Product Backlog à intégrer dans le sprint, les estimer en story points si ce n'est pas encore fait, et définir l'objectif du sprint (*Sprint Goal*).
- **Output** : Sprint Backlog validé et Sprint Goal énoncé clairement, jusqu'à la Mise en Production (MEP).

2.2.3.2 Daily Standup (asynchrone)

- **Quand** : chaque jour ouvré ou tout les 2-3 jours si taille d'équipe trop importante et daily trop long.
- **Format** : message court dans le canal de l'équipe ou réunion (teams ou présentielle).
- **Structure** : chaque membre répond en quelques lignes à :
 1. Qu'ai-je fait depuis hier ?
 2. Que vais-je faire aujourd'hui ?
 3. Ai-je un blocage ?
- **Durée** : < 5 min par personne.
- **Output** : visibilité quotidienne sur l'avancement et les blocages.

2.2.3.3 Affinage du backlog

- **Quand** : en milieu de sprint (autour du jour 7).
- **Durée** : 30min-1h en fonction du nombre de ticket.
- **Participants** : toute l'équipe.
- **Objectif** : préparer les prochains sprints en affinant, découpant et estimant les tickets du Product Backlog pas encore prêts.
- **Output** : items « prêts » (bien définis, estimés, priorisés) pour le prochain Sprint Planning.

2.2.3.4 Sprint Review

- **Quand** : dernier jour du sprint.
- **Durée** : 30-45 min.
- **Participants** : toute l'équipe.
- **Objectif** : démontrer ce qui a été livré pendant le sprint et recueillir des retours.
- **Output** : incrément démontré, Product Backlog ajusté en fonction des retours.

2.2.3.5 Sprint Retrospective

- **Quand** : après la Sprint Review, toujours en fin de sprint.
- **Durée** : 30 min.
- **Participants** : toute l'équipe.
- **Objectif** : améliorer en continu la façon de travailler (processus, communication, outillage).
- **Structure** : 3 questions :
 1. Qu'est-ce qui a bien fonctionné ?

2. Qu'est-ce qui a moins bien fonctionné ?
 3. Qu'allons-nous améliorer dès le prochain sprint ?
- **Output** : 1 à 3 actions concrètes d'amélioration actées pour le sprint suivant.

NB : Le Sprint Review et le Sprint Retrospective se feront au cours de la même réunion

2.3 3. Outil de gestion de projet

2.3.1 3.1 Outil utilisé

- Outil de gestion de tâches type **GitHub Projects** (Kanban) avec :
 - un **tableau unique POPolitics**,
 - des **colonnes** standard : Todo, In Progress, Code review, Deployable, In pre-production, Done.

2.3.2 3.2 Bonnes pratiques d'utilisation

- Chaque **tâche** correspond à un élément du **WBS** (ou à une sous-tâche associée).
- Chaque carte contient :
 - un titre clair,
 - une description courte,
 - un lien vers la spécification (WBS / fonctionnalités / exigences),
 - le ou les **responsables** (membres OBS).
- Les étiquettes peuvent représenter :
 - le pôle concerné (Data, Backend, Frontend, DevOps, UX),
 - la priorité (Must / Should / Could).

2.3.3 3.3 Suivi de l'avancement

- L'avancement est suivi par :
 - le **nombre de cartes** dans chaque colonne,
 - les dates limites associées aux livrables majeurs,
 - des captures d'écran régulières pour les rapports.
-

2.4 4. Lien avec WBS et OBS

- Chaque **lot WBS** est décliné en tâches dans l'outil de gestion.
- Chaque tâche est **assignée** à un ou plusieurs membres selon l'OBS (voir document de mapping OBS ↔ WBS).
- Le **chef de projet** (Samy) :
 - contrôle la cohérence backlog ↔ WBS ↔ MoSCoW,
 - arbitre les priorités avec l'équipe,

- s'assure que les tâches critiques sont bien couvertes par des responsables identifiés.

2.5 5. Jalons clés

Quelques jalons de référence (à adapter selon le planning) :

- Validation de l'architecture globale (plateforme data + backend + frontend).
- Livraison WBS + OBS + MoSCoW.
- Première démonstration interne du **Portail collectif**.
- Première démonstration interne de la **Page personnelle**.
- Livraison du **Plan Qualité** et mise en place des premiers tests.
- Préparation de la **soutenance** (diapositives, scénario de démo).

3 POPolitics – Mapping OBS ↔ WBS

Ce document fait le lien entre : - la **structure organisationnelle** (OBS), - les **lots de travail** définis dans le WBS.

L'objectif est de montrer **qui est responsable de quoi** pour chaque grande brique du projet.

3.1 1. Principaux lots WBS (niveau 1)

Les intitulés ci-dessous correspondent aux grands blocs fonctionnels et techniques du WBS POPolitics.

1. Pilotage & Produit
2. Plateforme Data & ETL
3. Datamarts & Analytique
4. Couche IA & Modèles
5. Backend Django (Services Data / IA / Auth)
6. Frontend Next.js (Portail collectif & Page personnelle)
7. DevOps & Orchestration (Kestra, déploiements)
8. Qualité, Tests & Sécurité

3.2 2. Tableau de responsabilités (vue synthétique)

Lot WBS (niveau 1)	Responsable principal (OBS)	Contributeurs clés (OBS)
1. Pilotage & Produit	Samy (Chef de	Tous les leads techniques (Arthur,

Lot WBS (niveau 1)	Responsable principal (OBS)	Contributeurs clés (OBS)
	projet / PO)	Mehdi, Chloé, Lazare, Front)
2. Plateforme Data & ETL	Mehdi	Chloé (IA), Lazare (DevOps)
3. Datamarts & Analytique	Mehdi	Arthur (consommation via APIs), Chloé (besoins analytiques)
4. Couche IA & Modèles	Chloé	Mehdi (données), Arthur (intégration via IA Service)
5. Backend Django (Data / IA / Auth Services)	Arthur	Jaures (support tech), Lazare (déploiement), Chloé (IA)
6. Frontend Next.js (Portail & Page personnelle)	Evilavy & Raphaël	Samy (UX/UI), Arthur (contrats d'API)
7. DevOps & Orchestration (Kestra, déploiements)	Lazare	Mehdi (workflows data), Arthur (services backend)
8. Qualité, Tests & Sécurité	Samy & Jaures	Tous les membres selon leur périmètre

3.3 3. Exemple de mapping détaillé (extrait)

3.3.1 3.1 Lot WBS : Plateforme Data & ETL

- **Tâches principales :**
 - Conception pipelines d'ingestion (AN, Sénat, UE).
 - Mise en place de la Landing Zone (Bronze / Silver / Gold).
 - Orchestration des jobs dans Kestra.
- **Responsable :** Mehdi.
- **Contributeurs :**
 - Chloé (besoins IA sur les données Silver/Gold),
 - Lazare (environnements d'exécution, monitoring).

3.3.2 3.2 Lot WBS : Backend Django – Data Service

- **Tâches principales :**
 - Exposition des datamarts PostgreSQL via des APIs REST.
 - Gestion des contrats d'API avec le frontend.
 - Gestion des erreurs et des performances côté backend.
- **Responsable :** Arthur.
- **Contributeurs :**
 - Mehdi (schémas des datamarts),
 - Evilavy & Raphaël (besoins côté UI),
 - Lazare (déploiement).

3.3.3 3.3 Lot WBS : Frontend – Portail collectif

- **Tâches principales :**
 - Développement des écrans de suivi des votes et tendances.
 - Intégration des visualisations (graphes, tableaux, filtres).
 - Intégration avec les APIs backend.
- **Responsables :** Evilavy & Raphaël.
- **Contributeurs :**
 - Sammy (priorisation fonctionnelle, UX),
 - Arthur (contrats d'API et endpoints).

Ce document peut être complété avec d'autres lots WBS détaillés, de façon similaire, si nécessaire.

GRILLE D'EVALUATION

Notation suggérée :

1 = insuffisant · 2 = fragile · 3 = correct · 4 = solide · 5 = excellent
(pondération indiquée si tu veux calculer un score global)

1. Compétences techniques backend

Critère	Ce que tu évalues	Indices attendus	Note
Langage backend	Maîtrise d'au moins un langage (Python / Node / Java...)	Syntaxe, bonnes pratiques, clarté	/5
Logique métier	Capacité à structurer des règles métier	Séparation des responsabilités	/5
Conception d'API	REST / GraphQL	Routes claires, statuts HTTP, versioning	/5
Bases de données	SQL / NoSQL	Modélisation, choix cohérents	/5
Qualité du code	Lisibilité, organisation	Nommage, modularité, tests	/5

- As-tu déjà travaillé sur un projet backend ?
- Quel était ton rôle exact dans ce projet ? Quelles parties as-tu développées ?

2. Fiabilité, qualité

Critère	Ce que tu évalues	Indices attendus	Note
Gestion des erreurs	Robustesse	Messages clairs, cas limites	/5

Critère	Ce que tu évalues	Indices attendus	Note
Sensibilité sécurité	Réflexes de base	Données sensibles, prudence	/5
Stabilité	Comportement attendu	Évite les crashes, tests simples	/5
• Y a-t-il des données ou des actions auxquelles tu fais plus attention ? • As-tu déjà dû corriger un bug important ?			

3. Profil, posture & motivation

Critère	Ce que tu évalues	Indices attendus	Note
Autonomie	Capacité à avancer	Recherche, compréhension	/5
Prise de recul	Capacité à analyser	Ce qui a bien/mal fonctionné	/5
Motivation projet	Intérêt pour POPolitics	Sens, impact citoyen	/5
• Qu'est-ce qui te plaît dans le développement backend ? • Qu'est-ce qui t'intéresse dans le projet POPolitics ?			

GRILLE D'EVALUATION

Notation suggérée

1 = insuffisant · 2 = fragile · 3 = correct · 4 = solide · 5 = excellent

1. Compétences techniques frontend

Critère	Ce que tu évalues	Indices attendus	Note
HTML / CSS	Bases solides	Sémantique, responsive, layout	/5
JavaScript	Maîtrise du langage	États, événements, async	/5
Framework front	React / Vue / Angular	Composants, props, state	/5
Intégration API	Communication backend	Fetch, gestion erreurs	/5
Performance front	Optimisation	Lazy loading, bundle	/5
• Peux-tu montrer ou décrire une interface que tu as réalisée ? • À quoi servait-elle ? Pour qui était-elle faite ?			

2. UX / UI & design d'interfaces

Critère	Ce que tu évalues	Indices attendus	Note
---------	-------------------	------------------	------

Critère	Ce que tu évalues	Indices attendus	Note
Intégration des maquettes	Fidélité & adaptation	Respect design, pragmatisme	/5
Parcours utilisateur	Clarté et logique	Hiérarchie, feedback	/5
Lisibilité & pédagogie	Compréhension des infos	Simplification, priorisation	/5
Cohérence visuelle	Design system	Couleurs, typographie	/5
Sens utilisateur	Empathie UX	Justification des choix	/5

- À quoi fais-tu attention quand tu finalises une interface ?
- Qu'est-ce qui t'agace dans une mauvaise interface ?

3. Visualisation de données

Critère	Ce que tu évalues	Indices attendus	Note
Choix des graphiques	Pertinence	Graphique adapté au message	/5
Outils de dataviz	D3, Chart.js, etc.	Usage raisonné	/5
Interactions	UX data	Tooltips, filtres, hover	/5

- As-tu déjà créé une interface avec de la dataviz ?
- Comment as-tu choisi les types de graphiques ? Intérêt pour l'UX data ?

4. Profil, posture & motivation

- Qu'est-ce qui t'intéresse dans ce projet Popolitics ?

GRILLE D'EVALUATION

Notation suggérée :

1 = insuffisant · 2 = fragile · 3 = correct · 4 = solide · 5 = excellent
(pondération indiquée si tu veux calculer un score global)

1. Architecture applicative & vision globale – Tech Lead

Critère	Ce que tu évalues	Indices attendus	Note
Compréhension des architectures web	Vision end-to-end	Front ↔ Backend ↔ Data	/5

Critère	Ce que tu évalues	Indices attendus	Note
Structuration du projet	Organisation des modules	Clean architecture, layers	/5
Évolutivité & maintenabilité	Anticipation du futur	Scalabilité, refactor	/5
Choix techniques	Capacité à argumenter	Avantages / limites	/5
Flux inter-systèmes	Backend ↔ Data & IA	APIs, events, pipelines	/5
Robustesse	Gestion des erreurs	Logs, exceptions, fallback	/5
• As-tu déjà participé à l'élaboration d'un projet end-to-end ? Quel était ton rôle ? • Sur quels modules es-tu intervenu ? • Qu'est-ce que tu as le + apprécié ? • Qu'est-ce qui t'a posé le plus de problèmes ? • Si tu devais refaire ce projet aujourd'hui, qu'est-ce que tu ferais différemment ?			

2. Sécurité applicative & architecture (15 %)

Critère	Ce que tu évalues	Indices attendus	Note
Principes de base sécurité	Culture générale	AuthN/AuthZ, OWASP Top 10	/5
Sécurité des API	Bonnes pratiques	JWT, OAuth, rate limiting	/5
Protection des données	Données sensibles	Hash, chiffrement, RGPD	/5
Sécurité architecture	Vision globale	Moindre privilège, isolation	/5
Gestion des erreurs & logs	Sécurité opérationnelle	Pas de leak d'infos	/5
• As-tu déjà assuré le module cybersécu d'une appli ? • As-tu des notions de RGPD / Protection des données ?			

3. Profil, posture & motivation

Critère	Ce que tu évalues	Indices attendus	Note
Appétence architecture	Intérêt réel	Curiosité, références	/5
Prise de recul	Maturité technique	"Pourquoi" plutôt que "comment"	/5
Motivation projet	Intérêt pour l'impact		

Critère	Ce que tu évalues	Indices attendus	Note
	citoyen		

- Qu'est-ce qui te motive dans Popolitics ?

Tech Lead

Mission principale

Le/la Tech Lead aide à structurer et faire avancer le projet POPolitics dans sa globalité. Il/elle participe au développement (backend, frontend, parfois devops) tout en veillant à la cohérence technique, aux bons choix d'architecture et à la bonne communication entre les pôles du projet.

Ce que tu feras

- Participer au développement du projet (backend, front et un peu de devops selon les besoins)
- Réfléchir à l'architecture globale de l'application
- Aider à choisir les technologies, outils et bonnes pratiques
- Veiller à ce que le code reste propre, compréhensible et cohérent
- Aider les autres membres de l'équipe sur les sujets techniques
- Documenter simplement le fonctionnement et les choix du projet
- Aider à mettre en place de bonnes pratiques de sécurité côté front, back et déploiement
- Vérifier la cohérence entre les besoins fonctionnels et la solution technique proposée

Ce qu'on attend de toi

- Être à l'aise avec le développement web en général (front et/ou back)
- Comprendre comment fonctionne une application web de bout en bout
- Avoir des bases en APIs, bases de données et déploiement
- Aimer structurer un projet et prendre du recul sur les choix techniques
- Avoir envie d'aider, d'expliquer et de travailler en équipe

Ce que tu vas apporter au projet

- Une vision technique globale et cohérente
- Des choix techniques clairs et adaptés au projet
- Un code plus lisible et plus facile à maintenir
- Une meilleure coordination technique dans l'équipe

Profil recherché

- Étudiant(e) avec une bonne culture technique générale
- À l'aise pour prendre un rôle de référent technique sans hiérarchie
- Curieux(se), polyvalent(e) et impliqué(e)
- Intéressé(e) par les projets à impact citoyen

Développeur Backend

Mission principale

Le/la Développeur Backend participe à la construction du cœur technique de POpolitics. Il/elle développe les fonctionnalités côté serveur et contribue à une application fiable, cohérente et bien structurée, en lien avec le front-end et la partie Data / IA.

Ce que tu feras

- Développer les fonctionnalités backend de l'application
- Concevoir et implémenter des APIs (logique métier, accès aux données)
- Travailler avec les bases de données du projet
- Collaborer avec l'équipe front pour assurer une bonne intégration
- Appliquer les bonnes pratiques de code et de sécurité
- Participer aux choix techniques et aux discussions d'architecture
- Documenter simplement ce que tu développes

Ce qu'on attend de toi

- Être à l'aise en développement backend (Python, Node.js, Java ou autre)
- Comprendre le fonctionnement des APIs et des bases de données
- Avoir une vision globale d'une application web
- Être attentif(ve) à la qualité et à la lisibilité du code
- Avoir envie de travailler en équipe et d'apprendre

Ce que tu vas apporter au projet

- Des fonctionnalités backend fiables et bien pensées
- Des APIs claires et faciles à utiliser

- Un code maintenable et compréhensible
- Une contribution active à l'avancement du projet

Profil recherché

- Étudiant(e) avec un bon niveau en backend
- Curieux(se), impliqué(e) et motivé(e)
- À l'aise pour travailler en équipe sur un projet commun
- Intéressé(e) par les projets à impact citoyen

Développeur Frontend — UX/UI Designer

Mission principale

Le Développeur Frontend – UX/UI Designer est responsable du **développement des interfaces utilisateurs**, en veillant à leur **lisibilité, accessibilité et pédagogie**, à partir de données politiques complexes.

Ce que tu feras

- Développer les interfaces web de l'application
- Intégrer les maquettes UX/UI dans le code
- Concevoir des parcours utilisateurs clairs et intuitifs
- Travailler la visualisation de données (graphiques, interactions)
- Garantir la cohérence visuelle et ergonomique
- Optimiser les performances front-end

Ce qu'on attends de toi

- Développement front-end (HTML, CSS, JavaScript)
- Frameworks front (React, Vue, Angular ou équivalent)
- Sensibilité UX/UI et design d'interfaces
- Bases en data visualisation (D3, Chart.js, etc.)
- Souci de l'accessibilité et de la clarté

Ce que tu vas apporter au projet

- Interfaces fonctionnelles et responsive
- Intégration complète avec les API backend
- Composants UI réutilisables

- Améliorations continues de l'expérience utilisateur

Profil recherché

- Étudiant(e) développeur(se) frontend
- Appétence forte pour le design et l'UX
- Capacité à vulgariser l'information par l'interface
- Sens du détail et de l'utilisateur final

DevOps

Mission principale

Le/la DevOps aide à mettre en place et maintenir l'infrastructure technique du projet POPolitics.

Il/elle s'assure que l'application peut être **déployée, testée et mise à jour facilement**, tout en garantissant la **stabilité, la sécurité et la performance** de la plateforme.

Ce que tu feras

- Mettre en place l'environnement technique du projet (hébergement, serveurs, services et automatiser les déploiements de l'application
- Mettre en place des pipelines CI/CD pour faciliter les tests et les mises en production et gérer l'infrastructure et les environnements (dev, test, production)
- Aider l'équipe à comprendre comment déployer et tester leurs développements
- Surveiller la performance et la stabilité de l'application et participer à la mise en place des bonnes pratiques de sécurité et d'accès
- Documenter simplement l'infrastructure et les procédures de déploiement et travailler avec les équipes backend, frontend et data pour faciliter l'intégration technique

Ce qu'on attend de toi

- Avoir des bases en infrastructure web et en déploiement d'applications
- Comprendre comment fonctionne une application web en production
- Connaître les principes de CI/CD et d'automatisation
- Avoir des bases sur les environnements cloud ou serveurs
- Aimer structurer, automatiser et fiabiliser les processus techniques

Ce que tu vas apporter au projet

- Une application plus simple à déployer et à maintenir
- Une infrastructure plus stable et sécurisée
- Des mises à jour plus rapides et plus fiables
- Une meilleure collaboration entre les équipes techniques

Profil recherché

- Étudiant(e) intéressé(e) par l'infrastructure et les outils DevOps
- Curieux(se) des technologies cloud et d'automatisation
- Organisé(e) et rigoureux(se)
- À l'aise pour travailler avec plusieurs profils techniques

Product Owner

Mission principale

Le/la Product Owner est responsable de la **vision produit du projet POPolitics**.

Il/elle s'assure que le produit développé correspond bien aux **besoins des utilisateurs** et aux **objectifs du projet**, en priorisant les fonctionnalités et en coordonnant le travail avec les équipes techniques.

Ce que tu feras

- Définir la vision et les objectifs du produit et identifier les besoins des utilisateurs et les transformer en fonctionnalités
- Prioriser les tâches, les évolutions du produit et rédiger, structurer les user stories ou besoins fonctionnels
- Travailler avec les équipes front, backend et data pour clarifier les besoins et aider à organiser le travail et les priorités du projet
- Vérifier que les fonctionnalités développées correspondent bien aux attentes et participer à la réflexion sur l'évolution du produit
- Documenter les décisions fonctionnelles du projet

Ce qu'on attend de toi

- Aimer réfléchir à l'expérience utilisateur et à l'utilité du produit
- Être capable de transformer un besoin en fonctionnalité concrète

- Avoir un bon sens de la communication avec les équipes techniques
- Aimer comprendre les problématiques des utilisateurs
- Être capable de prendre du recul sur le produit et savoir organiser et prioriser des idées

Ce que tu vas apporter au projet

- Une vision claire de ce que le produit doit devenir
- Des priorités bien définies pour l'équipe
- Un produit plus utile et plus cohérent pour les utilisateurs
- Une meilleure coordination entre les besoins et le développement

Profil recherché

- Étudiant(e) intéressé(e) par les produits numériques et leur conception
- À l'aise pour organiser des idées et structurer un projet
- Curieux(se) des usages et des besoins utilisateurs
- Bon communicant, capable de faire le lien entre vision et technique

4 Rapport de recrutement – Projet POPolitics

4.1 1. Contexte du recrutement

Dans le cadre du développement du projet **POPolitics**, une phase de recrutement a été organisée afin de constituer une équipe pluridisciplinaire capable de répondre aux différents besoins techniques et organisationnels du projet.

L'objectif était de réunir des profils complémentaires permettant de structurer une équipe autour de plusieurs domaines clés :

- développement **Frontend**
- développement **Backend**
- **Data / Intelligence Artificielle**
- **Infrastructure / DevOps**
- **Pilotage produit**

Le processus de recrutement visait également à constituer une équipe capable de collaborer efficacement sur un projet technique complexe, en recherchant un équilibre entre **compétences techniques, motivation et capacité à travailler en équipe**.

4.2 2. Méthodologie d'évaluation

Les candidats ont été rencontrés et évalués selon plusieurs critères.

4.2.1 Motivation et intérêt pour le projet

Évaluation de la capacité du candidat à comprendre les enjeux du projet et à se projeter dans son développement.

4.2.2 Compétences techniques

Analyse du niveau de maîtrise des technologies nécessaires au développement de la solution.

4.2.3 Capacité de travail en équipe

Facilité à collaborer, communiquer et s'intégrer dans une équipe projet.

4.2.4 Polyvalence

Capacité à intervenir sur plusieurs aspects du projet ou à comprendre différents domaines techniques.

4.2.5 Engagement dans le projet

Priorité accordée au projet POPolitics par rapport à d'autres candidatures ou projets.

4.3 3. Analyse des candidatures

4.4 Lazare

- Intérêt pour le projet : +++
- Travail en groupe : +++
- Profil polyvalent pouvant intervenir sur plusieurs postes

Lazare présente un profil stable et collaboratif, particulièrement adapté à un rôle lié à **l'infrastructure et au support technique de l'équipe.**

4.5 Samy

- Orientation : Frontend / UX-UI

- Compétences UX/UI : +++
- Travail en groupe : ++++
- Capacités transverses
- Technologies proches de celles utilisées par Raphaël

Samy possède de bonnes compétences relationnelles et organisationnelles, adaptées à un rôle de **coordination et de structuration du produit**.

4.6 Arthur

- Orientation : Frontend / Full Stack / DevOps
- Intérêt pour le projet : +++
- Compétences UX/UI : ++++
- Niveau technique : ++++
- Kotlin
- Sécurité et gestion de version : +++
- Dataviz : ++
- Animation 3D
- Travail en équipe : ++++
- Motivation : ++++

Arthur présente un **profil technique très complet et polyvalent**, avec une forte motivation pour le projet.

Il avait également candidaté dans une autre équipe mais a indiqué **préférer rejoindre notre projet**.

4.7 Jaures

- Poste envisagé : Tech Lead / Backend
- Travail en équipe : ++++
- Capacité d'architecture : +++
- Python
- Expérience avec **API et Machine Learning**

Jaures possède un profil particulièrement adapté à un rôle de **réfèrent technique et de structuration de l'architecture du projet**.

4.8 Alexandre Benoist

- Orientation : Backend
- Connaissances Frontend
- Compétences sur différents frameworks et langages
- Intérêt pour le projet : ++

Profil technique intéressant mais avec un niveau d'engagement légèrement inférieur à certains autres candidats.

4.9 Alexandre Vallon

- Orientation : Backend / Frontend
- Backend : Django
- Frontend : Angular
- Travail en équipe : ++++
- Candidature partagée avec le projet **Tides (50/50)**

Profil technique solide mais disponibilité partagée entre plusieurs projets.

4.10 Evilavy

- Orientation : Frontend
- Stack frontend complète : ++++
- Notions de backend
- Sensibilité UX/UI : ++++
- Intérêt pour le projet : +++
- Motivation à s'impliquer dans le projet

Evilavy a également candidaté sur **Tides et Triply**, mais a indiqué **donner la priorité à notre projet**.

Profil particulièrement adapté au **développement frontend et à l'expérience utilisateur**.

4.11 4. Tableau comparatif des profils

Candidat	Orientation	Technique	Travail équipe	Motivation projet	Particularités
Lazare	DevOps / Infra	++	+++	+++	Profil polyvalent
Samy	Front / UX	+++	++++	+++	Capacités transverses
Arthur	Full Stack	++++	++++	++++	Profil très technique
Jaures	Backend / Lead	+++	++++	+++	Architecture / API / ML
A. Benoist	Backend	+++	++	++	Connaissances front
A. Vallon	Backend / Front	+++	++++	++	Django / Angular
Evilavy	Frontend	++++	+++	+++	Forte sensibilité UX/UI

4.12 5. Composition finale de l'équipe

Suite à l'analyse des candidatures, l'équipe suivante a été retenue.

4.13 Direction technique

4.13.1 Tech Lead

- Jaures
-

4.14 Développement

4.14.1 Frontend

- Evilavy
- Raphaël

4.14.2 Backend

- Arthur
-

4.15 Infrastructure

4.15.1 DevOps

- Lazare
-

4.16 Pilotage produit

4.16.1 Chef de projet / Product Owner

- Samy
-

4.17 Data

4.17.1 IA

- Chloé

4.17.2 ETL / Data Engineering

- Mehdi
-

4.18 6. Justification des choix

La composition finale de l'équipe repose sur plusieurs principes.

4.19 Complémentarité des compétences

L'équipe couvre l'ensemble des domaines nécessaires :

- développement frontend
- développement backend
- infrastructure
- data et intelligence artificielle
- pilotage produit

4.20 Équilibre technique et organisationnel

La présence d'un **Tech Lead**, d'un **DevOps** et d'un **Product Owner** permet de structurer efficacement le projet et de clarifier les responsabilités.

4.21 Capacité de collaboration

Les profils retenus présentent une bonne capacité à travailler en équipe et à communiquer entre les différents pôles techniques.

4.22 Engagement dans le projet

Les candidats sélectionnés ont démontré une motivation réelle pour s'investir dans le projet POPolitics.

4.23 7. Candidats non retenus

Les candidats suivants n'ont pas été retenus dans la composition finale de l'équipe :

- Alexandre Vallon
- Alexandre Benoist

Bien que leurs profils soient intéressants, la sélection finale a privilégié **la cohérence globale de l'équipe, la complémentarité des compétences et l'engagement dans le projet.**

5 Team Composition & Skills

5.1 1. Justification du choix de l'équipe

5.1.1 1.1 Besoins du projet

- **Type de solution :**
Plateforme web permettant d'exploiter, visualiser et analyser des données politiques et citoyennes.
 - **Technologies envisagées :**
 - Frontend : HTML, CSS, JavaScript / framework frontend
 - Backend : Python (API)
 - Data : pipelines ETL et modèles IA
 - Base de données : base relationnelle ou data warehouse
 - Infrastructure : déploiement cloud / CI-CD
 - **Contraintes (délai, complexité, etc.) :**
 - Développement dans un cadre académique avec un temps limité
 - Coordination entre plusieurs pôles techniques
 - Besoin de structurer un projet complet (frontend, backend, data, IA, infrastructure)
-

5.1.2 1.2 Arguments professionnels

- **Complémentarité des compétences :**

L'équipe a été construite pour couvrir les différents besoins techniques du projet :

- Frontend et UX/UI pour l'interface utilisateur
 - Backend pour la logique applicative et les APIs
 - Data / IA pour l'analyse et le traitement des données
 - DevOps pour l'infrastructure et le déploiement
 - Gestion produit pour coordonner les développements
- **Expériences passées :**

Certains membres ont déjà travaillé sur :

- développement d'API
- projets Python et data
- UX/UI et développement frontend
- architecture technique

- **Disponibilités :**

Les membres sélectionnés ont montré une motivation forte et une volonté de s'investir prioritairement dans le projet POPolitics.

- **Adéquation avec les besoins du projet :**

La structure de l'équipe permet d'assurer :

- une organisation technique claire
 - une répartition des responsabilités
 - une bonne communication entre les pôles.
-

5.2 2. Compétences requises

5.2.1 2.1 Gestion de projet

Responsable : **Samy (Chef de projet / Product Owner)**

Compétences nécessaires :

- Planification du projet
- Organisation des tâches
- Priorisation des fonctionnalités
- Coordination entre les équipes techniques

Outils envisagés :

- Trello / Notion / Planner
 - GitHub Projects
 - Suivi des issues
 - Kanban ou backlog produit
-

5.2.2 2.2 Implémentation technique

5.2.2.1 Backend

Responsable : **Arthur**

Support technique : **Jaures (Tech Lead)**

Compétences :

- Développement d'API
 - Python
 - Framework backend (ex : Django / FastAPI)
 - Gestion des accès aux données
 - Intégration avec les services data et IA
-

5.2.2.2Frontend

Responsables :

- **Evilavy**
- **Raphaël**

Compétences :

- HTML / CSS / JavaScript
 - intégration UI
 - consommation d'API
 - optimisation de l'expérience utilisateur
 - gestion de l'interface et visualisation des données
-

5.2.2.3Base de données / Data

Responsables :

- **Mehdi (ETL / Data engineering)**
- **Chloé (IA / Data science)**

Compétences :

- conception de pipelines ETL
 - préparation des données
 - stockage et structuration des données
 - modèles d'analyse ou de machine learning
 - exploitation des données pour le produit
-

5.2.2.4DevOps / déploiement

Responsable : **Lazare**

Compétences :

- gestion des environnements (dev / prod)

- automatisation des déploiements
 - CI/CD
 - gestion de l'infrastructure
 - monitoring et stabilité de l'application
-

5.2.3 2.3 Autres

5.2.3.1 UX/UI

Responsables :

- **Samy**
- **Evilavy**
- **Raphaël**

Compétences :

- conception de l'interface
 - ergonomie
 - expérience utilisateur
 - cohérence visuelle de la plateforme
-

5.2.3.2 Tests

Responsabilités partagées dans l'équipe :

- tests des fonctionnalités
 - validation des développements
 - vérification des intégrations entre les modules
-

5.2.3.3 Sécurité

Responsabilités :

- validation des accès API
 - gestion des droits
 - bonnes pratiques de développement sécurisé
 - revue de code
-

5.3 3. Gap de compétences & plan de montée en compétences

5.3.1 Manques identifiés

- structuration complète d'un projet full-stack
 - mise en place d'une architecture DevOps
 - intégration entre data, backend et frontend
 - gestion de pipelines data et IA dans une application
-

5.3.2 Actions prévues

Pour combler ces écarts :

- consultation de **documentations techniques**
- réalisation de **proof of concept (POC)**
- partage de ressources et tutoriels entre membres
- travail collaboratif et entraide dans l'équipe
- mise en place d'une documentation projet

Ces actions permettront à l'équipe de **monter en compétences progressivement tout en développant le projet.**

6 POPolitics – Plan de communication

6.1 1. Introduction

Ce document décrit les règles et outils de communication de l'équipe POPolitics, en complément de la [Méthodologie de gestion de projet](#). Il couvre la communication interne (entre membres) et externe (avec l'école et les parties prenantes).

6.2 2. Équipe & rôles de communication

Membre	Rôle	Responsabilité communication
Samy	Chef de projet / Product Owner / UX-UI	Animation des cérémonies Scrum, point de contact principal avec l'école
Jaures	Tech Lead	Coordination technique inter-pôles, arbitrage des décisions d'architecture
Arthur	Développeur Backend	Communication pôle

Membre	Rôle	Responsabilité communication
		Backend
Evilavy	Frontend / UX-UI	Communication pôle Frontend
Raphaël	Frontend / UX-UI	Communication pôle Frontend
Mehdi	Data Engineer	Communication pôle Data
Chloé	Data Scientist / IA	Communication pôle Data / IA
Lazare	DevOps	Communication pôle Infrastructure

6.3 3. Outils de communication

Outil	Usage	Audience
Discord / Slack	Communication quotidienne, daily asynchrone, alertes rapides	Toute l'équipe
GitHub Projects	Suivi des tâches, commentaires sur les issues, revues de PR	Toute l'équipe
GitHub (Issues / PR)	Discussion technique sur le code, revues, décisions liées aux tickets	Développeurs
Google Meet / Teams	Cérémonies Scrum (Sprint Planning, Review, Retro), réunions ad hoc	Toute l'équipe
Email	Communication officielle avec l'école, documents formels	Samy (+ membres concernés)
Google Drive / Notion	Partage de documents, comptes rendus de réunion	Toute l'équipe

6.4 4. Canaux par type de communication

Type de message	Canal	Délai de réponse attendu
Question rapide / blocage	Canal Discord principal	< 2 heures en journée
Daily standup (asynchrone)	Canal Discord #daily	Chaque matin avant 10h

Type de message	Canal	Délai de réponse attendu
Décision technique importante	Issue GitHub + mention en Discord	Réponse sous 24h
Revue de code (PR)	GitHub Pull Request	Sous 24–48h
Alerte critique (bug bloquant, pipeline down)	Canal Discord #alertes + message direct	< 1 heure
Communication avec l'école	Email	Selon délais de l'institution

6.5 5. Cérémonies Scrum (rappel)

Les cérémonies sont décrites en détail dans la [Méthodologie](#). Récapitulatif :

Cérémonie	Fréquence	Durée	Animateur
Sprint Planning	Début de chaque sprint	1h max	Samy (PO)
Daily Standup	Chaque jour ouvré (asynchrone)	< 5 min/membre	Chaque membre
Affinage du backlog	Milieu de sprint	30–60 min	Samy + Jaures
Sprint Review	Fin de sprint	30–45 min	Samy (PO)
Sprint Retrospective	Après la Review	30 min	Toute l'équipe

Format du daily asynchrone (message dans #daily) :

Fait : [ce que j'ai accompli]
 En cours : [ce que je fais aujourd'hui]
 Blocage : [rien / description du problème]

6.6 6. Règles de communication

6.6.1 6.1 Règles générales

- Toute décision technique structurante est **documentée par écrit** (issue GitHub ou compte rendu), pas de décision uniquement à l'oral.
- Les absences prévisibles (maladie, contrainte entreprise) sont **signalées dès que possible** dans le canal Discord.
- Les blocages non résolus en 24h sont remontés au Tech Lead (Jaures) ou au PO (Samy).
- Le canal Discord principal reste **professionnel**, les discussions hors-sujet ont leur propre canal.

6.6.2 6.2 Revues de code

- Toute PR doit être relue par **au moins un autre membre** avant merge.
- Le reviewer doit laisser un commentaire explicite même si la PR est approuvée sans modification.
- Une PR en attente de review depuis plus de **48h** est relancée en Discord.

6.6.3 6.3 Gestion des désaccords

1. Discussion directe entre les membres concernés (Discord ou appel).
 2. Si non résolu : arbitrage par le Tech Lead (Jaures) pour les sujets techniques, par le PO (Samy) pour les sujets de priorité produit.
 3. La décision finale est consignée dans une issue GitHub ou un compte rendu.
-

6.7 7. Communication externe

6.7.1 7.1 Avec l'école

- Points de contact : **Samy, Mehdi et Raphaël**.
- Toute communication officielle (email, livrables) passe par l'un d'eux ou est validée par eux avant envoi.
- Les livrables académiques sont déposés avec un délai de **48h de marge** avant la deadline pour permettre une relecture collective.

6.7.2 7.2 Comptes rendus de réunion

- Chaque Sprint Review et Sprint Retrospective donne lieu à un **compte rendu court** (5–10 lignes) partagé dans Google Drive / Notion.
 - Structure minimale d'un compte rendu :
 - Date, participants
 - Ce qui a été livré (Review)
 - Ce qui s'est bien passé / moins bien (Retro)
 - Actions décidées (1 à 3 max, avec responsable et délai)
-

6.8 8. Escalade des problèmes

Niveau	Situation	Action
1 – Individuel	Blocage technique personnel	Daily standup + message Discord
2 – Pôle	Blocage impactant un pôle entier	Réunion de pôle + alerte Jaures
3 – Projet	Risque sur un livrable ou le planning	Réunion d'urgence toute l'équipe + PO (Samy)

Niveau	Situation	Action
4 – Externe	Impact sur les livrables académiques	Communication avec l'école via Samy, Mehdi ou Raphaël

7 POPolitics – Personas

7.1 Introduction

Ce document décrit les **trois personas** du projet POPolitics. Chaque persona est une représentation fictive mais réaliste d'un type d'utilisateur cible. Ils servent de référence commune pour les décisions de design, de priorisation des fonctionnalités et de rédaction des user stories.

Ces personas sont référencés dans [09-user-stories.md](#).

7.2 Persona 1 — Le Citoyen engagé

7.2.1 Profil

Nom	Marie Fontaine
Âge	34 ans
Profession	Infirmière en CHU
Localisation	Bordeaux (33)
Situation	En couple, deux enfants
Niveau numérique	Intermédiaire — utilise smartphone, réseaux sociaux, quelques apps métier

7.2.2 Biographie

Marie travaille en service de pédiatrie depuis 10 ans. Entre ses gardes, elle suit l'actualité politique sur son téléphone — principalement via les médias sociaux et des podcasts. Elle vote à chaque élection mais se sent souvent perdue face à la complexité des textes de loi et des positions politiques. Elle a développé une méfiance vis-à-vis des médias traditionnels qu'elle trouve trop orientés, et cherche à former ses propres opinions à partir de **données brutes et vérifiables**.

Elle entend souvent parler d'un élu en particulier sur les sujets de santé et d'hôpital public, mais n'a jamais trouvé un outil simple pour suivre concrètement ce que cet élu vote réellement au parlement.

7.2.3 Objectifs

- Comprendre comment **ses représentants locaux** votent sur les sujets qui la touchent (santé, éducation, logement).
- Pouvoir **vérifier rapidement** une information politique sans lire des dizaines de pages.
- Suivre l'**actualité parlementaire** en moins de 10 minutes par jour.
- Former son opinion sur la base de **faits, pas de discours**.

7.2.4 Frustrations

- Les sites officiels (Assemblée nationale, Sénat) sont **complexes et peu intuitifs**.
- Les médias simplifient trop ou au contraire noient l'information dans du contexte.
- Elle ne sait pas toujours **qui représente sa circonscription** ni ce qu'il a voté récemment.
- Les alertes d'actualité sont soit trop nombreuses, soit trop vagues.

7.2.5 Comportement numérique

Habitude	Détail
Appareils	Smartphone (70 %), ordinateur (25 %), tablette occasionnellement
Réseaux	Instagram, X (Twitter), podcasts
Sessions	Courtes (5–10 min), souvent en transport ou pendant les pauses
Tolérance aux frictions	Faible — abandonne si une page est trop lente ou confuse

7.2.6 Scénarios d'usage sur POPolitics

1. **Vérification rapide** : Elle entend à la radio qu'un député a voté contre une loi sur les infirmières. Elle ouvre POPolitics, tape le nom du député et vérifie le vote en 30 secondes.
2. **Suivi de son élu local** : Elle ajoute son député de circonscription à ses élus suivis et reçoit une notification quand il vote sur un texte lié à la santé.
3. **Découverte** : Via le portail collectif, elle découvre qu'un groupe politique qu'elle pensait proche de ses valeurs vote régulièrement contre les textes sur le service public hospitalier.

7.2.7 Citation représentative

« Je veux juste savoir ce que font réellement les gens que j'ai élus. Pas ce qu'ils disent à la télé — ce qu'ils votent vraiment. »

7.3 Persona 2 — Le Journaliste / Expert politique

7.3.1 Profil

Nom	Thomas Marchand
Âge	41 ans
Profession	Journaliste politique indépendant & blogueur data
Localisation	Paris (75)
Situation	Célibataire, très mobile
Niveau numérique	Intermédiaire-avancé — à l'aise avec Excel, Google Sheets, outils de visualisation en ligne

7.3.2 Biographie

Thomas a travaillé 12 ans en presse nationale avant de se mettre à son compte. Il tient un blog d'analyse politique très suivi (~40 000 abonnés) et collabore ponctuellement avec des médias en ligne pour des articles de fond. Sa spécialité : le **data journalisme politique** — il transforme des données brutes en récits visuels compréhensibles pour le grand public.

Il passe plusieurs heures par semaine à extraire manuellement des données du site de l'Assemblée nationale et du Sénat, à les nettoyer dans des tableurs et à construire ses propres analyses. Ce travail fastidieux lui prend un temps précieux qu'il préférerait consacrer à l'écriture et à l'analyse.

7.3.3 Objectifs

- Accéder à des **données propres, structurées et à jour** sans effort d'extraction manuel.
- Pouvoir **croiser les données** entre AN, Sénat et Parlement européen sur un même élu ou groupe.
- Identifier rapidement des **tendances et anomalies** (votes atypiques, ruptures, alliances inattendues).
- **Exporter les données** pour les réutiliser dans ses articles et visualisations.
- Comparer des élus ou groupes sur une période longue.

7.3.4 Frustrations

- Les sites officiels (AN, Sénat, UE) sont **disparates et peu lisibles** : il faut naviguer entre plusieurs interfaces pour trouver les mêmes infos.
- Le croisement manuel entre AN et Sénat sur un même élu est **extrêmement chronophage**.
- Les outils existants sont soit trop généralistes, soit derrière des paywalls prohibitifs.

- Les données brutes disponibles sont dans des formats difficiles à exploiter sans bagage technique.

7.3.5 Comportement numérique

Habitude	Détail
Appareils	Ordinateur (Mac, 95 %)
Outils	Google Sheets, Datawrapper, Flourish, Canva
Sessions	Longues (1–3h), le matin ou tard le soir
Tolérance aux frictions	Haute — accepte la complexité si les données sont fiables et riches

7.3.6 Scénarios d’usage sur POPolitics

1. **Enquête thématique** : Il prépare un article sur les votes concernant le nucléaire depuis 2015. Il filtre par thème, période et institution, exporte le CSV et l’importe dans Datawrapper pour créer une visualisation.
2. **Portrait d’élus** : Il accède à la fiche d’un sénateur, consulte son score de cohérence avec son groupe, identifie 3 votes atypiques et les utilise comme angle d’attaque pour son article.
3. **Analyse des alliances** : Il utilise la cartographie des alliances pour identifier quels groupes ont voté ensemble sur les textes économiques lors de la dernière session. Il inclut la capture dans son article.
4. **Croisement AN/Sénat/UE** : Pour un article sur un eurodéputé qui a également été député AN, il croise ses votes sur les trois institutions pour montrer une évolution de positionnement au fil du temps.

7.3.7 Citation représentative

« Ce dont j’ai besoin, c’est d’un accès propre aux données — pas d’une interface grand public. Donnez-moi les filtres, les exports et les comparateurs, et je fais le reste. »

7.4 Persona 3 — L’Administrateur plateforme

7.4.1 Profil

Nom	Lucas Perrin
Âge	26 ans
Profession	Étudiant M2 Informatique / Membre de l’équipe POPolitics
Localisation	Lyon (69)
Rôle dans l’équipe	DevOps / Data Engineer en alternance

7.4.2 Biographie

Lucas est en dernière année de master à Epitech. Il a rejoint le projet POPolitics dès le départ, convaincu par la dimension civic-tech du projet. En entreprise, il travaille chez un éditeur SaaS B2B où il gère l'infrastructure cloud et les pipelines de données. Il apporte à l'équipe ses compétences en orchestration, déploiement et fiabilité des systèmes.

Au quotidien, il est l'un des seuls membres de l'équipe à interagir directement avec l'infrastructure de production : il surveille les pipelines Kestra, diagnostique les erreurs d'ingestion, gère les accès et s'assure que la plateforme est disponible avant chaque démo ou soutenance.

7.4.3 Objectifs

- **Maintenir les pipelines ETL** opérationnels sans intervention manuelle quotidienne.
- Détecter et corriger rapidement les **erreurs d'ingestion** (API source indisponible, format modifié, rejet de données).
- Gérer les **comptes utilisateurs** et les droits d'accès à la plateforme.
- Préparer et valider l'environnement avant les **démos et soutenances**.
- Monitorer la **santé globale** du système (services, base de données, jobs Kestra).

7.4.4 Frustrations

- Les erreurs silencieuses dans les pipelines : un job ETL échoue mais aucune alerte n'est déclenchée, les données en base sont obsolètes sans que personne ne le remarque.
- La **dépendance aux APIs publiques** instables : un changement de format sur data.assemblee-nationale.fr peut casser toute la chaîne d'ingestion.
- La gestion manuelle des **migrations de base de données** lors des mises à jour.
- Le manque de visibilité sur l'état global de la plateforme en dehors de la console Kestra.

7.4.5 Comportement numérique

Habitude	Détail
Appareils	Ordinateur (Linux/Mac), terminal omniprésent
Outils	Docker, GitHub Actions, Kestra UI, psql, logs système
Sessions	Ponctuelles et réactives — intervient sur incident ou lors des déploiements
Tolérance aux frictions	Très haute pour les outils techniques, très

7.4.6 Scénarios d’usage sur POPolitics

1. **Monitoring quotidien** : Chaque matin, il consulte le tableau de bord Kestra pour vérifier que les jobs ETL de la nuit se sont bien exécutés. Si un job est en erreur, il accède aux logs, identifie la cause (timeout API, schéma modifié) et relance manuellement ou déclenche un correctif.
2. **Gestion des utilisateurs** : Un membre de l’équipe pédagogique demande un accès démo à la plateforme. Lucas crée un compte via l’interface d’administration, lui attribue le rôle “lecture seule” et lui communique ses identifiants.
3. **Déploiement avant soutenance** : 48h avant une soutenance, il exécute le script de déploiement complet, vérifie les migrations, charge un jeu de données de démo figé et effectue les smoke tests pour s’assurer que tous les parcours utilisateurs critiques fonctionnent.
4. **Gestion d’incident** : Un pipeline d’ingestion des données du Sénat échoue deux jours de suite. Lucas analyse les logs, identifie que l’URL de l’API source a changé, met à jour le connecteur Kestra et relance le pipeline avec rejeu des données manquantes.

7.4.7 Citation représentative

« Mon job, c’est que personne ne remarque que j’existe — parce que si la plateforme tourne sans accroc, c’est que j’ai bien fait mon travail. »

7.5 Récapitulatif

Persona	Représente	Priorité fonctionnelle
Marie Fontaine — Citoyenne	Grand public, usage mobile, sessions courtes	Simplicité, notifications, filtres de base
Thomas Marchand — Journaliste/Expert	Utilisateur avancé, analyse data, desktop	Export, filtres avancés, cross-match, comparateurs
Lucas Perrin — Administrateur	Équipe interne, opérations, infrastructure	Monitoring, gestion utilisateurs, fiabilité ETL

8 POPolitics – Choix techniques

8.1 1. Vision générale

- Projet de **collecte, transformation, analyse** et **vulgarisation** de données politiques publiques.
- **Sources** : Assemblée nationale, Sénat, Parlement européen.

- **Contraintes :**
 - Budget nul au départ
 - Données publiques
 - Architecture évolutive
 - **Message clé :** Construire un pipeline **moderne, robuste et scalable, gratuit au départ.**
-

8.2 2. Objectifs fonctionnels de l'application

8.2.1 2.1 Objectifs principaux

- Centraliser les données politiques issues de plusieurs institutions.
- Nettoyer, structurer et croiser les données.
- Rendre l'information accessible et compréhensible pour un public non expert.
- Offrir une interface web performante et sécurisée.

8.2.2 2.2 Contraintes clés

- Sources multiples (AN, Sénat, UE).
 - Volumétrie évolutive.
 - Besoin de traçabilité et de reproductibilité.
 - Séparation claire des responsabilités (data / backend / frontend).
-

8.3 3. Vision globale de l'architecture

8.3.1 3.1 Approche retenue

- Architecture **modulaire en couches**.
- Séparation claire entre :
 - Pipeline ETL
 - Stockage analytique
 - Services backend
 - Interface frontend

8.3.2 3.2 Motivations

- Scalabilité (MVP local → évolution Cloud, ajout de sources, etc.).
 - Maintenabilité.
 - Possibilité d'évolution indépendante de chaque brique.
 - Alignement avec des architectures utilisées en production.
-

8.4 4. Sources de données & ingestion

8.4.1 4.1 Sources exploitées

- Données du **Sénat**.
- Données de l'**Assemblée nationale**.
- Données du **Parlement européen**.

8.4.2 4.2 Caractéristiques

- Formats hétérogènes.
- Fréquences de mise à jour variables.
- Structures parfois non normalisées.

8.4.3 4.3 Choix

- Phase **EXTRACT** dédiée.
 - Centralisation initiale avant toute transformation.
-

8.5 5. Landing Zone & Data Lake

8.5.1 5.1 Landing Zone – Data Lake

- Stockage de type fichiers/objets, pensé pour être **scalable**, qu'il soit hébergé en local (MVP, V1 serveur) ou dans le **Cloud**.
- Format **Parquet** / fichiers bruts.

8.5.2 5.2 Organisation en 3 niveaux

- **Bronze** : données brutes (traçabilité maximale).
- **Silver** : données nettoyées et pré-traitées.
- **Gold** : données prêtes à être consommées.

8.5.3 5.3 Avantages

- Séparation des responsabilités.
 - Reproductibilité des traitements.
 - Facilité de debug et d'audit.
-

8.6 6. Orchestration des pipelines ETL

8.6.1 6.1 Outil : Kestra

Rôle :

- Orchestration des tâches ETL.

- Gestion des dépendances.
- Automatisation des workflows.

8.6.2 6.2 Pourquoi Kestra ?

- Orienté **data engineering**.
 - Déclaratif et lisible (**YAML**).
 - Facilement observable (logs, retries, monitoring).
 - Outil **français**.
-

8.7 7. Stockage analytique : Datamarts

8.7.1 7.1 Outil : PostgreSQL

Organisation :

- Datamart 1 : Assemblée nationale.
- Datamart 2 : Sénat.
- Datamart 3 : Parlement européen.
- Datamart 4 : Cross-match multi-institutions.

8.7.2 7.2 Raisons du choix

- SQL standard et robuste.
 - Adapté à l'analyse tabulaire.
 - Bonne intégration avec les services backend.
-

8.8 8. Couche IA & Model Training

8.8.1 8.1 Socle IA

- Données issues de la couche **Silver/Gold**.

8.8.2 8.2 Cas d'usage

- Résumés automatiques des lois et amendements votés (MVP)
- Score de similarité lois / amendements et déclaration de chaque groupe (MVP)
- Chatbot (V1)

8.8.3 8.3 Intégration

- Les modèles IA sont exécutés dans un service dédié (IA Service), indépendant du frontend.
- Le backend Django consomme ce service via API pour les fonctionnalités IA (chatbot, résumé, similarité, classification).

- Le service IA orchestre les appels LLM, la recherche vectorielle (RAG) et les requêtes SQL.
 - Déploiement possible avec API LLM externes.
-

8.9 9. Backend – Django

8.9.1 9.1 Découpage en services

- **IA Service** : accès aux modèles et prédictions.
- **Data Service** : exposition des données métier issues des datamarts.
- **Auth Service** : gestion des utilisateurs et des droits.

8.9.2 9.2 Communication

- APIs **REST**.
- Découplage fort entre services backend et plate-forme data.

8.9.3 9.3 Objectifs

- Clarté fonctionnelle.
 - Sécurité.
 - Évolutivité indépendante.
-

8.10 10. Authentification & gestion des utilisateurs

8.10.1 10.1 Base d'authentification dédiée

- Stockage des utilisateurs.
- Rôles et permissions.

8.10.2 10.2 Flux

- Frontend → Auth Service → Auth Database.

8.10.3 10.3 Objectifs

- Sécurité.
 - Séparation des données métier et utilisateurs.
 - Préparation à des fonctionnalités avancées (comptes, profils, favoris).
-

8.11 11. Frontend – Next.js

8.11.1 11.1 Architecture

- **Next Server** : BFF (Backend For Frontend).

- **Server Actions** : logique côté serveur.
- **Next Client** : rendu côté utilisateur.

8.11.2 11.2 Motivations

- Performance (**SSR / SSG**).
- Sécurité (logique sensible côté serveur).
- Intégration naturelle avec des APIs REST.

8.12 12. Synthèse

- Séparation claire des responsabilités (ETL, stockage, services, UI).
- Application des bonnes pratiques du **data engineering moderne**.
- Structuration des données via **Data Lake** (Bronze / Silver / Gold).
- Datamarts dédiés aux usages analytiques.
- Exposition des données via **APIs REST**.
- BFF garantissant sécurité et cohérence côté frontend.

L'architecture est cohérente avec les objectifs fonctionnels, réaliste pour un projet de Master 2 et alignée avec les standards professionnels, tout en restant généralisable à d'autres contextes institutionnels.

9 POPolitics – Priorisation MoSCoW

9.1 Méthode

La priorisation MoSCoW est utilisée ici pour distinguer :

- **Must have** : indispensable pour un MVP crédible à la soutenance.
- **Should have** : important, fortement souhaité si le temps le permet.
- **Could have** : bonus / nice-to-have si la charge le permet.
- **Won't have (for now)** : hors périmètre de ce projet (phase ultérieure).

9.2 1. Must have

9.2.1 1.1 Données & ETL (plateforme minimale)

- Ingestion des données **AN, Sénat et UE** via un pipeline ETL minimal.
- Mise en place de la Landing Zone avec au moins les couches **Bronze** (brut) et **Silver** (nettoyé / normalisé).
- Orchestration de base des jobs ETL avec **Kestra** (lancement, logs, statut des exécutions).

- Création de datamarts PostgreSQL pour au moins **AN, Sénat et UE**, consommables par le backend.

9.2.2 1.2 Backend & services

- Backend **Django** opérationnel, structuré en services :
 - **Data Service** exposant des APIs REST pour le **Portail collectif** (votes, indicateurs de base),
 - **Auth Service** avec authentification basique (création de compte, login, rôles simples).
- Intégration minimale avec la plate-forme data (requêtes sur les datamarts principaux).

9.2.3 1.3 Frontend (MVP Portail collectif)

- Frontend **Next.js** avec BFF (Next Server) consommant les APIs backend.
- Pages principales :
 - Page de **connexion / inscription**,
 - Première version du **Portail collectif** :
 - tableau des votes avec filtres essentiels (institution, période, groupe politique, élu),
 - premiers graphiques simples (évolution, répartition des votes).
- Gestion minimale des erreurs et états de chargement.

9.2.4 1.4 Qualité, sécurité & documentation

- Traçabilité minimale des traitements ETL (logs Kestra, identifiants de batch, statut des jobs).
 - Séparation claire des données métier et des données d'authentification (base Auth dédiée).
 - Livraison d'une **documentation d'architecture** (choix techniques, schémas) et d'un **Plan Qualité** décrivant méthode, tests et déploiement.
-

9.3 2. Should have

9.3.1 2.1 Données & analytique

- Mise en place de la couche **Gold** dans le Data Lake (données prêtes à l'analyse).
- Datamart **Cross-match** permettant de croiser AN / Sénat / UE.
- Jeux d'**indicateurs et KPI** plus avancés (activité, typologie de votes, cohérence, opposition, alliances).

9.3.2 2.2 Couche IA

- Premier modèle IA en production (ex. **résumé automatique** de débats ou **classification** de textes politiques).

- **IA Service** exposant au moins un endpoint d'inférence vers le backend.

9.3.3 2.3 Frontend (tendances & page personnelle v1)

- Filtres avancés (dates, institutions, catégories, types de texte).
- Visualisations graphiques plus évoluées (courbes, barres, heatmaps simples).
- Première version de la **Page personnelle** :
 - sélection d'élus suivis et de thèmes d'intérêt,
 - timeline simplifiée des actions des élus suivis,
 - premières **notifications personnalisées** basées sur ces préférences.

9.3.4 2.4 Opérations & outillage

- Monitoring plus détaillé des workflows Kestra (retries, alertes simples, tableau de bord minimal).
 - Scripts de déploiement / configuration plus industrialisés (Docker ou équivalent) pour la stack Django / Next.js / Kestra.
-

9.4 3. Could have

9.4.1 3.1 Données & IA

- Modèles IA supplémentaires (analyse de tendance plus fine, détection d'anomalies, recommandation de contenus, etc.).
- Scénarios de simulation ou de projection simples à partir des données.

9.4.2 3.2 Frontend & UX

- Tableau de bord **hautement personnalisable** par utilisateur (widgets, réorganisation des blocs).
- Export avancé des résultats (CSV, PDF, infographies prêtes pour les réseaux sociaux ou la presse).
- Pages explicatives / pédagogiques détaillées sur les données, les indicateurs et la méthodologie.

9.4.3 3.3 Plateforme & DevOps

- Pipeline CI/CD automatisé (tests, build, déploiement automatique sur un environnement de démo).
 - Supervision centralisée (tableau de bord de santé des services et des jobs Kestra, alertes proactives).
-

9.5 4. Won't have (for now)

- Ingestion **temps réel** ou streaming massif de données.

- Infrastructure cloud complexe et coûteuse (cluster managé, data warehouse propriétaire payant, etc.).
- Application mobile native dédiée (Android / iOS).
- Fonctionnalités sociales avancées (commentaires, interactions entre utilisateurs).
- Mécanismes de recommandation complexes basés sur l'IA.

Ces éléments sont considérés hors périmètre pour le projet et pourront être envisagés dans une phase ultérieure si le projet est pérennisé.

10 POPolitics – Exigences fonctionnelles et non fonctionnelles liées au WBS

Ce document complète le **WBS** en décrivant, pour chaque grand lot, les **exigences fonctionnelles** (ce que le système doit faire) et **non fonctionnelles** (qualité, performances, sécurité, UX, etc.).

10.1 1. Portail collectif (WBS – Fonctionnalité principale)

10.1.1 1.1 Description

Interface principale permettant de **suivre les votes, tendances et débats** à l'échelle collective (groupes politiques, institutions, textes de loi).

10.1.2 1.2 Exigences fonctionnelles

- Afficher l'historique des votes avec filtres par :
 - institution (AN, Sénat, UE),
 - période,
 - type de texte (loi, amendement, résolution...),
 - groupe politique, élu, commission.
- Afficher les résultats de vote avec codes couleurs (Pour / Contre / Abstention / Non-votant).
- Calculer et afficher :
 - taux de participation,
 - cohésion de groupe,
 - évolution temporelle.
- Fournir des visualisations de base (tableaux + graphiques).

10.1.3 1.3 Exigences non fonctionnelles

- Temps de réponse acceptable pour les principales requêtes (ex. < 2–3 secondes sur un dataset de taille raisonnable).

- Interface responsive (desktop en priorité, utilisation possible sur tablette).
- Lisibilité : couleurs cohérentes, légendes et titres explicites.

10.1.4 1.4 Critères d'acceptation (exemples)

- Un utilisateur peut, en moins de 5 clics, afficher les votes d'un groupe politique sur une période donnée.
 - Les indicateurs de cohésion et de participation sont systématiquement mis à jour quand les filtres changent.
-

10.2 2. Page personnelle & notifications (WBS – Fonctionnalité utilisateur)

10.2.1 2.1 Description

Tableau de bord individuel permettant à un utilisateur de suivre **ses élus, ses thèmes** et de recevoir des **notifications personnalisées**.

10.2.2 2.2 Exigences fonctionnelles

- L'utilisateur peut :
 - sélectionner des élus suivis,
 - choisir des thèmes d'intérêt (économie, social, environnement...).
- Le système génère des notifications en fonction :
 - d'événements importants (vote majeur, changement de position, amendement marquant),
 - des préférences de l'utilisateur.
- Afficher une timeline des principales actions des élus suivis.

10.2.3 2.3 Exigences non fonctionnelles

- Volume de notifications limité pour éviter la fatigue (priorisation par importance / urgence / intérêt).
- Possibilité de désactiver certaines catégories d'alertes.

10.2.4 2.4 Critères d'acceptation

- Un utilisateur peut configurer ses préférences et ne reçoit ensuite que des notifications en lien avec celles-ci.
 - Les notifications affichent un lien direct vers la page correspondante (vote, texte, fiche élu).
-

10.3 3. Plateforme Data & ETL (WBS – Infrastructure data)

10.3.1 3.1 Description

Ensemble des pipelines ETL et du Data Lake (Landing Zone Bronze / Silver / Gold) permettant d'ingérer, nettoyer et structurer les données.

10.3.2 3.2 Exigences fonctionnelles

- Ingestion régulière des données AN, Sénat, UE.
- Stockage des données brutes en zone **Bronze** (traçabilité).
- Transformation vers **Silver** (nettoyage, normalisation).
- Préparation des données prêtes pour analyse en zone **Gold**.
- Orchestration des pipelines via **Kestra**.

10.3.3 3.3 Exigences non fonctionnelles

- Reproductibilité des traitements (mêmes entrées → mêmes sorties).
- Traçabilité des exécutions (logs, statut des jobs).
- Capacité à relancer un job en erreur sans casser les données.

10.3.4 3.4 Critères d'acceptation

- Pour une nouvelle source intégrée, le pipeline peut être relancé de bout en bout sans intervention manuelle lourde.
 - Les jeux de données Bronze, Silver et Gold sont clairement identifiables et documentés.
-

10.4 4. Datamarts & analytique (WBS – Stockage analytique)

10.4.1 4.1 Description

Datamarts PostgreSQL dédiés aux analyses politiques (AN, Sénat, UE, Cross-match).

10.4.2 4.2 Exigences fonctionnelles

- Mise à disposition de tables analytiques pour :
 - les votes par texte / élu / groupe,
 - les indicateurs agrégés,
 - les croisements entre institutions (datamart Cross-match).
- Actualisation des datamarts en cohérence avec les pipelines ETL.

10.4.3 4.3 Exigences non fonctionnelles

- Schémas normalisés et documentés.
- Requêtes principales exécutables en temps raisonnable (index adaptés).

10.4.4 4.4 Critères d'acceptation

- Les APIs backend peuvent récupérer les données nécessaires via des requêtes SQL stables et documentées.
-

10.5 5. Couche IA & modèles (WBS – IA Service)

10.5.1 5.1 Description

Socle IA fournissant des **résumés**, **classifications** ou autres analyses automatiques.

10.5.2 5.2 Exigences fonctionnelles

- Entraîner au moins un modèle (ex. résumé de débat ou classification de texte).
- Exposer un endpoint d'inférence via l'**IA Service**.

10.5.3 5.3 Exigences non fonctionnelles

- Temps de réponse compatible avec une utilisation interactive (ex. < 5 s pour un résumé court).
- Possibilité de journaliser les appels pour améliorer les modèles ultérieurement.

10.5.4 5.4 Critères d'acceptation

- Depuis le frontend, un utilisateur peut demander un résumé ou une analyse IA et obtenir une réponse lisible.
-

10.6 6. Authentification & gestion des utilisateurs (WBS – Auth Service)

10.6.1 6.1 Description

Service d'authentification et base d'utilisateurs dédiés.

10.6.2 6.2 Exigences fonctionnelles

- Création de compte, login / logout.
- Rôles simples (ex. utilisateur standard, admin).
- Stockage sécurisé des mots de passe.

10.6.3 6.3 Exigences non fonctionnelles

- Respect des bonnes pratiques de sécurité (hashage robuste, pas de mot de passe en clair).
- Séparation stricte entre données d'authentification et données métier.

10.6.4 6.4 Critères d'acceptation

- Impossible d'accéder aux pages protégées sans être authentifié.

- Les droits d'accès sont appliqués conformément au rôle de l'utilisateur.
-

10.7 7. Qualité, monitoring & opérations (WBS – Support)

10.7.1 7.1 Description

Ensemble des pratiques de qualité, de tests et de supervision de la plateforme.

10.7.2 7.2 Exigences fonctionnelles

- Mise en place de tests (voir Plan Qualité).
- Supervision minimale des jobs Kestra et des services backend.

10.7.3 7.3 Exigences non fonctionnelles

- Logs accessibles et compréhensibles pour diagnostiquer un incident.
- Documentation à jour des procédures de redémarrage ou de réinstallation.

Ce document est volontairement synthétique et doit être lu conjointement avec : - le **WBS** (découpage du travail), - les documents de **fonctionnalités** et de **MoSCoW**, - le **Plan Qualité**.

11 POPolitics – Plan Qualité & Pratiques de développement

11.1 1. Objectif

Définir les **règles de qualité**, les **pratiques de développement** et les **processus de test et de déploiement** pour le projet POPolitics.

11.2 2. Méthodologie de développement

- Utilisation d'une approche **Agile / Kanban** (voir document "Méthodologie et gestion de projet").
- Gestion du code source via **Git** (GitHub ou équivalent) :
 - Branche principale : main (ou master) toujours stable.
 - Branches de fonctionnalités : feature/<nom-fonctionnalite>.
 - Branches de correctifs : fix/<description-courte>.
- Règles de merge :
 - Pull request obligatoire.
 - Revue de code par au moins un autre membre pour les changements significatifs.

- Lancement des tests pertinents avant fusion.
-

11.3 3. Standards de code et conventions

11.3.1 3.1 Backend (Python / Django)

- Respect du style **PEP 8**.
- Utilisation d'un linter (par ex. flake8 ou ruff) et/ou d'un formateur (black).
- Découpage clair entre :
 - vues / endpoints,
 - services métier,
 - accès aux données (ORM / requêtes).

11.3.2 3.2 Frontend (Next.js / React)

- Linting via **ESLint** + formateur automatique (Prettier).
- Composants réutilisables pour les éléments d'UI récurrents (boutons, cartes, graphiques).
- Fichiers organisés par domaines fonctionnels (ex. portal/, profile/).

11.3.3 3.3 Data & ETL

- Scripts ETL clairs, documentés, versionnés.
 - Nommage explicite des pipelines Kestra.
 - Conservation d'un historique minimal des versions de schémas.
-

11.4 4. Stratégie de tests

11.4.1 4.1 Types de tests

- **Tests unitaires :**
 - Fonctions critiques (transformations ETL, calculs d'indicateurs, endpoints sensibles).
- **Tests d'intégration :**
 - Interactions backend ↔ datamarts PostgreSQL.
 - Appels frontend ↔ APIs REST.
- **Tests manuels :**
 - Parcours utilisateurs principaux (portail collectif, page personnelle, authentification).

11.4.2 4.2 Organisation

- Chaque nouvelle fonctionnalité critique doit être accompagnée **d'au moins un test** (unitaire ou d'intégration).

- Les tests sont exécutés :
 - localement par le développeur,
 - idéalement automatiquement via un pipeline CI (si mis en place).
-

11.5 5. Processus de déploiement

11.5.1 5.1 Environnements

- **Développement** : exécution locale de la **stack complète en Docker** (7–8 conteneurs : Django, Next.js, PostgreSQL/datamarts, base d’auth, Kestra, etc.), orchestrée via docker-compose.
- **Démo / pré-prod** : même stack Docker déployée sur une machine/VM partagée, utilisée pour les démonstrations et la maintenance.

11.5.2 5.2 Étapes de déploiement (vue générale)

1. Mise à jour du code sur la branche principale.
2. Construction / mise à jour des images ou services (Django, Next.js, Kestra).
3. Application des migrations de base de données si nécessaire.
4. Vérification rapide des parcours critiques (smoke tests).

Les détails concrets (scripts, commandes) pourront être précisés en fonction de l’infrastructure retenue.

11.6 6. Bonnes pratiques & sécurité

- Ne jamais committer de secrets (mots de passe, tokens) dans le dépôt.
 - Utiliser des **variables d’environnement** et/ou un gestionnaire de secrets.
 - Journaliser les événements importants (erreurs, échecs de jobs Kestra) sans stocker de données sensibles inutiles.
 - Mettre en place des règles minimales de contrôle d’accès :
 - vérification systématique de l’authentification,
 - vérification des droits avant chaque action sensible.
-

11.7 7. Onboarding d’un nouveau membre

Lorsqu’un nouveau membre rejoint le projet :

1. Lecture des documents clés :
 - Vision / objectifs du projet.
 - OBS et mapping OBS ↔ WBS.
 - Choix techniques et architecture.

- Présent document de **Plan Qualité**.
- 2. Mise en place de l'environnement de développement (suivant un guide rapide à rédiger dans le dépôt technique).
- 3. Attribution d'une première tâche simple (correction de bug mineur ou petite fonctionnalité) avec accompagnement.
- 4. Revue de code systématique sur ses premières contributions.

Ce plan qualité pourra être détaillé au fur et à mesure de l'avancement du projet et des retours d'expérience de l'équipe.

12 POPolitics – Definition of Done (DoD)

12.1 1. Objectif

La **Definition of Done (DoD)** définit les critères permettant de considérer qu'une tâche, une fonctionnalité ou une user story est terminée et prête à être livrée.

Elle garantit que :

- toutes les fonctionnalités respectent un niveau de qualité défini ;
- le code est testé, documenté et validé ;
- l'équipe partage une définition commune du travail terminé.

La DoD permet également de maintenir une qualité constante du produit tout au long du projet.

12.2 2. Critères généraux de complétion

Une tâche est considérée comme **terminée** lorsque les conditions suivantes sont respectées.

12.2.1 2.1 Développement terminé

- Le code correspondant à la fonctionnalité est entièrement implémenté.
- Le code respecte les bonnes pratiques de développement définies par l'équipe.
- Le code est structuré et lisible.

12.2.2 2.2 Tests réalisés

Les tests nécessaires ont été réalisés :

- tests unitaires ;
- tests d'intégration (si applicable) ;
- tests manuels de validation.

Les critères suivants doivent être respectés :

- les tests passent **sans erreur** ;
- aucun bug critique n'est détecté.

12.2.3 2.3 Revue de code

Une **code review** a été réalisée par au moins un autre membre de l'équipe. La revue vérifie :

- la qualité du code ;
- la cohérence avec l'architecture ;
- le respect des standards de développement.

Les retours de relecture doivent être **corrigés avant validation**.

12.2.4 2.4 Intégration dans le repository

La fonctionnalité doit être :

- commit dans une branche dédiée ;
- fusionnée dans la branche principale via Pull Request ;
- validée après revue de code.

12.2.5 2.5 Documentation mise à jour

La documentation liée à la fonctionnalité doit être mise à jour si nécessaire :

- README ;
- documentation technique ;
- documentation API ;
- commentaires dans le code.

12.2.6 2.6 Fonctionnalité testée sur l'environnement de développement

La fonctionnalité doit être :

- testée dans l'application ;
- vérifiée pour s'assurer qu'elle fonctionne correctement ;
- validée par l'équipe.

12.2.7 2.7 Issue mise à jour dans l'outil de gestion de projet

Lorsque la tâche est terminée :

- l'issue correspondante est mise à jour ;
- elle est déplacée dans la colonne **Done** du board de gestion de projet ;
- les commentaires nécessaires sont ajoutés (lien vers la PR, notes importantes, etc.).

12.3 3. Critères spécifiques selon le type de tâche

12.3.1 3.1 Fonctionnalité Backend

Pour une fonctionnalité backend :

- l'API fonctionne correctement ;
- les endpoints répondent avec les bons formats et codes de retour ;
- les erreurs sont correctement gérées (messages clairs, statuts HTTP adaptés).

12.3.2 3.2 Fonctionnalité Frontend

Pour une fonctionnalité frontend :

- l'interface fonctionne sans erreur ;
- l'affichage est cohérent avec le design défini (UX/UI) ;
- les interactions utilisateur fonctionnent correctement sur les parcours ciblés.

12.3.3 3.3 Traitement de données / IA

Pour les fonctionnalités liées aux données ou à l'IA :

- les données sont correctement traitées (qualité, complétude minimale) ;
- les résultats produits sont cohérents par rapport aux attentes fonctionnelles ;
- les performances sont acceptables (temps de traitement raisonnable pour l'usage visé).

12.4 4. Validation finale

Une tâche est officiellement considérée comme **terminée** lorsque :

- tous les critères de la Definition of Done sont respectés ;
- la Pull Request est validée ;
- la fonctionnalité est intégrée au projet ;
- l'issue correspondante est marquée comme **Done** dans l'outil de gestion.

12.5 5. Évolution de la Definition of Done

La Definition of Done peut évoluer au cours du projet afin de :

- améliorer la qualité du produit ;
- adapter les processus de développement ;

- intégrer les retours de l'équipe.

Toute modification doit être **discutée et validée collectivement** par l'équipe, puis répercutée dans le présent document.

13 POPolitics – Fonctionnalités

13.1 1. Objectif général

Objectif : rendre le suivi de l'activité parlementaire simple, clair et interactif.

Popolitics centralise les informations issues des institutions politiques et les transforme en insights actionnables pour :

- les citoyens,
- les journalistes,
- les experts politiques.

Valeur ajoutée :

- Données fiables et mises à jour quotidiennement.
- Analyses automatisées via des fonctionnalités IA.
- Expérience utilisateur soignée : interface et visualisations claires et personnalisées.
- Gain de temps : résumés rapides, notifications personnalisées.

Les fonctionnalités présentées restent à préciser, tout comme les technologies et frameworks utilisés : ils seront définis lorsque l'équipe sera au complet.

13.2 2. Portail collectif

Le cœur de la plateforme, pour analyser les votes, tendances et débats.

13.2.1 2.1 Historique et suivi des votes

« Qui vote quoi, en un clin d'œil ! »

- Mise à jour quotidienne automatisée.
- Filtres avancés : thème, période, commission, type de texte, amendement vs vote final.
- Filtres de catégories : groupe politique, élu, rôle (rapporteur, ministre...).
- Codes couleurs intuitifs (Pour / Contre / Abstention / Non-votant).
- Calculs dynamiques : taux de participation, cohésion du groupe, évolution temporelle.
- Export intelligent : PDF, CSV, infographies.

Mockup suggéré : tableau dynamique + graphique d'évolution des votes.

13.2.2 2.2 Analyse des grandes tendances

« Quelles dynamiques politiques en temps réel ? »

- Détection des sujets émergents (NLP + clustering).
- Indice d'opposition entre groupes.
- Cartographie des alliances (ex. LR + RN sur certains textes).
- Heatmaps d'alignement.
- Graphes de centralité politique : « qui influence réellement quoi ? ».

13.2.3 2.3 Synthèses automatiques des débats

« Un résumé flash pour rester connecté·e facilement à l'information politique »

- Résumé en 5 points clés.
- Mise en avant des arguments dominants.
- Détection des “mots forts” et du ton des interventions.
- Accès direct aux passages vidéo liés à chaque point.

Mockup suggéré : bouton pour accéder au débat complet et aux documents sources.

13.2.4 2.4 Indicateurs de cohérence des représentants

« Quel représentant se démarque ? Qui suit le groupe ? »

- Indice de stabilité des positions.
- Alignement au groupe politique (cohésion verticale).
- Analyse des exceptions : votes atypiques, ruptures, évolutions.
- Comparaison entre élus ou groupes.
- Détection des profils atypiques.

Mockup suggéré : jauges et comparateurs par élu et par groupe.

13.2.5 2.5 Visualisations interactives

« Comment rendre l'information politique ludique ? »

- Graphe de réseau politique (similarité des votes).
- Chronologie d'un texte (amendements → passage en commission → vote final).
- Clusters de positionnement idéologique (t-SNE, PCA...).
- Comparateur d'élus ou de groupes.
- Carte thermique des votes par thème.

Mockup suggéré : timeline + graphe de clusters avec zones de couleur.

13.3 3. Page personnelle

Un tableau de bord individuel pour rester informé·e rapidement, selon ses centres d'intérêts.

13.3.1 3.1 Notifications personnalisées

« Impliquer le/la citoyen·ne tout en respectant ses choix »

- Système de priorisation : importance + urgence + intérêt utilisateur.
- Alertes « texte majeur », changement de position d'un élu, amendement surprenant :
 - des alertes qui ne fatiguent pas, en prévenant uniquement quand cela compte vraiment.
- Ajustement dynamique : la plateforme apprend les centres d'intérêt de l'utilisateur.

Mockup suggéré : bannière de notification colorée avec icône de catégorie (économie, social, environnement...).

13.3.2 3.2 Vue simplifiée des actions et décisions des élus suivis

« Qu'est-ce qu'a fait ce représentant ? »

- Timeline des actions de chaque élu : interventions, amendements, rendez-vous publics.
- Profil express : bio, rôle, commissions.

Mockup suggéré : mini-carte avec photo et tendance des votes.

13.3.3 3.3 Fiches éclair express sur les lois, projets et amendements

« Comprendre l'essentiel des textes en moins d'une minute »

- Résumé immédiat : objectif, mécanisme, impacts.
- Analyse des enjeux clés.
- Qui est pour / qui est contre ? Pourquoi ?
- Schéma simplifié du processus législatif du texte.
- « Carte d'influence » : penseurs, lobbies, grandes organisations impliquées.

Mockup suggéré : flashcards interactives avec 3 vues : **Résumé / Enjeux / Acteurs**.

14 POPolitics – Comparaison des stacks techniques

Ce document présente, de manière synthétique, les principales **options techniques étudiées** pour POPolitics, séparées par couche (ETL, Backend, Frontend), et justifie les choix retenus.

14.1 1. ETL & Orchestration

14.1.1 1.1 Kestra vs Airflow

O
p
t
i
o
n

	Avantages principaux	Limites / remarques	Choix retenu
Kestra	Orienté data engineering, workflows déclaratifs en YAML, interface moderne, déploiement simple en Docker, bonne observabilité, outil français	Communauté plus jeune qu’Airflow, moins de documentation historique	Oui – bon compromis pédagogie / simplicité / modernité
Airflow	Standard industriel très répandu, très puissant, riche en opérateurs	Plus lourd à mettre en place et à administrer pour un projet étudiant, courbe d’apprentissage plus importante	Non pour le MVP (envisageable plus tard)

Conclusion : Kestra est choisi pour **orchestrer les pipelines ETL** car il offre un bon équilibre entre puissance, lisibilité et simplicité de déploiement dans une stack Docker multi-conteneurs, tout en étant un **outil français**, cohérent éthiquement avec un projet centré sur la politique française et la souveraineté numérique. Ce choix complète l’architecture **Data Lake (Bronze / Silver / Gold)** décrite dans 01-technological-choices.md en apportant une orchestration moderne et facilement reproductible.

14.2 2. Backend

14.2.1 2.1 Django vs FastAPI

O
p
t
i
o
n

	Avantages principaux	Limites / remarques	Choix retenu
Django	Framework web complet (ORM, auth, admin), écosystème mature, très adapté aux CRUD et aux APIs REST classiques	Plus « lourd » que FastAPI pour de la simple API, courbe d’apprentissage un peu plus large	Oui – cohérent avec le besoin d’un backend complet (Auth Service, Data Service, IA Service)
FastAPI	Très rapide, orienté API, typing	Nécessite d’assembler plus de	Non, réservé

O
p
ti
o
n
a
s
t
A
P
I

Options	Avantages principaux	Limites / remarques	Choix retenu
	moderne, excellent pour des micro-services légers	briques (auth, admin, etc.), moins aligné avec le besoin d'un backend structuré « tout-en-un »	éventuellement à des services spécifiques ultérieurs

Conclusion : Django est retenu comme **socle backend principal** pour POPolitics, afin de bénéficier d'un cadre complet (authentification, ORM, structuration des apps) adapté au MVP et à la taille de l'équipe, et conforme à l'architecture **multi-services** (Data Service, IA Service, Auth Service) décrite dans 01-technological-choices.md.

14.2.2 2.2 Backend Python vs Backend JavaScript

Option	Avantages principaux	Limites / remarques	Choix retenu
Backend Python (Django)	Aligné avec l'écosystème data/IA (ETL en Python, socle IA exposé en FastAPI), forte cohérence technique, mutualisation des compétences	Moins homogène avec le frontend JavaScript, nécessité de maîtriser deux langages sur le projet	Oui – meilleure intégration avec ETL & IA, moins de friction technique globale
Backend JavaScript (Node.js, Express/Nest)	Homogénéité full-JS (front + back), grande communauté, écosystème riche de packages	Moins naturel pour interfacer directement les pipelines ETL Python et les modèles IA Python ; nécessite plus de "colle" entre mondes JS et Python	Non, écarté pour éviter la complexité d'un pont JS ↔ Python permanent

Conclusion : un **backend Python** est privilégié pour rester cohérent avec : - la plateforme data et les pipelines ETL majoritairement en Python ; - le socle IA également en Python (FastAPI) ; ce qui simplifie les échanges entre services et réduit les risques d'incompréhension technique au sein de l'équipe.

14.3 3. Frontend

14.3.1 3.1 Next.js vs SPA React classique

Option	Avantages principaux	Limites / remarques	Choix retenu
Next.js	SSR/SSG, très bonne intégration avec React, BFF intégré (Next Server), gestion des Server Actions, performance et SEO	Architecture un peu plus avancée à appréhender	Oui – idéal pour un front web moderne consommant des APIs REST
React SPA classique	Simplicité de mise en place, très répandu, courbe d'apprentissage déjà connue par beaucoup d'étudiants	Pas de SSR natif, SEO plus faible, logique sensible côté client	Non, moins adapté aux besoins de performance et de sécurité

Conclusion : Next.js est retenu pour bénéficier du **BFF** et du rendu côté serveur, ce qui améliore à la fois la performance perçue, la sécurité (logique côté serveur) et l'expérience utilisateur, en cohérence avec l'objectif de proposer une **interface web performante et sécurisée** détaillé dans 01-technological-choices.md.

14.4 4. IA & NLP

14.4.1 4.1 Framework de modélisation NLP

Option	Avantages principaux	Limites / remarques	Choix retenu
HuggingFace Transformers	Écosystème complet pour le NLP, modèles francophones disponibles (CamemBERT, Mistral, etc.), usage local sans coût ni transfert de données, forte intégration Python	Ressources GPU recommandées pour l'entraînement (gérable en inférence CPU pour le MVP)	Oui – aligné avec les contraintes de budget nul, de souveraineté des données et de cohérence Python
OpenAI API / Claude API	Zéro infrastructure IA, très rapide à intégrer	Coût à l'usage (incompatible avec un budget nul), données envoyées hors infrastructure (souveraineté)	Non, écarté pour des raisons de coût et de souveraineté des données politiques
spaCy	Léger, rapide en production, bon pour la NER et le parsing	Moins puissant pour la classification de textes et la génération de résumés	Non, insuffisant pour les cas d'usage de résumés et de classification

Option	Avantages principaux	Limites / remarques	Choix retenu retenus
--------	----------------------	---------------------	-------------------------

Conclusion : HuggingFace Transformers est retenu comme **socle de modélisation NLP** pour POPolitics, en cohérence avec les cas d’usage définis dans 01-technological-choices.md (section 8.2 : résumés automatiques, classification des textes politiques, analyse de tendances). Il garantit un usage local des modèles francophones, sans coût et sans transfert de données, aligné avec la contrainte de **souveraineté numérique** déjà mise en avant pour le choix de Kestra.

14.4.2 4.2 Exposition du service IA

Option	Avantages principaux	Limites / remarques	Choix retenu
FastAPI (IA Service dédié)	Léger, asynchrone, typage moderne, découplage propre entre IA Service et backend Django	Service supplémentaire à maintenir, géré via Docker Compose	Oui – conforme à l’architecture multi-services décrite dans 01-technological-choices.md
IA embarquée dans Django	Moins de services à orchestrer	Couplage fort entre logique IA et logique métier, contredit l’architecture multi-services	Non, écarté pour préserver la séparation des responsabilités

Conclusion : FastAPI est retenu comme **IA Service dédié**, exposé via une API REST au backend Django. Ce choix acte formellement le découplage décrit dans 01-technological-choices.md (sections 8.3 et 9.1), et lève l’ambiguïté laissée dans ce document quant à la technologie d’exposition des modèles IA.

14.5 5. Synthèse

- **ETL & Data : Kestra** pour l’orchestration des pipelines vers le Data Lake (Bronze / Silver / Gold) et les datamarts PostgreSQL, garantissant traçabilité et reproductibilité des traitements.
- **Backend : Django / Python** comme framework complet pour les services Data, IA et Auth, fortement aligné avec les pipelines ETL et les modèles IA développés en Python.
- **Frontend : Next.js** pour un frontend moderne, performant et sécurisé (BFF + SSR/SSG) offrant une expérience claire et accessible aux utilisateurs finaux.
- **IA & NLP : HuggingFace Transformers** pour la modélisation NLP locale (CamemBERT, Mistral), exposé via un **FastAPI IA Service** dédié consommé par le backend Django.

Ces choix prolongent les principes exposés dans 01-technological-choices.md : **séparation claire des responsabilités**, scalabilité progressive (MVP local → cloud), et alignement avec les bonnes pratiques de data engineering et de développement web modernes.

15 POPolitics – Ressources matérielles & coûts

Ce document complète les choix techniques de POPolitics en détaillant les ressources matérielles nécessaires, les coûts estimés et la logique d'évolution de l'infrastructure : - MVP en local via Docker sur machine de développement - V1 sur serveur local (on-premise léger) - Version Cloud managée

15.1 1. MVP local – Docker sur machine de développement

- **Contexte** : exécution en local sur la machine d'un membre de l'équipe (ou d'une machine partagée ponctuellement) avec l'ensemble des services Docker (Django, Next.js, PostgreSQL, Kestra, etc.).
 - **Configuration cible** :
 - CPU : 4 vCPU
 - RAM : 16 Go
 - Stockage : 256–512 Go SSD
 - **Coût estimatif** :
 - Utilisation d'un laptop/PC déjà existant → **0 € de coût supplémentaire** pour le projet.
 - **Justification** :
 - Suffisant pour développer, lancer les containers Docker et exécuter les premiers pipelines ETL et modèles IA sur des volumes réduits.
 - Permet de démarrer le projet sans frais, en cohérence avec la contrainte « budget nul au départ ».
-

15.2 2. V1 – Serveur local (on-premise léger)

- **Contexte** : hébergement sur un petit serveur local ou une machine dédiée dans l'école / l'organisation, toujours via Docker Compose.
- **Configuration cible** :
 - CPU : 4–8 vCPU
 - RAM : 16–32 Go
 - Stockage : 512 Go–1 To SSD (pour Data Lake + datamarts + logs)
- **Coût estimatif** :

- Réutilisation d'un serveur existant → **0–200 €** (maintenance mineure, disques supplémentaires éventuels).
 - Achat d'un serveur d'entrée de gamme → **800–1 200 €** (one-shot, amortissable sur plusieurs projets).
 - **Justification :**
 - Offre un environnement plus stable pour les démos internes et tests d'intégration.
 - Permet d'augmenter la volumétrie manipulée sans saturer les machines personnelles.
-

15.3 3. Version Cloud – Hébergement managé

- **Contexte** : déploiement sur un fournisseur Cloud (type AWS, GCP, Azure ou équivalent souverain) avec une machine virtuelle ou un petit cluster de services managés.
 - **Configuration cible (exemple VM unique) :**
 - CPU : 4 vCPU
 - RAM : 16 Go
 - Stockage : 200–500 Go SSD (Data Lake et bases de données sur volumes attachés)
 - **Coût estimatif** (ordre de grandeur) :
 - VM 4 vCPU / 16 Go RAM : **40–80 €/mois** selon le fournisseur et la région.
 - Stockage : **5–20 €/mois** pour 200–500 Go.
 - **Justification :**
 - Permet une haute disponibilité, un accès distant pour tous les membres de l'équipe et une montée en charge progressive.
 - Facilite l'exposition sécurisée de l'application à des bêta-testeurs externes et son passage à l'échelle.
-

15.4 4. Cohérence avec les choix techniques

L'architecture et la stack retenues dans 01-technological-choices.md (Docker, Django, Next.js, PostgreSQL, Kestra, Data Lake Bronze/Silver/Gold) restent identiques dans ces trois scénarios : - on ne change que le niveau d'infrastructure (machine locale, serveur local, Cloud), - ce qui limite les coûts de migration entre MVP local, V1 sur serveur local et future version Cloud.

16 POPolitics – User Stories

16.1 1. Introduction

Ce document liste les **user stories** du projet POPolitics, organisées par **Epic** (grand domaine fonctionnel).

Chaque user story suit le format Scrum standard :

En tant que [persona], **je veux** [action] **afin de** [bénéfice].

Chaque story est accompagnée de :

- sa **priorité MoSCoW** (Must / Should / Could),
- ses **critères d'acceptation** (conditions à remplir pour considérer la story comme Done).

Note : les story points ne sont pas pré-assignés ici. Conformément au processus Scrum, ils sont **estimés collectivement par l'équipe lors des séances d'affinage du backlog** (voir [Méthodologie](#)).

16.2 2. Personas

Les personas détaillés (profil fictif complet, objectifs, frustrations, scénarios d'usage) sont documentés dans [00-personas.md](#).

Persona	Description	Profil fictif
Citoyen	Utilisateur grand public souhaitant suivre l'activité politique de ses représentants.	Marie Fontaine
Journaliste / Expert	Utilisateur averti cherchant à analyser des données politiques en profondeur.	Thomas Marchand
Administrateur	Membre de l'équipe POPolitics gérant la plateforme et les données.	Lucas Perrin

16.3 3. Épics

- [Epic 1 – Authentification & Gestion de compte](#)
- [Epic 2 – Portail collectif : Votes](#)
- [Epic 3 – Portail collectif : Tendances & Analyses](#)

- Epic 4 – Portail collectif : Synthèses IA
 - Epic 5 – Page personnelle : Suivi & Notifications
 - Epic 6 – Plateforme Data & ETL
-

16.4 Epic 1 – Authentification & Gestion de compte

16.4.1 US-101 – Créer un compte

En tant que citoyen, **je veux** créer un compte sur POPolitics **afin de** bénéficier d'une expérience personnalisée et accéder aux fonctionnalités protégées.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux renseigner un email et un mot de passe pour créer mon compte.
 - Le mot de passe est soumis à une règle de complexité minimale (8 caractères, majuscule, chiffre).
 - Un message de confirmation est affiché après création réussie.
 - Il m'est impossible de créer deux comptes avec le même email.
-

16.4.2 US-102 – Se connecter

En tant que citoyen, **je veux** me connecter avec mes identifiants **afin d'** accéder à mon espace personnel et aux pages protégées.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux me connecter avec email + mot de passe.
 - Une erreur claire s'affiche si les identifiants sont incorrects.
 - Je suis redirigé vers le Portail collectif après connexion réussie.
 - Les pages protégées sont inaccessibles sans être connecté.
-

16.4.3 US-103 – Se déconnecter

En tant que citoyen, **je veux** me déconnecter de la plateforme **afin de** sécuriser mon compte lorsque je quitte l'application.

- **Priorité** : Must
- **Story points** : *À estimer lors de l'affinage du backlog*
- **Critères d'acceptation** :

- Un bouton de déconnexion est accessible depuis toutes les pages authentifiées.
 - Après déconnexion, je suis redirigé vers la page de connexion.
 - Ma session est bien invalidée côté serveur.
-

16.4.4 US-104 – Réinitialiser son mot de passe

En tant que citoyen, **je veux** pouvoir réinitialiser mon mot de passe oublié **afin de** retrouver l'accès à mon compte.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux saisir mon email pour recevoir un lien de réinitialisation.
 - Le lien de réinitialisation expire après 1 heure.
 - Je peux définir un nouveau mot de passe via ce lien.
-

16.5 Epic 2 – Portail collectif : Votes

16.5.1 US-201 – Consulter le tableau des votes

En tant que citoyen, **je veux** consulter un tableau des votes des élus **afin de** savoir comment mes représentants votent sur les textes de loi.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Le tableau affiche pour chaque vote : l' élu, le texte, l'institution (AN / Sénat / UE), la date et la position (Pour / Contre / Abstention / Non-votant).
 - Les données sont mises à jour au moins une fois par jour.
 - Un code couleur distinct est appliqué à chaque position.
-

16.5.2 US-202 – Filtrer les votes

En tant que citoyen, **je veux** filtrer les votes par institution, période, groupe politique et élu **afin de** retrouver rapidement les informations qui m'intéressent.

- **Priorité** : Must
- **Story points** : *À estimer lors de l'affinage du backlog*
- **Critères d'acceptation** :
 - Les filtres disponibles sont : institution, période (date début / fin), groupe politique, nom de l' élu.

- Les résultats se mettent à jour dynamiquement lors de l'application des filtres.
 - Il est possible de réinitialiser tous les filtres en un clic.
-

16.5.3 US-203 – Filtrer les votes par thème, commission et type de texte

En tant que journaliste, **je veux** filtrer les votes par thème, commission et type de texte **afin d'** affiner mes analyses sur un sujet précis.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Les filtres thème, commission et type de texte (amendement vs vote final) sont disponibles en complément des filtres de base.
 - Les filtres sont combinables entre eux.
-

16.5.4 US-204 – Consulter les indicateurs d'un vote

En tant que citoyen, **je veux** voir le taux de participation et la cohésion du groupe pour un vote donné **afin de** comprendre la dynamique politique derrière le résultat.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Le taux de participation (% de votants) est affiché pour chaque vote.
 - Un indice de cohésion de groupe (% de membres ayant voté dans le même sens) est calculé et affiché.
-

16.5.5 US-205 – Exporter les résultats de vote

En tant que journaliste, **je veux** exporter les données de vote au format CSV ou PDF **afin de** les réutiliser dans mes articles et analyses.

- **Priorité** : Could
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Un bouton d'export CSV et PDF est disponible sur le tableau des votes.
 - L'export respecte les filtres actifs au moment du téléchargement.
 - Le fichier exporté contient les colonnes : élu, texte, institution, date, position.
-

16.6 Epic 3 – Portail collectif : Tendances & Analyses

16.6.1 US-301 – Visualiser les indicateurs de cohérence d'un élu

En tant que citoyen, **je veux** visualiser l'indice de cohérence d'un élu avec son groupe politique **afin de** savoir s'il suit les positions de son parti ou s'en écarte.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Une jauge ou score de cohérence est affiché sur la fiche d'un élu.
 - Les votes atypiques (où l'élu a voté contre son groupe) sont mis en évidence.
-

16.6.2 US-302 – Comparer deux élus ou deux groupes

En tant que journaliste, **je veux** comparer les positions de deux élus ou de deux groupes politiques **afin d'** identifier leurs convergences et divergences.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux sélectionner deux élus ou deux groupes pour lancer une comparaison.
 - Une vue côte-à-côte affiche leurs positions respectives sur les mêmes textes.
 - Un score de similarité globale est calculé et affiché.
-

16.6.3 US-303 – Visualiser la cartographie des alliances

En tant que journaliste, **je veux** voir une cartographie des alliances entre groupes politiques **afin d'** identifier quels groupes votent ensemble et sur quels sujets.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Un graphe de réseau affiche les groupes politiques et les liens entre eux pondérés par la fréquence d'alignement.
 - Il est possible de filtrer la cartographie par institution ou par thème.
-

16.6.4 US-304 – Croiser les données AN / Sénat / UE

En tant que expert, **je veux** croiser les données de vote entre l'Assemblée nationale, le Sénat et le Parlement européen **afin d'** analyser la cohérence des positions politiques entre institutions.

- **Priorité** : Should
 - **Story points** : À estimer lors de l’affinage du backlog
 - **Critères d’acceptation** :
 - Le datamart Cross-match est disponible et interrogeable via l’interface.
 - Je peux filtrer les résultats par élu ou groupe pour voir ses positions sur plusieurs institutions.
-

16.7 Epic 4 – Portail collectif : Synthèses IA

16.7.1 US-401 – Obtenir un résumé automatique d’un débat

En tant que citoyen, **je veux** obtenir un résumé automatique d’un débat parlementaire **afin de** comprendre l’essentiel en moins d’une minute sans lire le transcript complet.

- **Priorité** : Should
 - **Story points** : À estimer lors de l’affinage du backlog
 - **Critères d’acceptation** :
 - Un bouton “Résumé IA” est accessible sur la page d’un débat.
 - Le résumé est présenté en 5 points clés maximum.
 - Le temps de génération est inférieur à 5 secondes.
 - Un lien vers le transcript complet est proposé sous le résumé.
-

16.7.2 US-402 – Consulter une fiche éclair sur un texte de loi

En tant que citoyen, **je veux** accéder à une fiche éclair sur un projet de loi ou amendement **afin de** comprendre son objectif, ses enjeux et les positions des acteurs en quelques secondes.

- **Priorité** : Should
 - **Story points** : À estimer lors de l’affinage du backlog
 - **Critères d’acceptation** :
 - La fiche présente 3 vues : Résumé / Enjeux / Acteurs.
 - La section Acteurs indique qui est pour et qui est contre, avec une courte justification.
 - La fiche est accessible depuis le tableau des votes.
-

16.8 Epic 5 – Page personnelle : Suivi & Notifications

16.8.1 US-501 – Suivre des élus

En tant que citoyen, **je veux** sélectionner des élus à suivre **afin d'** être informé de leurs actions et votes sans avoir à les chercher à chaque visite.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je peux rechercher un élu par nom et l'ajouter à ma liste de suivi.
 - Ma liste de suivi est persistante entre les sessions.
 - Je peux retirer un élu de ma liste de suivi à tout moment.
-

16.8.2 US-502 – Consulter la timeline d'un élu suivi

En tant que citoyen, **je veux** consulter la timeline des actions d'un élu que je suis **afin de** voir d'un coup d'œil ses votes récents, interventions et amendements.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - La timeline affiche les événements des élus suivis dans l'ordre chronologique inverse.
 - Chaque événement indique le type (vote, intervention, amendement), la date et un bref libellé.
 - Je peux cliquer sur un événement pour accéder au détail.
-

16.8.3 US-503 – Configurer ses thèmes d'intérêt

En tant que citoyen, **je veux** choisir des thèmes politiques qui m'intéressent (économie, environnement, social...) **afin de** recevoir uniquement les alertes pertinentes pour moi.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Une liste de thèmes prédéfinis est proposée lors du paramétrage du profil.
 - Je peux en sélectionner plusieurs.
 - Mes préférences de thèmes peuvent être modifiées à tout moment depuis mon profil.
-

16.8.4 US-504 – Recevoir des notifications personnalisées

En tant que citoyen, **je veux** recevoir des notifications sur les événements importants liés à mes élus suivis et à mes thèmes d'intérêt **afin d'** être informé sans être submergé d'alertes.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Je ne reçois des notifications que pour les événements liés à mes élus suivis ou à mes thèmes d'intérêt.
 - Chaque notification affiche un libellé clair et un lien direct vers la page concernée.
 - Je peux désactiver certaines catégories de notifications depuis mes paramètres.
-

16.9 Epic 6 – Plateforme Data & ETL

16.9.1 US-601 – Ingérer les données de l'Assemblée nationale

En tant qu' administrateur, **je veux** que les données de vote de l'AN soient ingérées automatiquement **afin de** disposer d'une base à jour sans intervention manuelle.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l'affinage du backlog*
 - **Critères d'acceptation** :
 - Le pipeline ETL AN s'exécute automatiquement selon une fréquence définie (quotidienne minimum).
 - Les données brutes sont stockées en zone Bronze.
 - Les exécutions sont tracées (logs, statut, horodatage) dans Kestra.
 - En cas d'échec, une alerte est émise et le job peut être relancé manuellement.
-

16.9.2 US-602 – Ingérer les données du Sénat

En tant qu' administrateur, **je veux** que les données de vote du Sénat soient ingérées automatiquement **afin de** couvrir les deux chambres du Parlement français.

- **Priorité** : Must
- **Story points** : *À estimer lors de l'affinage du backlog*
- **Critères d'acceptation** :
 - Identiques à US-601, appliquées à la source Sénat.

- Les données Sénat sont stockées dans un datamart PostgreSQL distinct et documenté.
-

16.9.3 US-603 – Ingérer les données du Parlement européen

En tant qu’ administrateur, **je veux** que les données du Parlement européen soient ingérées automatiquement **afin d’** offrir une dimension européenne à l’analyse.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - Identiques à US-601, appliquées à la source UE.
 - Les données UE sont stockées dans un datamart PostgreSQL distinct et documenté.
-

16.9.4 US-604 – Normaliser les données en zone Silver

En tant qu’ administrateur, **je veux** que les données brutes soient nettoyées et normalisées en zone Silver **afin de** disposer d’une base cohérente et exploitable par le backend.

- **Priorité** : Must
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - Les données Silver respectent un schéma normalisé documenté.
 - Les doublons et données incohérentes sont traités et tracés.
 - La transformation Bronze → Silver est reproductible (mêmes entrées → mêmes sorties).
-

16.9.5 US-605 – Préparer les données analytiques en zone Gold

En tant qu’ administrateur, **je veux** que des données agrégées et prêtes à l’analyse soient disponibles en zone Gold **afin de** réduire les temps de requête du backend et enrichir les indicateurs.

- **Priorité** : Should
 - **Story points** : *À estimer lors de l’affinage du backlog*
 - **Critères d’acceptation** :
 - La zone Gold contient des tables pré-agrégées (indicateurs, KPI) consommables directement par les APIs.
 - La transformation Silver → Gold est orchestrée par Kestra.
-

16.10 4. Récapitulatif

Les story points sont laissés vides et seront renseignés lors des séances d'affinage du backlog.

ID	Titre court	Epic	Priorité	Points
US-101	Créer un compte	Auth	Must	
US-102	Se connecter	Auth	Must	
US-103	Se déconnecter	Auth	Must	
US-104	Réinitialiser mot de passe	Auth	Should	
US-201	Consulter tableau des votes	Votes	Must	
US-202	Filtrer les votes (base)	Votes	Must	
US-203	Filtrer les votes (avancé)	Votes	Should	
US-204	Indicateurs d'un vote	Votes	Should	
US-205	Exporter les votes	Votes	Could	
US-301	Cohérence d'un élu	Tendances	Should	
US-302	Comparer élus / groupes	Tendances	Should	
US-303	Cartographie des alliances	Tendances	Should	
US-304	Croisement AN / Sénat / UE	Tendances	Should	
US-401	Résumé IA d'un débat	IA	Should	
US-402	Fiche éclair sur un texte	IA	Should	
US-501	Suivre des élus	Page perso	Should	
US-502	Timeline d'un élu suivi	Page perso	Should	
US-503	Configurer thèmes d'intérêt	Page perso	Should	
US-504	Notifications personnalisées	Page perso	Should	
US-601	Ingestion données AN	Data	Must	
US-602	Ingestion données Sénat	Data	Must	
US-603	Ingestion données UE	Data	Must	
US-604	Normalisation zone Silver	Data	Must	
US-605	Agrégation zone Gold	Data	Should	

17 POPolitics - Plan de Sprints

17.1 1. Introduction

Ce document décrit le **decoupage du projet en sprints**, aligne sur le rythme d'alternance Master (voir [Methodologie](#)).

17.1.1 Principes

- Les sprints ont une **duree fixe de 9 semaines**.
- Le planning démarre le **21/09/2026** et se termine le **25/06/2027**.
- Les semaines de presentiel sont integrees dans les sprints en cours.
- Le dernier sprint peut etre plus court que 9 semaines pour s'aligner sur la date de fin d'annee.
- Le contenu exact de chaque sprint (user stories, story points) est valide lors du Sprint Planning et affine lors des seances d'affinage du backlog.

17.2 2. Vue d'ensemble

Periode	Dates	Duree	Objectif global
Sprint 1	21/09/2026 -> 20/11/2026	9 semaines	Fondations (Auth + ETL)
Sprint 2	23/11/2026 -> 22/01/2027	9 semaines	MVP Portail collectif
Sprint 3	25/01/2027 -> 26/03/2027	9 semaines	Enrichissements (tendances + page perso)
Sprint 4	29/03/2027 -> 28/05/2027	9 semaines	IA + finitions
Sprint 5	31/05/2027 -> 25/06/2027	4 semaines	Stabilisation finale + soutenance

17.3 3. Detail des sprints

17.3.1 Sprint 1 - Fondations (Must)

Duree	9 semaines
Dates	21/09/2026 -> 20/11/2026
Sprint Goal	Poser les fondations techniques: authentication operationnelle et pipelines ETL de base.

User stories cibles :

ID	Titre	Priorite
US-101	Creer un compte	Must
US-102	Se connecter	Must
US-103	Se deconnecter	Must

ID	Titre	Priorite
US-601	Ingestion donnees AN	Must
US-602	Ingestion donnees Senat	Must
US-603	Ingestion donnees UE	Must
US-604	Normalisation zone Silver	Must

17.3.2 Sprint 2 - MVP Portail collectif

Duree	9 semaines
Dates	23/11/2026 -> 22/01/2027
Sprint Goal	Livrer un portail collectif utilisable avec donnees reelles, filtres et premiers indicateurs.

User stories cibles :

ID	Titre	Priorite
US-201	Consulter tableau des votes	Must
US-202	Filtrer les votes (base)	Must
US-605	Agregation zone Gold	Should
US-203	Filtrer les votes (avance)	Should
US-204	Indicateurs d'un vote	Should

17.3.3 Sprint 3 - Enrichissements (Should)

Duree	9 semaines
Dates	25/01/2027 -> 26/03/2027
Sprint Goal	Ajouter les fonctionnalites d'analyse politique et les usages personnalisés.

User stories cibles :

ID	Titre	Priorite
US-301	Coherence d'un élu	Should
US-302	Comparer élus / groupes	Should
US-304	Croisement AN / Senat / UE	Should
US-501	Suivre des élus	Should
US-502	Timeline d'un élu suivi	Should
US-503	Configurer themes d'interet	Should

ID	Titre	Priorite
US-504	Notifications personnalisées	Should

17.3.4 Sprint 4 - IA et finitions (Should/Could)

Duree	9 semaines
Dates	29/03/2027 -> 28/05/2027
Sprint Goal	Integrer la couche IA, finaliser les visualisations avancees et traiter les finitions.

User stories cibles :

ID	Titre	Priorite
US-401	Resume IA d'un debat	Should
US-402	Fiche éclair sur un texte	Should
US-303	Cartographie des alliances	Should
US-104	Reinitialiser mot de passe	Should
US-205	Exporter les votes	Could

17.4 4. Sprint 5 - Stabilisation & soutenance

Duree	4 semaines
Dates	31/05/2027 -> 25/06/2027
Objectif	Stabiliser, corriger, preparer la soutenance et finaliser les livrables academiques.

Travaux prevus :

- Corrections de bugs transverses.
- Tests de bout en bout sur les parcours critiques.
- Preparation du scenario de demonstration.
- Mise a jour finale de la documentation technique.
- Preparation des slides de soutenance.

17.5 5. Recapitulatif

Periode	Duree	Stories Must	Stories Should	Stories Could
Sprint 1	9 sem.	7	0	0
Sprint 2	9 sem.	2	3	0

Periode	Duree	Stories Must	Stories Should	Stories Could
Sprint 3	9 sem.	0	7	0
Sprint 4	9 sem.	0	4	1
Sprint 5	4 sem.	-	-	-

Total stories Must Have couvertes : 9 / 9

Total stories Should Have couvertes : 14 / 14

Total stories Could Have couvertes : 1 / 1

18 POPolitics – Registre des risques

18.1 1. Méthode d'évaluation

Chaque risque est évalué selon deux axes :

- **Probabilité** : Faible (1) / Moyen (2) / Élevé (3)
- **Impact** : Faible (1) / Moyen (2) / Élevé (3)
- **Criticité** = Probabilité × Impact → score de 1 à 9

Criticité	Niveau
1 – 2	Faible
3 – 4	Moyen
6 – 9	Critique

18.2 2. Registre des risques

18.2.1 Catégorie : Données & Sources externes

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-01	Change ment de format ou d'URL d'une API source (AN, Sénat, UE) sans préavis	2	3	6	Version ner les connecteurs ETL ; surveiller les changelogs des portails open data ; couche	Data Engineer

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
					Bronze comme filet de sécurité	
R-02	Qualité des données insuffisante (doublets, valeurs manquantes, incohérences entre sources)	3	2	6	Tests de qualité systématiques en phase Silver ; logs de rejet explicites dans Kestra ; validation d'un échantillon à chaque ingestion	Data Engineer
R-03	Indisponibilité temporaire d'une source (maintenance portail, panne)	2	2	4	Gestion des runs vides dans Kestra (no-op) ; retry automatique ; stocker les dernières données	Data Engineer

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
					valides en Bronze	
R-04	Absence d'identification commune entre AN, Sénat et UE pour le cross-match	3	2	6	Table de correspondance manuelle (nom + prénom + date de naissance) ; documenter les cas ambigus ; limiter le cross-match aux élus avec mandat simultané	Data Engineer
R-05	Volume de données plus important que prévu (saturation disque)	1	2	2	Surveiller la taille des données dès le Sprint 1 ; prévoir un	DevOps

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	ou mémoire)				quota disque minimum (50 Go) sur la VM de démo	

18.2.2 Catégorie : Technique & Architecture

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-06	Complexité de la stack locale (7-8 conteneurs Docker) bloquant le démarrage d'un membre	2	2	4	Guide de démarrage rapide obligatoire dès le Sprint 0 ; critère de fin de Sprint 0 : stack opérationnelle en < 30 min pour chacun	Tech Lead
R-07	Couche IA (Sprint 9) sous-estimée —	3	3	9	Réduire le périmètre IA au strict minimum	Tech Lead / IA

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	modèle non livrable dans les temps				m (résumé automatique uniquement) ; prévoir un fallback sans IA si nécessaire ; anticiper en Sprint 8	
R-08	Divergence des schémas de données entre AN et Sénat rendant la normalisation Silver complexe	3	2	6	Documenter les deux schémas dès le Sprint 1 ; définir le schéma unifié avant le Sprint 2 ; tester la couche Silver avec données réelles	Data Engineer
R-09	Régression	2	3	6	Tests unitaire	Backend / Data

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	backen d ou ETL non détecté e faute de tests automa tisés suffisan ts				s sur les transfor mations critique s ; tests d'intégr ation backen d ↔ datama rts ; smoke tests avant chaque démon	
R-10	Perform ance insuffis ante des requête s Postgre SQL sur de gros volume s	2	2	4	Indexer les colonne s clés dès la créatio n des datama rts ; surveill er les temps de réponse des endpoin ts ; optimis er en Gold si nécess aire	Backen d / Data

18.2.3 Catégorie : Organisation & Équipe

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-11	Manque de disponibilité de membres en période de distanciel (alternance, charge entreprise)	3	2	6	Sprints de 2 semaines avec objectifs réalistes ; revue de charge lors du Sprint Planning ; périmètre ajustable (Should → Could)	Scrum Master / PO
R-12	Départ ou indisponibilité prolongée d'un membre clé (maladie, abandon)	1	3	3	Documentation continue des choix et décisions ; pas de compétence en silo ; pair programming encouragé	Tech Lead

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-13	Mauvaise synchronisation entre équipes data, backend et frontend (dépendances bloquantes)	2	3	6	Définir les contrats d'API et les schémas de données avant le début du Sprint concerné ; daily ou point de synchro hebdo madair e	Scrum Master
R-14	Sous-estimation générale de la charge sur un sprint	2	2	4	Vélocité suivie sprint par sprint ; affinage du backlog régulier ; règle : mieux vaut finir moins et bien que livrer un sprint instable	PO / Scrum Master

18.2.4 Catégorie : Périmètre & Livraison

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
R-15	Périmètre Must Have non livré avant la soutenance	1	3	3	Prioriser strictement le Must Have ; bloquer toute fonctionnalité Should si un Must est en retard	PO
R-16	Scénario de démonstration non préparé ou instable le jour J	2	3	6	Dédier le Sprint 11 à la stabilisation et à la préparation de la démo ; jeu de données de démonstration figé et testé à l'avance	Tech Lead / PO
R-17	Documentation technique	2	2	4	Mise à jour des docs incluse dans la	Toute l'équipe

ID	Risque	Probabilité	Impact	Criticité	Mitigation	Responsable
	incomplète ou désynchronisé avec le code				Definition of Done ; revue documentation lors du Sprint 11	

18.3 3. Récapitulatif par criticité

18.3.1 Critiques (score 6–9)

ID	Risque	Score
R-07	Couche IA sous-estimée	9
R-01	Changement d'API source	6
R-02	Qualité des données insuffisante	6
R-04	Pas d'identifiant commun cross-match	6
R-08	Divergence schémas AN / Sénat	6
R-09	Régressions non détectées	6
R-11	Disponibilité en distanciel	6
R-13	Désynchronisation équipes	6
R-16	Démo instable le jour J	6

18.3.2 Moyens (score 3–4)

ID	Risque	Score
R-06	Stack locale bloquante	4
R-10	Performance PostgreSQL	4
R-14	Sous-estimation de charge	4
R-17	Documentation désynchronisée	4
R-12	Départ d'un membre clé	3
R-15	Must Have non livré	3

18.3.3 Faibles (score 1–2)

ID	Risque	Score
----	--------	-------

ID	Risque	Score
R-05	Volume de données excessif	2

18.4 4. Suivi

Ce registre est à réviser à **chaque Sprint Review**. Pour chaque risque critique actif, une ligne de suivi doit être ajoutée lors de la révision :

ID	Date révision	Statut	Évolution
----	---------------	--------	-----------

Statuts possibles : Actif / Atténué / Clos / Matérialisé

19 POPolitics – Exigences non-fonctionnelles (NFR)

19.1 1. Introduction

Les exigences non-fonctionnelles définissent **comment** le système doit se comporter, indépendamment des fonctionnalités métier. Elles s'appliquent à l'ensemble de la stack : pipeline ETL, backend Django, frontend Next.js et infrastructure Docker.

19.2 2. Performance

19.2.1 2.1 Temps de réponse API

Endpoint	Cible MVP	Cible V1
Consultation tableau des votes (filtre simple)	< 2 s	< 1 s
Consultation tableau des votes (filtre avancé / cross-match)	< 5 s	< 2 s
Page personnelle (timeline d'un élu suivi)	< 3 s	< 1,5 s
Authentification (login)	< 1 s	< 500 ms
Endpoint IA (résumé automatique)	< 10 s	< 5 s

19.2.2 2.2 Pipeline ETL

- Ingestion complète AN + Sénat + UE : **< 2 heures** pour un run journalier complet.
- Ingestion différentielle (données du jour uniquement) : **< 30 minutes**.

- Kestra doit loguer le temps d'exécution de chaque step pour permettre l'identification des goulots d'étranglement.

19.2.3 2.3 Chargement frontend

- **LCP (Largest Contentful Paint)** : < 2,5 s sur connexion standard.
- **TTI (Time to Interactive)** : < 4 s.
- Les pages lourdes (tableaux, graphiques) doivent implémenter une **pagination** ou un **chargement progressif** pour ne jamais dépasser 500 lignes chargées en une seule requête.

19.3 3. Scalabilité

19.3.1 3.1 Charge utilisateurs

Phase	Utilisateurs simultanés cibles
MVP local (démonstration)	5–10
V1 serveur local	50–100
Version Cloud	500+

19.3.2 3.2 Volume de données

Source	Volume estimé (initial)	Croissance annuelle
Assemblée nationale	~5 Go (historique depuis 2007)	~500 Mo/an
Sénat	~3 Go (historique depuis 2010)	~300 Mo/an
Parlement européen	~2 Go (historique depuis 2004)	~200 Mo/an
Total	~10 Go	~1 Go/an

- L'architecture Bronze/Silver/Gold doit rester fonctionnelle jusqu'à **100 Go** sans modification structurelle.
- Le passage à un stockage Cloud (S3 ou équivalent) doit être possible sans refonte du pipeline ETL.

19.4 4. Disponibilité & fiabilité

19.4.1 4.1 Disponibilité cible

Environnement	Disponibilité cible
MVP local (démonstration)	Best-effort (pas de SLA)
V1 serveur local	95 % (tolérance maintenance planifiée)
Version Cloud	99 %

19.4.2 4.2 Gestion des pannes

- Un job ETL en échec ne doit **pas bloquer les autres jobs** , isolation par pipeline dans Kestra.
- En cas d'indisponibilité d'une source externe, le système doit servir les **dernières données valides** sans erreur visible pour l'utilisateur.
- Le backend Django doit retourner des **codes HTTP explicites** (404, 503, etc.) et jamais exposer de stack trace en production.

19.4.3 4.3 Reprise après incident

- Temps de redémarrage de la stack complète (Docker Compose) : **< 5 minutes**.
 - Les pipelines Kestra doivent supporter le **rejeu manuel** d'un run en échec sans duplication de données.
-

19.5 5. Sécurité

19.5.1 5.1 Authentification & autorisation

- Authentification via **JWT** ou session sécurisée (HttpOnly cookie).
- Toute route API Django doit vérifier l'authentification **avant** tout traitement.
- Les rôles (utilisateur standard, administrateur) doivent être vérifiés côté serveur, jamais côté client seul.
- Les tokens JWT doivent avoir une **durée de vie courte** (< 1 heure) avec mécanisme de refresh.

19.5.2 5.2 Protection des données

- Aucun secret (clés API, mots de passe, tokens) ne doit être commité dans le dépôt Git.
- Les secrets sont gérés via **variables d'environnement** et/ou fichier .env non versionné.
- Les mots de passe utilisateurs sont stockés **hashés** (bcrypt ou argon2), jamais en clair.

19.5.3 5.3 Sécurité des échanges

- Toutes les communications entre frontend et backend doivent transiter en **HTTPS** (dès la V1 serveur).
- Les APIs doivent implémenter une protection **CORS** explicite (whitelist des origines autorisées).
- Protection contre les injections SQL via l'ORM Django — pas de requêtes SQL concaténées manuellement.

19.5.4 5.4 Données personnelles (RGPD)

- Les données traitées sont des **données publiques d'élus** dans l'exercice de leur mandat, pas de contrainte RGPD spécifique sur ces données.
 - Les données des **utilisateurs inscrits** (email, préférences) sont soumises au RGPD :
 - Consentement explicite à l'inscription.
 - Possibilité de suppression du compte et des données associées.
 - Pas de partage avec des tiers.
-

19.6 6. Maintenabilité

19.6.1 6.1 Lisibilité du code

- Couverture de tests unitaires sur les fonctions critiques : **> 60 %** (cible MVP).
- Chaque pipeline Kestra doit être nommé explicitement et documenté en en-tête YAML.
- Pas de valeurs en dur (*hardcoded*) dans les scripts ETL, utiliser des variables de configuration.

19.6.2 6.2 Traçabilité

- Chaque run ETL doit produire un log structuré contenant : date, source, nombre de lignes ingérées, nombre de rejets, statut final.
- Les erreurs backend doivent être loguées avec leur contexte (endpoint, paramètres, timestamp), sans données sensibles.

19.6.3 6.3 Évolutivité de l'architecture

- L'ajout d'une nouvelle source de données (ex. conseils régionaux) doit être possible **sans modifier** les couches Silver/Gold existantes.
 - L'ajout d'un nouveau modèle IA doit se faire via un nouveau service indépendant, sans modifier le backend Django existant.
-

19.7 7. Compatibilité

19.7.1 7.1 Navigateurs supportés

Navigateur	Support
Chrome / Chromium (dernière version)	Complet
Firefox (dernière version)	Complet
Safari (dernière version)	Complet
Edge (dernière version)	Complet
Internet Explorer	Non supporté

19.7.2 7.2 Résolutions

- L'interface doit être utilisable sur **desktop** (largeur ≥ 1024 px) en priorité.
- Une adaptation **tablette** (≥ 768 px) est un objectif Should Have.
- Le support mobile n'est pas dans le périmètre du MVP.

19.8 8. Contraintes d'infrastructure (MVP)

En cohérence avec [08-projet-ressources.md](#) :

- Stack complète opérationnelle sur une machine **4 vCPU / 16 Go RAM / 256 Go SSD**.
- Démarrage de la stack via docker-compose up en **< 5 minutes** sur la machine de démo.
- Aucun service cloud payant requis pour le MVP — tout doit fonctionner **offline** (sauf les appels aux sources externes lors des runs ETL).

20 POPolitics – Sources de données

20.1 1. Vue d'ensemble

POPolitics s'appuie exclusivement sur des **données publiques ouvertes** issues de trois institutions :

Institution	Portail open data	Licence
Assemblée nationale	data.assemblee-nationale.fr	Licence Ouverte Etalab 2.0
Sénat	data.senat.fr	Licence Ouverte Etalab 2.0
Parlement européen	data.europarl.europa.eu	Licence CC BY 4.0

Toutes les sources sont **gratuites**, sans clé d'API requise pour l'accès de base.

20.2 2. Assemblée nationale

20.2.1 2.1 Portail

- URL : <https://data.assemblee-nationale.fr>
- API REST (Recherche AN) : <https://recherche.assemblee-nationale.fr/api>

20.2.2 2.2 Données disponibles

Dataset	Description	Format
Scrutins / votes	Résultats des votes en séance publique	JSON, XML

Dataset	Description	Format
Députés	Identité, groupe politique, circonscription, mandats	JSON, XML
Amendements	Dépôt, sort, auteur, texte de référence	JSON, XML
Dossiers législatifs	Textes de loi, étapes, commissions	JSON, XML
Agenda	Ordre du jour des séances plénières	JSON, iCal
Comptes rendus	Verbatim des débats en séance	HTML, PDF

20.2.3 2.3 Fréquence de mise à jour

- Votes et dossiers : **quotidienne** (jours de séance).
- Référentiels élus : **à chaque changement** (élection partielle, démission, décès).
- Comptes rendus : **sous 24–48h** après la séance.

20.2.4 2.4 Particularités

- Les identifiants d'élus (acteur_uid) sont stables entre les législatures.
- Les scrutins contiennent le vote individuel de chaque député (pour / contre / abstention / absent).
- Les données remontent jusqu'à la **13e législature** (2007) selon les datasets.

20.3 3. Sénat

20.3.1 3.1 Portail

- URL : <https://data.senat.fr>
- API REST : <https://api.senat.fr>

20.3.2 3.2 Données disponibles

Dataset	Description	Format
Scrutins / votes	Votes en séance publique et en commission	JSON, XML
Sénateurs	Identité, groupe, département, mandats	JSON, XML
Amendements	Dépôt, sort, co-signataires	JSON, XML
Dossiers législatifs	Textes, navettes, commissions	JSON, XML
Agenda	Calendrier des séances	JSON

Dataset	Description	Format
Comptes rendus	Verbatim des débats	HTML, PDF

20.3.3 3.3 Fréquence de mise à jour

- Votes : **sous 24h** après la séance.
- Référentiels sénateurs : **temps quasi réel**.
- Comptes rendus : **48–72h** après la séance.

20.3.4 3.4 Particularités

- Le modèle de données Sénat est **structurellement différent** de celui de l'AN (schémas non identiques — transformation requise en phase Silver).
- Les votes en commission sont moins systématiquement disponibles que ceux en plénière.
- Certains anciens datasets (avant 2010) sont en **XML uniquement**.

20.4 4. Parlement européen

20.4.1 4.1 Portail

- URL : <https://data.europarl.europa.eu>
- API REST (SPARQL endpoint & REST) : <https://data.europarl.europa.eu/api/v1>

20.4.2 4.2 Données disponibles

Dataset	Description	Format
Votes en plénière	Résultats par eurodéputé	JSON-LD, RDF, XML
Eurodéputés (MEPs)	Identité, groupe politique, pays, mandats	JSON, XML
Activités parlementaires	Questions, rapports, résolutions	JSON, XML
Documents officiels	Textes législatifs (règlements, directives)	JSON, PDF
Agenda	Sessions plénières et ordre du jour	JSON, iCal

20.4.3 4.3 Fréquence de mise à jour

- Votes : **après chaque session plénière** (hebdomadaire à Strasbourg, mensuelle à Bruxelles).
- Référentiels élus : **à chaque changement** (rotation liée aux élections nationales ou décès).

20.4.4 4.4 Particularités

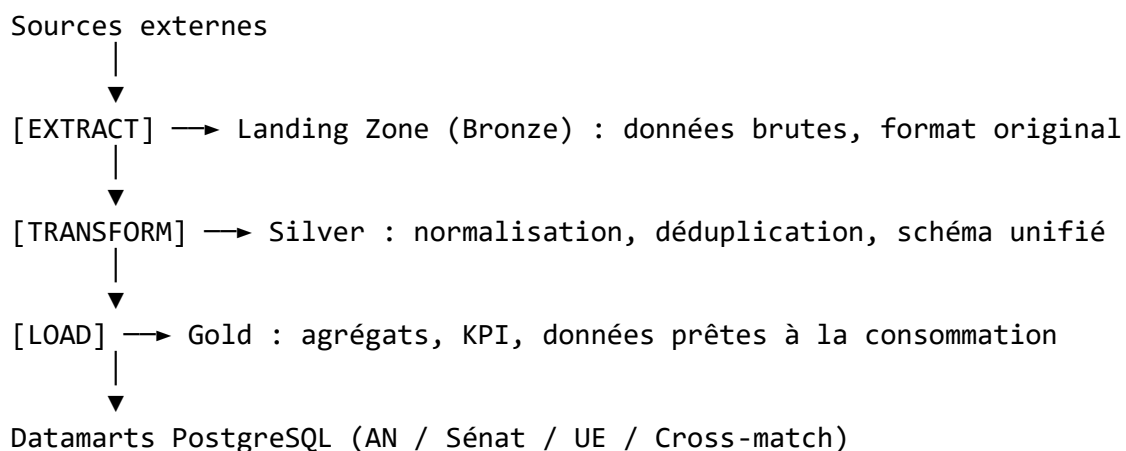
- L'API utilise partiellement le format **JSON-LD / RDF** (données liées) — parsing plus complexe.
- Les identifiants MEP (mepId) sont stables sur toute la mandature.
- Les données sont disponibles en **24 langues** ; nous ciblons le **français et l'anglais** uniquement.
- La couverture remonte à la **6e législature** (2004).

20.5 5. Formats et ingestion

20.5.1 5.1 Formats bruts attendus

Format	Sources concernées	Traitement requis
JSON	AN, Sénat, UE	Parsing direct
XML	AN, Sénat (anciens datasets)	Conversion JSON ou parsing XML
JSON-LD / RDF	UE (API SPARQL)	Parsing spécifique, aplatissement
HTML	Comptes rendus AN, Sénat	Scraping + nettoyage texte
PDF	Comptes rendus, textes officiels	Extraction texte (pdfplumber ou équivalent)

20.5.2 5.2 Stratégie d'ingestion (pipeline ETL)



20.5.3 5.3 Identifiants croisés

Pour le datamart **Cross-match**, le rapprochement entre institutions s'appuie sur :

- **Nom + prénom + date de naissance** (aucun identifiant commun inter-institutionnel officiel).

- **Groupe politique européen** ↔ **groupe parlementaire français** (table de correspondance à maintenir manuellement).

20.6 6. Licences et conditions d'utilisation

Source	Licence	Conditions
Assemblée nationale	Licence Ouverte Etalab 2.0	Mention de la source obligatoire
Sénat	Licence Ouverte Etalab 2.0	Mention de la source obligatoire
Parlement européen	CC BY 4.0	Mention de la source + intégrité des données

- **Utilisation commerciale** : autorisée sous ces licences.
- **Modification / redistribution** : autorisées, sous réserve de mention.
- **Données personnelles** : les données des élus sont des données publiques dans l'exercice de leur mandat — pas de contrainte RGPD particulière pour leur traitement dans ce contexte.

20.7 7. Limitations connues

Limitation	Source(s)	Impact	Mitigation
Historique partiel (< 2007 pour l'AN)	AN	Analyses longue durée limitées	Cadrer les analyses à partir de 2007
Schémas non normalisés entre AN et Sénat	AN, Sénat	Complexité de la couche Silver	Table de mapping explicite
Format JSON-LD complexe	UE	Temps de parsing plus long	Librairie RDFLib ou conversion préalable
Comptes rendus en PDF/HTML	AN, Sénat	Extraction texte fragile	Prétraitement dédié, validation manuelle d'un échantillon
Absences non distinguées	AN	Vote "absent" ≠ "ne participe pas"	Annoter dans le schéma Silver
Mises à jour irrégulières hors sessions	Toutes	Pipelines vides certains jours	Gestion des runs vides dans Kestra (no-op)

20.8 8. Évolutions envisagées

- **Could have** : ajout d'autres sources (conseils régionaux, assemblées d'outre-mer).
- **Won't have (for now)** : flux temps réel ou streaming — ingestion batch journalière suffisante pour le MVP.

21 POPolitics – Sécurité & RGPD

21.1 1. Introduction

Ce document décrit les mesures de sécurité applicatives et les obligations RGPD du projet POPolitics. Il complète le [Plan Qualité](#) et les [Exigences non-fonctionnelles](#).

21.2 2. Périmètre des données

POPolitics manipule deux catégories de données distinctes :

Catégorie	Exemples	Régime juridique
Données publiques d'élus	Votes, mandats, groupes politiques, amendements	Licence Ouverte Etalab 2.0 / CC BY 4.0, pas de contrainte RGPD
Données utilisateurs	Email, mot de passe hashé, élus suivis, thèmes d'intérêt	Données personnelles soumises au RGPD

Les données d'élus sont des **données publiques** liées à l'exercice d'un mandat électif. Leur traitement est légalement autorisé sans consentement individuel. Seules les données des **utilisateurs inscrits** relèvent du RGPD.

21.3 3. Authentification & gestion des accès

21.3.1 3.1 Mécanisme d'authentification

- Authentification via **JWT** (JSON Web Token) ou session sécurisée avec cookie **HttpOnly** et flag **Secure**.
- Les tokens JWT ont une durée de vie courte : **< 1 heure** avec mécanisme de refresh token.
- Les refresh tokens sont **révocables** (invalidation côté serveur en cas de déconnexion ou de compromission).

21.3.2 3.2 Stockage des mots de passe

- Les mots de passe sont hashés avec **bcrypt** ou **argon2** avant stockage.

- Aucun mot de passe n'est stocké en clair, ni loggé, ni transmis en clair dans les logs.
- Longueur minimale imposée : **8 caractères**.

21.3.3 3.3 Gestion des rôles

Rôle	Accès
Anonyme	Lecture publique (si activée) uniquement
Utilisateur standard	Accès complet au portail collectif + page personnelle
Administrateur	Accès aux fonctions d'administration (gestion utilisateurs, monitoring)

- Les droits sont vérifiés **côté serveur** (middleware Django), jamais uniquement côté client.
- Principe du **moindre privilège** : chaque rôle n'accède qu'aux ressources dont il a besoin.

21.4 4. Sécurité des échanges

21.4.1 4.1 Transport

- Toutes les communications frontend ↔ backend transitent en **HTTPS** dès la V1 serveur.
- En environnement local (MVP), HTTP est toléré uniquement en développement.
- Certificat TLS : Let's Encrypt (gratuit) ou certificat auto-signé pour les environnements internes.

21.4.2 4.2 Protection des APIs

- **CORS** configuré explicitement : whitelist des origines autorisées (pas de * en production).
- **CSRF** : protection activée sur toutes les routes POST/PUT/DELETE Django.
- **Rate limiting** : à implémenter sur les endpoints sensibles (login, refresh token) pour limiter les attaques par force brute.

21.4.3 4.3 Protection contre les injections

- **Injection SQL** : utilisation systématique de l'ORM Django — pas de requêtes SQL construites par concaténation de chaînes.
- **XSS** : échappement automatique des données côté Next.js (React échappe par défaut) ; pas d'utilisation de dangerouslySetInnerHTML sans validation préalable.
- **Injection de commandes** : aucune commande shell construite depuis des entrées utilisateur.

21.5 5. Protection des secrets

21.5.1 5.1 Règles

- Aucun secret (clé API, mot de passe, token, DSN de base de données) ne doit être **commité dans Git**.
- Le fichier `.env` est listé dans `.gitignore` — il n'est jamais versionné.
- Un fichier `.env.example` (sans valeurs réelles) est versionné pour documenter les variables requises.

21.5.2 5.2 Gestion en production

- Les secrets sont injectés via **variables d'environnement** dans Docker Compose.
- Pour la V1 Cloud : utilisation d'un gestionnaire de secrets (ex. AWS Secrets Manager, HashiCorp Vault) selon l'infrastructure retenue.

21.6 6. Obligations RGPD

21.6.1 6.1 Base légale du traitement

Articles de référence :

Art. 6§1a RGPD : « *La personne concernée a consenti au traitement de ses données à caractère personnel pour une ou plusieurs finalités spécifiques.* »

Art. 6§1b RGPD : « *Le traitement est nécessaire à l'exécution d'un contrat auquel la personne concernée est partie ou à l'exécution de mesures précontractuelles prises à la demande de celle-ci.* »

Art. 9§2e RGPD : « *Le traitement porte sur des données à caractère personnel qui sont manifestement rendues publiques par la personne concernée.* »

Traitement	Base légale
Données d'élus (votes, mandats)	Données manifestement rendues publiques par la personne concernée (Art. 9§2e RGPD)
Compte utilisateur	Consentement explicite (Art. 6§1a RGPD)
Préférences utilisateur (élus suivis, thèmes)	Exécution du contrat / consentement (Art. 6§1b RGPD)

21.6.2 6.2 Droits des utilisateurs

Les utilisateurs inscrits disposent des droits suivants, à exercer via l'interface ou par email :

Droit	Mise en œuvre
Droit d'accès	Export des données personnelles de l'utilisateur

Droit	Mise en œuvre
Droit de rectification	Modification du profil (email, préférences) depuis le compte
Droit à l'effacement	Suppression du compte et de toutes les données associées
Droit à la portabilité	Export des préférences au format JSON
Droit d'opposition	Désinscription et suppression du compte

21.6.3 6.3 Données collectées sur les utilisateurs

Donnée	Finalité	Durée de conservation
Email	Authentification, notifications	Durée du compte + 30 jours après suppression
Mot de passe hashé	Authentification	Durée du compte
Élus suivis	Personnalisation	Durée du compte
Thèmes d'intérêt	Personnalisation	Durée du compte
Logs de connexion	Sécurité	90 jours glissants

21.6.4 6.4 Minimisation des données

- Seules les données **strictement nécessaires** à la fonctionnalité sont collectées.
- Pas de collecte d'adresse postale, numéro de téléphone ou données de géolocalisation.
- Pas de partage des données utilisateurs avec des tiers.

21.6.5 6.5 Consentement

- À l'inscription, l'utilisateur doit **accepter explicitement** les conditions d'utilisation et la politique de confidentialité (case à cocher, non pré-cochée).
- Un lien vers la politique de confidentialité est accessible depuis toutes les pages.

21.7 7. Sécurité de l'infrastructure

21.7.1 7.1 Isolation des bases de données

- La base **Auth** (utilisateurs, tokens) est **strictement séparée** des datamarts métier (AN, Sénat, UE).
- Les services Django ne peuvent accéder qu'aux bases dont ils ont explicitement besoin (principe de ségrégation).

21.7.2 7.2 Journalisation

- Les événements de sécurité sont loggés : tentatives de connexion échouées, accès refusés, erreurs 5xx.

- Les logs ne contiennent **aucune donnée sensible** (pas de mot de passe, pas de token JWT complet).
- Les logs sont conservés **90 jours** puis supprimés.

21.7.3 7.3 Mises à jour de sécurité

- Les dépendances Python (Django, DRF, etc.) et Node.js (Next.js) doivent être maintenues à jour.
- Une vérification des vulnérabilités connues est effectuée avant chaque déploiement en production (`pip audit`, `npm audit`).

21.8 8. Responsabilités

Responsabilité	Rôle
Mise en œuvre des mesures de sécurité backend	Développeur Backend
Mise en œuvre CORS / HTTPS / CSRF	Tech Lead / DevOps
Conformité RGPD & politique de confidentialité	Product Owner
Revue de sécurité avant déploiement	Tech Lead
Gestion des incidents de sécurité	Tech Lead + toute l'équipe

22 POPolitics – Stratégie de tests

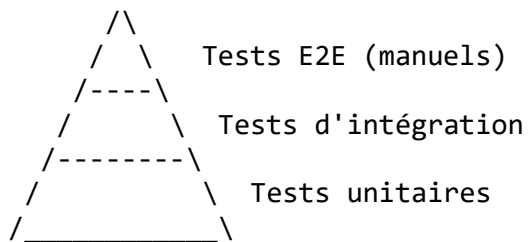
22.1 1. Introduction

Ce document décrit la stratégie de tests du projet POPolitics. Il complète le [Plan Qualité](#) et la [Definition of Done](#) en détaillant les types de tests, les outils, les responsabilités et les critères de couverture par pôle technique.

22.2 2. Principes généraux

- Tout code livré doit être accompagné des tests correspondants — c'est un critère de la Definition of Done.
 - Les tests sont **versionnés dans le dépôt Git** au même titre que le code de production.
 - On teste en priorité les **chemins critiques** (authentification, pipeline ETL, endpoints exposés au frontend).
 - Un test qui échoue **bloque le merge** de la Pull Request.
-

22.3 3. Pyramide de tests



- **Base large** : tests unitaires, rapides, nombreux.
- **Milieu** : tests d'intégration sur les interfaces entre composants.
- **Sommet** : tests E2E manuels sur les parcours utilisateurs critiques.

22.4 4. Types de tests par pôle

22.4.1 4.1 Backend (Django)

22.4.1.1 Tests unitaires

- **Outil** : pytest + pytest-django
- **Ce qu'on teste** :
 - Fonctions de validation et de transformation de données
 - Logique métier des services (Data Service, Auth Service, IA Service)
 - Serializers et permissions Django REST Framework
- **Cible de couverture** : > 80 % sur les modules critiques (MVP)

22.4.1.2 Tests d'intégration

- **Outil** : pytest-django avec base de données de test
- **Ce qu'on teste** :
 - Endpoints REST de bout en bout (requête HTTP → réponse JSON)
 - Interactions backend ↔ datamarts PostgreSQL
 - Flux d'authentification complet (inscription → login → accès protégé → déconnexion)
- **Base de données** : base PostgreSQL dédiée aux tests, réinitialisée à chaque run

22.4.1.3 Exemple de cas de test prioritaires

Cas	Type	Priorité
Login avec identifiants valides → token retourné	Intégration	Must
Login avec mauvais mot de passe → 401	Intégration	Must
GET /votes sans token → 401	Intégration	Must

Cas	Type	Priorité
GET /votes avec token valide → liste paginée	Intégration	Must
Serializer vote : champs manquants → erreur explicite	Unitaire	Should

22.4.2 4.2 Pipeline ETL (Kestra / Python)

22.4.2.1 Tests unitaires

- **Outil** : pytest
- **Ce qu'on teste** :
 - Fonctions de transformation (nettoyage, normalisation, déduplication)
 - Parsers par source (AN, Sénat, UE)
 - Validateurs de schéma (structure attendue des données Silver)

22.4.2.2 Tests de données (data quality)

- **Outil** : assertions Python dans les scripts ETL ou great_expectations (optionnel)
- **Ce qu'on teste** :
 - Nombre de lignes ingérées > 0 après un run
 - Absence de doublons sur les identifiants clés
 - Valeurs nulles dans les colonnes obligatoires
 - Cohérence des types de données (dates, entiers, chaînes)

22.4.2.3 Exemple de cas de test prioritaires

Cas	Type	Priorité
Fonction de normalisation AN : entrée JSON → sortie schéma Silver correct	Unitaire	Must
Run ETL AN : nombre de votes ingérés > 0	Data quality	Must
Cross-match : pas de doublon sur (nom, prenom, date_naissance)	Data quality	Should
Parser UE JSON-LD : champ mepId toujours présent	Unitaire	Should

22.4.3 4.3 Frontend (Next.js)

22.4.3.1 Tests unitaires

- **Outil** : Jest + React Testing Library
- **Ce qu'on teste** :

- Composants réutilisables (boutons, cartes, filtres)
- Fonctions utilitaires (formatage de dates, tri, pagination)
- Hooks personnalisés

22.4.3.2 Tests d'intégration

- **Outil** : Jest + mocks des appels API (MSW — Mock Service Worker)
- **Ce qu'on teste** :
 - Pages complètes avec données mockées (portail collectif, page personnelle)
 - Comportement des filtres et de la pagination
 - Gestion des états de chargement et d'erreur

22.4.3.3 Tests manuels (E2E)

- **Outil** : navigation manuelle dans le navigateur (Chrome / Firefox)
- **Parcours testés à chaque fin de sprint** :

Parcours	Description
Authentification	Inscription → connexion → déconnexion
Portail collectif	Consultation du tableau des votes + filtres de base
Filtres avancés	Filtrer par institution, période, groupe politique
Page personnelle	Suivre un élu → consulter sa timeline
Gestion d'erreur	Accès sans connexion → redirection login

22.5 5. Environnements de test

Environnement	Usage	Données
Local (machine développeur)	Tests unitaires et d'intégration au quotidien	Données de test synthétiques
CI (GitHub Actions)	Exécution automatique à chaque PR	Données de test synthétiques
Démo / pré-prod	Tests manuels E2E avant soutenance	Données réelles (échantillon)

22.6 6. Intégration continue (CI)

À chaque Pull Request sur la branche principale, GitHub Actions exécute automatiquement :

1. Linting du code (flake8 / ruff pour Python, ESLint pour JS)
2. Tests unitaires backend (pytest)
3. Tests unitaires frontend (jest)

4. Tests d'intégration backend (pytest-django)

Un PR ne peut pas être mergé si l'un de ces steps échoue.

22.7 7. Données de test

- Les tests utilisent des **jeux de données synthétiques** (fixtures) représentatives des données réelles.
- Les fixtures sont versionnées dans le dépôt (tests/fixtures/).
- Aucune donnée de production n'est utilisée dans les tests automatisés.
- Pour les tests manuels E2E en pré-prod, un **échantillon de données réelles** (votes AN sur 3 mois) est utilisé.

22.8 8. Responsabilités

Pôle	Responsable des tests	Type de tests
Backend	Arthur (+ relecture Jaures)	Unitaires + intégration
ETL / Data	Mehdi	Unitaires + data quality
IA	Chloé	Unitaires sur les pipelines de traitement
Frontend	Evilavy + Raphaël	Unitaires + intégration + E2E manuels
Revue globale avant soutenance	Jaures (Tech Lead)	Smoke tests sur tous les parcours

22.9 9. Critères de qualité minimaux (MVP)

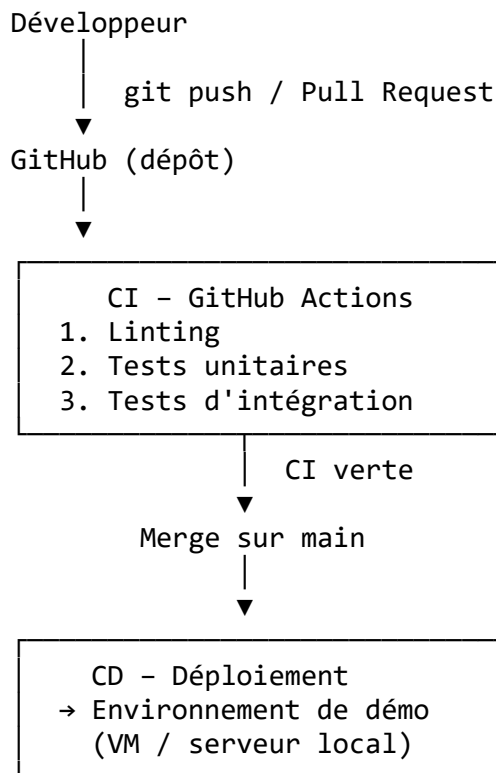
Critère	Cible
Couverture tests unitaires backend	> 80 % sur les modules critiques
Couverture tests unitaires frontend	> 50 % sur les composants partagés
Parcours E2E manuels validés	100 % des parcours Must Have
Zéro bug critique ouvert	Avant chaque démo
CI verte	Sur toutes les PR mergées

23 POPolitics – Pipeline CI/CD

23.1 1. Introduction

Ce document décrit le pipeline d'intégration continue (CI) et de déploiement continu (CD) du projet POPolitics. Il couvre les workflows automatisés, les environnements cibles et les règles de déploiement.

23.2 2. Vue d'ensemble



23.3 3. Dépôts Git

Le projet est découpé en plusieurs dépôts GitHub :

Dépôt	Contenu
popolitics/documentation	Documentation projet (ce dépôt)
popolitics/backend	Backend Django (APIs REST, Auth, IA Service)
popolitics/frontend	Frontend Next.js
popolitics/data	Pipelines ETL, scripts de transformation,

Dépôt	Contenu
	configurations Kestra

Chaque dépôt dispose de son propre pipeline CI indépendant.

23.4 4. Branches

Branche	Rôle
main	Branche stable, déployée en démo — aucun push direct
dev	Branche d'intégration — les features y sont mergées avant main
feature/<nom>	Branche de développement d'une fonctionnalité
fix/<description>	Branche de correction de bug

Règle : tout changement passe par une Pull Request. Le push direct sur main est interdit.

23.5 5. Pipeline CI (Intégration continue)

Le pipeline CI s'exécute automatiquement à chaque **push** et **Pull Request** sur dev et main.

23.5.1 5.1 Dépôt Documentation

Workflow existant : `markdown-lint.yml`

Step	Outil	Description
Markdown Lint	<code>markdownlint-cli</code>	Vérifie la syntaxe et le style de tous les fichiers <code>.md</code>

23.5.2 5.2 Dépôt Backend (Django)

Step	Outil	Description
1. Checkout	<code>actions/checkout</code>	Récupération du code
2. Setup Python	<code>actions/setup-python</code>	Installation de Python 3.11+
3. Installation des dépendances	<code>pip install -r requirements.txt</code>	Installation des packages
4. Linting	<code>ruff</code> ou <code>flake8</code>	Vérification du style PEP 8
5. Tests unitaires	<code>pytest</code>	Exécution des tests unitaires
6. Tests d'intégration	<code>pytest-django</code>	Tests avec base PostgreSQL de test

23.5.3 5.3 Dépôt Frontend (Next.js)

Step	Outil	Description
1. Checkout	actions/checkout	Récupération du code
2. Setup Node.js	actions/setup-node	Installation de Node.js 20+
3. Installation des dépendances	npm ci	Installation des packages
4. Linting	eslint	Vérification du style JS/TS
5. Tests unitaires	jest	Exécution des tests unitaires
6. Build	npm run build	Vérification que le build Next.js passe

23.5.4 5.4 Dépôt Data (ETL)

Step	Outil	Description
1. Checkout	actions/checkout	Récupération du code
2. Setup Python	actions/setup-python	Installation de Python 3.11+
3. Installation des dépendances	pip install -r requirements.txt	Installation des packages
4. Linting	ruff	Vérification du style PEP 8
5. Tests unitaires ETL	pytest	Tests des fonctions de transformation

23.6 6. Règles de merge

- Une PR ne peut être mergée que si **tous les steps CI sont verts**.
- La PR doit être approuvée par **au moins un autre membre** de l'équipe (revue de code).
- Les conflits Git doivent être résolus **par le demandeur**, pas le reviewer.

23.7 7. Pipeline CD (Déploiement continu)

23.7.1 7.1 Environnements

Environnement	Déclencheur	Infrastructure
Local (dev)	Manuel — docker-compose up	Machine du développeur
Démo / pré-prod	Merge sur main (manuel ou automatique)	VM partagée / serveur local
Production (futur)	Hors périmètre MVP	Cloud (à définir)

23.7.2 7.2 Déploiement en démo

Le déploiement sur l'environnement de démo suit les étapes suivantes :

1. Merge sur main (après CI verte + revue de code)
↓
2. Pull du code sur la VM de démo
`git pull origin main`
↓
3. Reconstruction des images Docker si nécessaire
`docker-compose build`
↓
4. Redémarrage des services
`docker-compose up -d`
↓
5. Application des migrations base de données
`docker-compose exec backend python manage.py migrate`
↓
6. Smoke tests manuels (parcours critiques)
↓
7. Démo disponible

23.7.3 7.3 Services déployés

Service	Image	Port exposé
Backend Django	popolitics/backend	8000
Frontend Next.js	popolitics/frontend	3000
PostgreSQL (Auth)	postgres:15	5432
PostgreSQL (Datamart AN)	postgres:15	5433
PostgreSQL (Datamart Sénat)	postgres:15	5434
PostgreSQL (Datamart UE)	postgres:15	5435
Kestra	kestra/kestra	8080

23.8 8. Variables d'environnement

Les variables sensibles ne sont **jamais committées** dans le dépôt. Elles sont gérées via :

- **En local** : fichier `.env` (non versionné, listé dans `.gitignore`)
- **En CI** : GitHub Actions Secrets (Settings > Secrets and variables > Actions)
- **En démo** : fichier `.env` sur la VM, accès restreint

Variables requises (exemple — voir `.env.example` dans chaque dépôt) :

```
DATABASE_URL=postgresql://user:password@localhost:5432/popolitics
SECRET_KEY=<django-secret-key>
JWT_SECRET=<jwt-secret>
KESTRA_URL=http://localhost:8080
```

23.9 9. Responsabilités

Responsabilité	Rôle
Mise en place et maintenance des workflows CI	Lazare (DevOps)
Déploiement sur la VM de démo	Lazare (DevOps)
Gestion des secrets GitHub	Lazare + Jaures (Tech Lead)
Validation post-déploiement (smoke tests)	Jaures (Tech Lead)

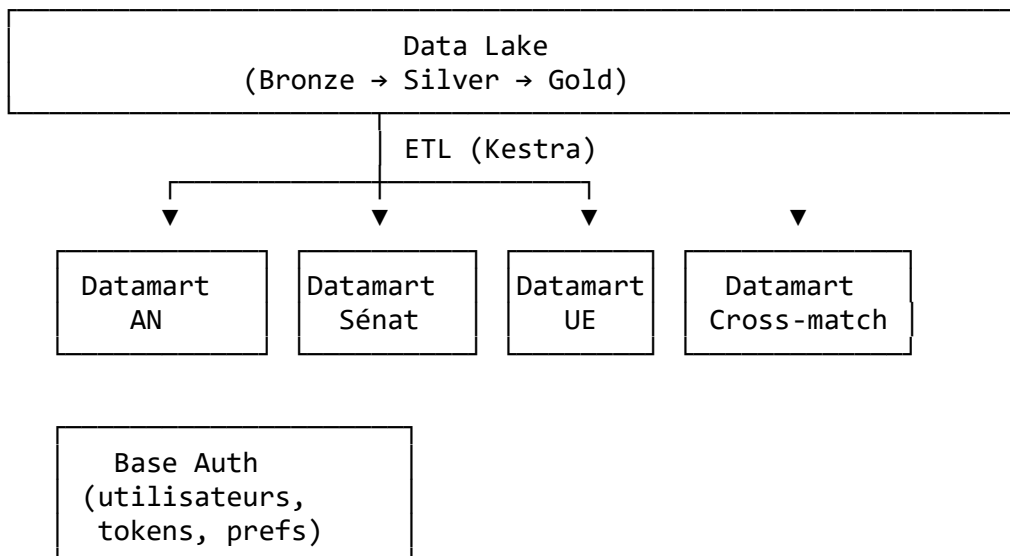
24 POPolitics – Modèle de données

24.1 1. Introduction

Ce document décrit le modèle de données de POPolitics : les entités principales, leurs attributs et leurs relations, répartis entre les 4 datamarts PostgreSQL et la base d'authentification.

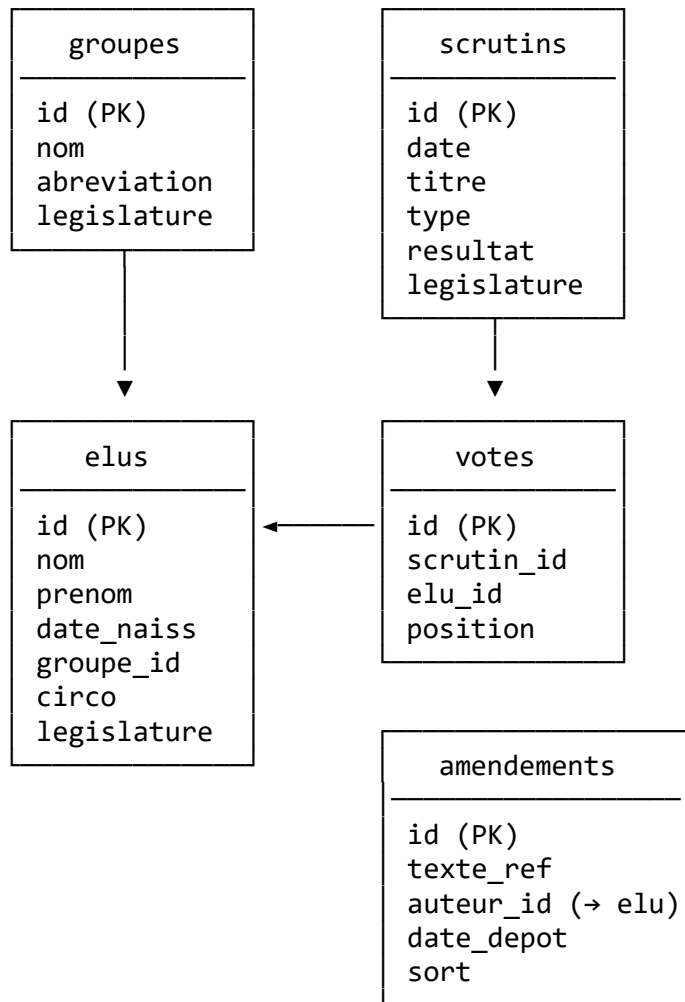
Il s'appuie sur l'architecture définie dans [01-technological-choices.md](#).

24.2 2. Vue d'ensemble des bases de données



24.3 3. Datamart Assemblée nationale

24.3.1 Schéma



24.3.2 Tables

groupes

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN
nom	VARCHAR	Nom complet du groupe
abreviation	VARCHAR	Sigle (ex. RN, RE, LFI)
legislature	INTEGER	Numéro de législature

élus

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN (acteur_uid)
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
date_naissance	DATE	Date de naissance
groupe_id	VARCHAR	Référence groupes.id
circonscription	VARCHAR	Nom de la circonscription
legislature	INTEGER	Numéro de législature

scrutins

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN
date	DATE	Date du scrutin
titre	TEXT	Intitulé du vote
type	VARCHAR	Type (solennel, ordinaire...)
resultat	VARCHAR	adopte ou rejete
legislature	INTEGER	Numéro de législature

votes

Colonne	Type	Description
id	SERIAL	Identifiant interne
scrutin_id	VARCHAR	Référence scrutins.id
elu_id	VARCHAR	Référence élus.id
position	VARCHAR	pour, contre, abstention, absent

amendements

Colonne	Type	Description
id	VARCHAR	Identifiant officiel AN
texte_ref	VARCHAR	Référence du texte de loi
auteur_id	VARCHAR	Référence élus.id
date_depot	DATE	Date de dépôt
sort	VARCHAR	adopte, rejete, retire

24.4 4. Datamart Sénat

Structure similaire au datamart AN, avec les spécificités du Sénat.

24.4.1 Tables principales

senateurs

Colonne	Type	Description
id	VARCHAR	Identifiant officiel Sénat
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
date_naissance	DATE	Date de naissance
groupe_id	VARCHAR	Référence groupes_senat.id
departement	VARCHAR	Département représenté
serie	INTEGER	Série de renouvellement (1 ou 2)

groupes_senat

Colonne	Type	Description
id	VARCHAR	Identifiant officiel Sénat
nom	VARCHAR	Nom complet
abreviation	VARCHAR	Sigle

scrutins_senat

Colonne	Type	Description
id	VARCHAR	Identifiant officiel Sénat
date	DATE	Date du scrutin
titre	TEXT	Intitulé du vote
type	VARCHAR	Type (public, commission...)
resultat	VARCHAR	adopte ou rejete

votes_senat

Colonne	Type	Description
id	SERIAL	Identifiant interne
scrutin_id	VARCHAR	Référence scrutins_senat.id
senateur_id	VARCHAR	Référence senateurs.id
position	VARCHAR	pour, contre, abstention, absent

24.5 5. Datamart Parlement européen

24.5.1 Tables principales

eurodeputes

Colonne	Type	Description
id	VARCHAR	Identifiant officiel UE (mepId)
nom	VARCHAR	Nom de famille
prenom	VARCHAR	Prénom
date_naissance	DATE	Date de naissance
pays	VARCHAR	Pays représenté
groupe_id	VARCHAR	Référence groupes_ue.id
legislature	INTEGER	Numéro de législature européenne

groupes_ue

Colonne	Type	Description
id	VARCHAR	Identifiant officiel UE
nom	VARCHAR	Nom complet (ex. PPE, S&D, Renew)
abreviation	VARCHAR	Sigle

scrutins_ue

Colonne	Type	Description
id	VARCHAR	Identifiant officiel UE
date	DATE	Date du scrutin
titre	TEXT	Intitulé
type	VARCHAR	Type de vote
resultat	VARCHAR	adopte ou rejete

votes_ue

Colonne	Type	Description
id	SERIAL	Identifiant interne
scrutin_id	VARCHAR	Référence scrutins_ue.id
eurodepute_id	VARCHAR	Référence eurodeputes.id
position	VARCHAR	pour, contre, abstention, absent

24.6 6. Datamart Cross-match

Permet de croiser les données entre les trois institutions. Alimenté depuis les couches Gold des trois autres datamarts.

24.6.1 Tables principales

elus_unifies

Colonne	Type	Description
id	SERIAL	Identifiant interne Cross-match
nom	VARCHAR	Nom normalisé
prenom	VARCHAR	Prénom normalisé
date_naissance	DATE	Date de naissance
elu_an_id	VARCHAR	Référence <code>elus.id</code> (NULL si pas de mandat AN)
senateur_id	VARCHAR	Référence <code>senateurs.id</code> (NULL si pas de mandat Sénat)
eurodepute_id	VARCHAR	Référence <code>eurodeputes.id</code> (NULL si pas de mandat UE)

correspondance_groupes

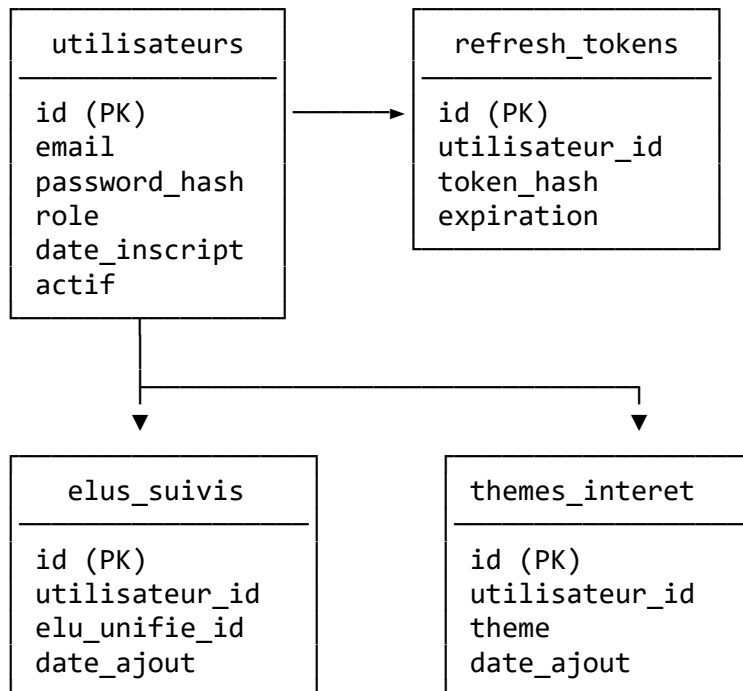
Table de mapping manuel entre les groupes des trois institutions.

Colonne	Type	Description
id	SERIAL	Identifiant interne
orientation	VARCHAR	Orientation politique (gauche, centre, droite...)
groupe_an_id	VARCHAR	Référence <code>groupes.id</code> (nullable)
groupe_senat_id	VARCHAR	Référence <code>groupes_senat.id</code> (nullable)
groupe_ue_id	VARCHAR	Référence <code>groupes_ue.id</code> (nullable)

24.7 7. Base Auth

Base strictement séparée des datamarts métier. Contient uniquement les données des utilisateurs de l'application.

24.7.1 Schéma



24.7.2 Tables

utilisateurs

Colonne	Type	Description
id	UUID	Identifiant unique utilisateur
email	VARCHAR	Email (unique)
password_hash	VARCHAR	Mot de passe hashé (bcrypt/argon2)
role	VARCHAR	user ou admin
date_inscription	TIMESTAMP	Date de création du compte
actif	BOOLEAN	Compte actif ou désactivé

refresh_tokens

Colonne	Type	Description
id	UUID	Identifiant du token
utilisateur_id	UUID	Référence utilisateurs.id
token_hash	VARCHAR	Hash du refresh token
expiration	TIMESTAMP	Date d'expiration

elus_suivis

Colonne	Type	Description
---------	------	-------------

Colonne	Type	Description
id	SERIAL	Identifiant interne
utilisateur_id	UUID	Référence utilisateurs.id
elu_unifie_id	INTEGER	Référence elus_unifies.id (Cross-match)
date_ajout	TIMESTAMP	Date de suivi
themes_interet		
Colonne	Type	Description
id	SERIAL	Identifiant interne
utilisateur_id	UUID	Référence utilisateurs.id
theme	VARCHAR	Thème choisi (ex. environnement, santé, éducation...)
date_ajout	TIMESTAMP	Date d'ajout

24.8 8. Index et performances

Table	Colonne indexée	Raison
votes	scrutin_id, elu_id	Requêtes filtrées par scrutin ou par élu
votes_senat	scrutin_id, senateur_id	Idem
votes_ue	scrutin_id, eurodepute_id	Idem
scrutins	date, legislature	Filtres temporels fréquents
elus	groupe_id	Filtres par groupe politique
elus_unifies	nom, prenom, date_naissance	Rapprochement cross-match
utilisateurs	email	Authentification (lookup unique)
elus_suivis	utilisateur_id	Chargement de la page personnelle

25 POPolitics - Diagramme de Gantt

Ce fichier centralise le planning projet sous forme de diagrammes de Gantt :

- une vue macro (sprints + jalons),
- une vue detaillee avec toutes les US planifiees par lot.

25.1 1) Vue macro

gantt

```
title POPolitics - Planning global (2026-2027)
dateFormat YYYY-MM-DD
axisFormat %d/%m/%Y
```

section Execution

```
Sprint 1 - Fondations :s1, 2026-09-21, 2026-11-20
Sprint 2 - MVP Portail collectif :s2, 2026-11-23, 2027-01-22
Sprint 3 - Enrichissements :s3, 2027-01-25, 2027-03-26
Sprint 4 - IA et finitions :s4, 2027-03-29, 2027-05-28
Sprint 5 - Stabilisation soutenance :s5, 2027-05-31, 2027-06-25
```

section Jalons

```
Lancement planning :milestone, m1, 2026-09-21, 0d
Fin Sprint 1 :milestone, m2, 2026-11-20, 0d
Fin Sprint 2 :milestone, m3, 2027-01-22, 0d
Fin Sprint 3 :milestone, m4, 2027-03-26, 0d
Fin Sprint 4 :milestone, m5, 2027-05-28, 0d
Fin de projet :milestone, m6, 2027-06-25, 0d
```

25.2 2) Vue detaillee (toutes les US)

gantt

```
title POPolitics - Gantt detaille (US par lot)
dateFormat YYYY-MM-DD
axisFormat %d/%m/%Y
```

section Cadre sprint

```
Sprint 1 :sp1, 2026-09-21, 2026-11-20
Sprint 2 :sp2, 2026-11-23, 2027-01-22
Sprint 3 :sp3, 2027-01-25, 2027-03-26
Sprint 4 :sp4, 2027-03-29, 2027-05-28
Sprint 5 :sp5, 2027-05-31, 2027-06-25
```

section Auth (auth-service)

```
0 US-101 Créer un compte :crit, us101, 2026-09-21, 2026-11-2
0 US-102 Se connecter :crit, us102, 2026-09-21, 2026-11-2
0 US-103 Se deconnecter :crit, us103, 2026-09-21, 2026-11-2
0 US-104 Reinitialiser mot de passe :us104, 2027-03-29, 2027-05-28
```

section Data ETL (data)

```
0 US-601 Ingestion AN :crit, us601, 2026-09-21, 2026-11-2
0 US-602 Ingestion Senat :crit, us602, 2026-09-21, 2026-11-2
```

0	US-603 Ingestion UE	:crit, us603, 2026-09-21, 2026-11-2
0	US-604 Normalisation Silver	:crit, us604, 2026-09-21, 2026-11-2
	US-605 Agregation Gold	:us605, 2026-11-23, 2027-01-22
	section API (api)	
2	US-201 Tableau des votes	:crit, us201, 2026-11-23, 2027-01-2
	US-204 Indicateurs d un vote	:us204, 2026-11-23, 2027-01-22
	US-301 Coherence d un élu	:us301, 2027-01-25, 2027-03-26
	US-302 Comparer élus/groupes	:us302, 2027-01-25, 2027-03-26
	US-304 Croisement AN/Senat/UE	:us304, 2027-01-25, 2027-03-26
	US-303 Cartographie alliances	:us303, 2027-03-29, 2027-05-28
	section Frontend (frontend)	
2	US-202 Filtres de base	:crit, us202, 2026-11-23, 2027-01-2
	US-203 Filtres avances	:us203, 2026-11-23, 2027-01-22
	US-501 Suivre des élus	:us501, 2027-01-25, 2027-03-26
	US-502 Timeline élu suivi	:us502, 2027-01-25, 2027-03-26
	US-503 Themes d interet	:us503, 2027-01-25, 2027-03-26
	US-504 Notifications personnalisées	:us504, 2027-01-25, 2027-03-26
	US-205 Export CSV/PDF	:us205, 2027-03-29, 2027-05-28
	section IA (ia-service)	
	US-401 Resume IA debat	:us401, 2027-03-29, 2027-05-28
	US-402 Fiche éclair texte	:us402, 2027-03-29, 2027-05-28
	section Stabilisation (Sprint 5)	
5	Tests E2E et corrections	:active, st1, 2027-05-31, 2027-06-2
	Preparation soutenance	:st2, 2027-05-31, 2027-06-25

26 POPolitics – Solutions IA & Model Training

26.1 1. Vue d'ensemble

POPolitics intègre une couche IA destinée à enrichir l'exploitation des données parlementaires issues des couches **Silver / Gold**.

Les fonctionnalités IA reposent sur un **IA Service indépendant du frontend**, consommé par le backend Django via API.

La couche IA permet :

- la génération automatique de résumés,
- l'analyse de similarité entre textes,

- un chatbot conversationnel sur les données parlementaires,
- la classification de documents,
- l'analyse de tendances.

Les modèles sont exécutés via un environnement dédié pouvant utiliser : - des modèles locaux auto-hébergés, - ou des API LLM externes.

27 2. Architecture IA

27.1 2.1 Vue logique

Datamarts PostgreSQL (Silver / Gold) | ▼ IA Service | | | ▼ ▼
 Traitements IA LLM Gateway | | ▼ ▼ Résultats enrichis LLM | ▼ PostgreSQL / Index
 vectoriel

27.2 2.2 Intégration applicative

L'IA Service est exposé comme un service indépendant.

Le backend Django consomme ses fonctionnalités via API REST.

Frontend NextJS | ▼ Backend Django | ▼ IA Service | |— Résumé automatique |—
 Similarité |— Chatbot RAG |— Classification |— Analyse

28 3. Cas d'usage IA

28.1 3.1 Résumés automatiques (MVP)

28.1.1 Objectif

Générer automatiquement un résumé compréhensible d'une loi, d'un amendement ou d'un débat parlementaire.

28.1.2 Sources

- textes de lois,
- amendements,
- comptes rendus,
- débats parlementaires.

Données utilisées depuis :

Data Gold PostgreSQL, documents nettoyés issus de la couche Silver.

28.1.3 Pipeline

Texte parlementaire | ▼ Nettoyage / découpage | ▼ LLM | ▼ Résumé structuré | ▼
Stockage résultat json

Exemple :

```
{ "objectif": "...", "mesures_principales": [ "...", "..." ], "impact": "...", "resume_court":  
  "..." }
```

28.2 3.2 Score de similarité textes (MVP)

28.2.1 Objectif

Comparer : - lois, - amendements, - déclarations de groupes, - positions politiques.

28.2.2 Approche

Utilisation d'embeddings vectoriels.

Texte | ▼ Embedding model | ▼ Vecteur | ▼ Recherche similarité | ▼ Score

28.2.3 Technologie

- Sentence Transformers (modèle français)
- Base vectorielle PostgreSQL + pgvector

28.2.4 Exemple de sortie

Loi A ↔ Amendement B

Similarité : 87%

28.3 3.3 Chatbot parlementaire (V1)

28.3.1 Objectif

Permettre une interrogation en langage naturel des données parlementaires.

28.3.2 Architecture

Approche RAG (Retrieval Augmented Generation).

Question utilisateur | ▼ Recherche vectorielle | ▼ Contexte parlementaire | ▼ LLM | ▼
Réponse sourcée

28.3.3 Sources interrogées

- lois,
- amendements,
- débats,

- votes,
- données parlementaires.

28.3.4 Capacités

- recherche documentaire,
- explication de textes,
- synthèse,
- questions/réponses.

29 4. Modèles IA

29.1 4.1 Modèles LLM

Deux modes de déploiement : Mode : Local / Usage : Développement, tests, confidentialité
 Mode: API externe / Usage : rodution, montée en charge

Exemples : - Mistral, - Qwen, - modèles compatibles OpenAI API.

29.2 4.2 Modèles d'embeddings

Utilisés pour : - recherche sémantique, - RAG, - similarité.

Pipeline :

Document | ▼ Découpage en chunks | ▼ Embedding | ▼ Index vectoriel

30 5. Stockage IA

30.1 5.1 Base vectorielle

Utilisation de pgvector pour : - embeddings, - recherche sémantique, - RAG.

Stockage : - documents - chunks - embeddings - metadata

31 6. Déploiement

31.1 6.1 Environnement

31.1.1 Développement :

Ollama + Modèle local

31.1.2 Production :

IA Service | ▼ LLM Gateway | └─ Modèle auto-hébergé ─┐ API LLM externe

32 7. Limites connues

Taille des textes parlementaires : Dépassement contexte LLM -> Découpage en chunks
Hallucinations LLM : Réponses incorrectes -> RAG + sources Coût API externe : Budget variable -> Modèles locaux Qualité des résumés : Dépend du modèle -> Validation humaine Similarité sémantique : Score approximatif -> Ajustement du modèle embedding

33 Benchmarks POPolitics

Scripts de mesure **reproductibles** servant à justifier, par des chiffres, les choix de stack documentés dans ../07-tech-stack-comparison.md.

L'objectif n'est **pas** de produire un benchmark de laboratoire parfait, mais de fournir une mesure honnête, reproductible sur la machine de l'équipe, qui illustre concrètement les écarts attendus entre les options envisagées.

Les chiffres dépendent fortement de la machine. Lancez toujours les deux côtés d'une comparaison **sur la même machine, dans la même session**, et reportez la config matérielle.

33.1 0. Démarrage rapide `bench.ps1`

Tout est piloté par un utilitaire unique à la racine, `bench.ps1` (Windows / PowerShell 7+) :

```
.\bench.ps1 setup          # installe les dépendances (venv Python + npm des 2
apps front)
.\bench.ps1 backend        # bench back : FastAPI vs Django (wrk via Docker)
.\bench.ps1 frontend      # bench front : Core Web Vitals (Lighthouse) + test
SEO
.\bench.ps1 seo            # uniquement le test SEO / crawlabilité
.\bench.ps1 all            # tout d'un coup
.\bench.ps1 help           # rappel des commandes
```

Prérequis : **Docker Desktop lancé** (back) et **Node.js + Chrome** (front). Les sections ci-dessous détaillent ce que chaque commande fait, et comment lancer les tests à la main.

33.2 1. Backend Django vs FastAPI (overhead du framework)

On compare l'overhead pur des deux frameworks à **serveur égal** : les deux apps sont servies par uvicorn (1 worker), exposent des endpoints identiques, et n'ont aucun middleware superflu.

La charge est générée par **wrk** (outil natif, lancé via Docker). On n'utilise pas de client Python maison : un client mono-process sature lui-même avant le serveur (~150 req/s) et

ne discrimine pas les frameworks. Endpoints : /json (réponse minimale), /deputes (200 objets).

33.2.1 Prérequis

- **Docker Desktop lancé** (pour wrk).
- venv Python avec les dépendances :

```
cd backend
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

33.2.2 Lancer (automatique) — recommandé

```
.\run.ps1 # 8 threads, 50 connexions, 12 s
.\run.ps1 -Connections 200 -Duration 15
```

run.ps1 démarre FastAPI (8001) puis Django (8002), les charge avec wrk sur /json et /deputes, affiche débit + latences, puis arrête tout. Une seule commande.

33.2.3 Lancer (manuel, 2 terminaux)

Terminal 1 – démarrer un serveur :

```
python -m uvicorn fastapi_app:app --host 0.0.0.0 --port 8001 --workers
1
python -m uvicorn django_app:application --host 0.0.0.0 --port 8002 --workers
1
```

Terminal 2 – charger via wrk (conteneur) :

```
docker run --rm williamyeh/wrk -t8 -c50 -d12s --latency http://host.docker.in
ternal:8001/deputes
```

wrk affiche le débit (Requests/sec) et la distribution de latence (p50/p90/p99).

33.2.4 Mesurer les services réels

wrk accepte n'importe quelle URL : on peut viser directement les services du docker-compose (api, ia-service) une fois lancés, plutôt que les apps de démo.

```
docker run --rm williamyeh/wrk -t8 -c50 -d15s --latency http://host.docker.in
ternal:8000/api/deputes
```

33.3 2. Frontend Next.js (SSR/SSG) vs SPA React

Deux apps témoins servent la **même page** (table de 200 députés) :

- frontend-next/ — Next.js 14, contenu pré-rendu côté serveur (SSG).
- frontend-spa/ — Vite + React 18, rendu 100 % côté client (CSR).

On mesure (a) les **Core Web Vitals** via Lighthouse et (b) la **crawlabilité / SEO** via le HTML brut. Toujours tester en mode **production** (build puis start/preview), jamais en dev.

33.3.1 Prérequis

Node.js. Lighthouse est appelé via npx. Chrome doit être installé (Lighthouse le pilote en headless).

33.3.2 a) Core Web Vitals (Lighthouse)

```
# 1. Builder puis servir Next.js (port 3000)
cd frontend-next ; npm install ; npm run build
node node_modules/next/dist/bin/next start -p 3000      # laisser tourner
# 2. Dans un autre terminal :
./frontend/lighthouse_bench.ps1 -Url "http://localhost:3000" -Runs 5 -Label "
Next.js"
```

```
# Puis la SPA (port 4173)
cd frontend-spa ; npm install ; npm run build
node node_modules/vite/bin/vite.js preview --port 4173 # laisser tourner
./frontend/lighthouse_bench.ps1 -Url "http://localhost:4173" -Runs 5 -Label "
SPA React"
```

En **localhost** (latence réseau ~0) et avec des données inlineées, la SPA peut faire jeu égal voire mieux : les avantages du SSR (réseau réel, cascade de fetch, SEO) ne s'y voient pas. Nous ne pouvons pas conclure « Next.js inutile » sur ce seul test, ce qui nous amène au point (b), décisif.

33.3.3 b) Crawlabilité / SEO (HTML brut)

Le critère qui justifie réellement Next.js pour un site public. On compare ce que reçoit un crawler **avant** d'exécuter le JS :

```
# Serveurs lancés (3000 = Next, 4173 = SPA) :
(Invoke-WebRequest http://localhost:3000 -UseBasicParsing).Content -match "Député" # → contenu présent
(Invoke-WebRequest http://localhost:4173 -UseBasicParsing).Content
# → <div id="root"></div> vide
```

Résultat type : Next.js sert ~90 ko de HTML contenant toute la table (« Député » ×404) ; la SPA sert ~0,5 ko avec un <body> vide (« Député » ×1, dans le <title>).

33.4 3. Où reporter les résultats

Section « **Performances mesurées** » de ../07-tech-stack-comparison.md. Compléter les tableaux et indiquer la machine de test (CPU, RAM, OS) ainsi que la date.

34 POPolitics - Dossier marketing

34.1 1. Introduction

34.1.1 1.1 Presentation du projet

POPolitics est une plateforme civic-tech qui aide les citoyens a comprendre l'activite parlementaire a partir de donnees publiques officielles. Le produit centralise les votes, prises de position, tendances et debats issus de plusieurs institutions, puis les transforme en informations lisibles, comparables et actionnables.

Le projet repose sur trois piliers :

- **transparence democratique,**
- **accessibilite de la donnee publique,**
- **personnalisation de l'experience utilisateur.**

34.1.2 1.2 Contexte et origine de l'idee

Le point de depart de POPolitics est un constat simple : les donnees parlementaires existent, mais elles restent difficiles a lire, a comparer et a exploiter pour un non-specialiste. Les portails officiels proposent une information riche mais peu pedagogique, tandis que les medias simplifient souvent les sujets ou se concentrent sur l'actualite immediate.

Dans ce contexte, POPolitics se positionne comme un **pont entre la donnee brute et la comprehension citoyenne**. Le projet nait dans un cadre academique Epitech, mais il est pense des l'origine comme une plateforme capable de continuer a vivre apres les etudes, avec une ambition produit et une trajectoire de croissance reelle.

34.1.3 1.3 Objectifs du projet

Les objectifs marketing et produit de POPolitics sont les suivants :

- rendre le suivi parlementaire comprehensible en moins de quelques minutes,
- creer une marque civic-tech reconnue pour sa fiabilite,
- attirer une premiere base d'utilisateurs engagees,
- fournir un outil utile aux journalistes et analystes,
- construire une trajectoire de perennisation post-etudes.

34.2 2. Presentation de l'offre

Note d'adaptation : le sommaire initial etait formule pour une agence. Il a ete adapte ici a une logique de **plateforme produit**.

34.2.1 2.1 Concept de la plateforme

POPolitics est une plateforme d'analyse politique accessible via le web. Elle combine :

- des données ouvertes issues de l'Assemblée nationale, du Sénat et du Parlement européen,
- des traitements ETL pour nettoyer et normaliser l'information,
- une interface claire pour explorer votes, tendances et débats,
- des fonctions de personnalisation pour suivre des élus et des thèmes,
- à terme, des fonctions IA pour accélérer la compréhension des textes et débats.

Le concept ne consiste pas à produire de l'opinion, mais à **mettre en scène des faits politiques vérifiables**.

34.2.2 2.2 Fonctionnement du service

Le service suit un cycle simple :

1. ingestion quotidienne des données publiques,
2. normalisation et enrichissement des informations,
3. exposition via une interface web et des APIs,
4. consultation par l'utilisateur selon son besoin,
5. personnalisation progressive via suivis, filtres et alertes.

Pour l'utilisateur final, la promesse est simple :

- rechercher un élu,
- voir ses votes et positions,
- comprendre le contexte d'un texte,
- comparer plusieurs profils ou groupes,
- recevoir des alertes sur les sujets qui comptent.

34.2.3 2.3 Description des offres et trajectoire de monétisation

Le projet est pensé selon une logique de lancement progressif.

Niveau d'offre	Cible	Contenu	Statut
Offre publique gratuite	Citoyens	Consultation des votes, filtres, pages élus, notifications de base	Cible MVP
Offre Pro	Journalistes, chercheurs, ONG	Filtres avancés, exports, comparateurs, historique plus	Hypothèse post-lancement

Niveau d'offre	Cible	Contenu	Statut
Offre Data / API	Medias, associations, partenaires institutionnels	profond, visualisations Acces API, extractions structurees, jeux de donnees enrichis, support	Hypothese de croissance

Position retenue a court terme :

- **gratuit pour le grand public** afin de maximiser l'adoption,
- **monetisation differenciee** seulement lorsque la valeur d'usage et la stabilite produit seront prouvees.

34.2.4 2.4 Experience client

L'experience souhaitee est fondee sur quatre moments clefs :

Etape	Experience attendue
Decouverte	Comprendre en quelques secondes ce que fait POPolitics et a qui cela sert
Premiere valeur	Trouver rapidement un vote, un élu ou un texte sans apprentissage complexe
Engagement	Revenir pour suivre des themes, sauvegarder des preferences et recevoir des alertes
Prescription	Partager une analyse, un graphique ou une fiche utile a d'autres utilisateurs

Le positionnement UX doit rester :

- lisible pour le grand public,
- robuste pour les utilisateurs avances,
- credible pour les journalistes et partenaires.

34.3 3. Etude de marche

34.3.1 3.1 Analyse du marche

POPolitics se situe a la jonction de quatre marches :

- la **civic-tech**,
- l'**open data publique**,

- les **medias et outils d'analyse politique**,
- les **outils d'aide a la decision et a la veille**.

Le contexte de marche est favorable pour plusieurs raisons :

- la demande de transparence democratique augmente,
- la fatigue informationnelle pousse vers des formats plus synthétiques,
- les jeux de donnees publics deviennent plus accessibles,
- les outils IA rendent possible une meilleure mediation de contenus complexes.

Le marche reste toutefois fragmenté : l'offre est dispersee entre sites institutionnels, associations, journalistes data et outils generiques d'analyse.

34.3.2 3.2 Analyse de la demande

La demande adressee par POPolitics repose sur trois besoins concrets :

Besoin	Description	Cible principale
Comprendre vite	Acceder en moins d'une minute a l'essentiel d'un vote ou d'un texte	Citoyens
Explorer en profondeur	Filtrer, croiser, comparer et exporter les donnees	Journalistes / experts
Suivre dans le temps	Recevoir des alertes et construire une veille personnalisee	Citoyens engages, ONG, experts

La demande est probablement plus forte sur les moments de tension democratique : elections, grandes reformes, crises sociales, textes clivants.

34.3.3 3.3 Personas

Les personas detailles sont documentes dans [02-technologie/00-personas.md](https://02-technologie.github.io/00-personas.md).

Synthese marketing :

Persona	Motivation principale	Attente marketing
Marie Fontaine	Comprendre ce que votent ses representants	Simplicité, rapidité, mobile-first
Thomas Marchand	Exploiter des donnees fiables pour produire des analyses	Precision, export, profondeur
Lucas Perrin	Garantir la fiabilite et la disponibilite de la plateforme	Robustesse, monitoring, credibilite technique

34.3.4 3.4 Analyse de la concurrence

La concurrence de POPolitics est surtout **indirecte** ou **partielle**.

Type de concurrent	Exemples	Forces	Limites face a POPolitics
Portails institutionnels	Assemblée nationale, Senat, Parlement europeen	Source officielle, exhaustivite	UX complexe, peu de personnalisation, croisement limite
Acteurs civic-tech	Regards Citoyens, projets open data citoyens	Credibilite democratique, transparence	Offre parfois technique, experience inegale selon les usages
Medias et data journalisme	Rubriques data de medias, formats interactifs ponctuels	Pedagogie, audience	Vision episodique, pas d'outil continu et personnalisable
Outils generiques data	Tableurs, BI, scraping perso	Flexibilite	Requiert du temps, des competences et peu d'accessibilite grand public

Conclusion concurrentielle : POPolitics peut se differencier en combinant **rigueur des donnees, UX accessible, croisement multi-institutionnel** et **personnalisation**.

34.3.5 3.5 Tendances du marche

Les tendances qui soutiennent POPolitics sont :

- progression des usages de l'open data,
- demande croissante de formats visuels et pedagogiques,
- recherche de produits de veille personnalisee,
- montee des attentes sur l'explicabilite de l'IA,
- essor des produits a impact public et civic-tech.

Les risques de marche a surveiller sont :

- saturation informationnelle,
- defiance envers les outils perçus comme politiques,
- dependance aux plateformes sociales pour l'acquisition,
- evolution des politiques d'acces aux APIs publiques.

34.3.6 3.6 Analyses strategiques

34.3.6.1 SWOT

Forces

Faiblesses

Forces	Faiblesses
Positionnement clair sur la transparence démocratique	Produit encore en phase de construction
Multi-source AN / Senat / UE	Marque encore inconnue
Forte valeur perçue pour plusieurs publics	Dépendance aux données et APIs externes
Différenciation par la lisibilité et la personnalisation	Ressources humaines et budget limités au départ

Opportunités	Menaces
Croissance de la civic-tech et de l'open data	Changement de formats des données publiques
Partenariats médias, associations, écoles	Récupération du sujet par un acteur plus installé
Besoin de veille politique outillée	Polemiques sur la neutralité de la plateforme
Valorisation post-études comme startup ou association	Faible rétention si l'usage n'est pas assez récurrent

34.3.6.2 PESTEL

Dimension	Impact sur POPolitics
Politique	Forte actualité parlementaire, besoin de transparence, sensibilité au positionnement partisan
Economique	Budget initial limité mais coûts d'infrastructure modérés au lancement
Social	Défiance envers les institutions mais forte attente de compréhension simple
Technologique	Open data, ETL, IA et visualisation favorisent la proposition de valeur
Environnemental	Impact faible, mais choix d'une infra sobre et progressive valorisable
Legal	RGPD, licences ouvertes, réutilisation des données publiques à maîtriser

34.3.6.3 Five Forces de Porter

Force	Niveau	Lecture
Intensité concurrentielle	Moyen	Peu d'acteurs couvrent toute la chaîne, mais plusieurs alternatives partielles existent

Force	Niveau	Lecture
Menace des nouveaux entrants	Moyen	Les donnees sont ouvertes, mais l'execution technique et UX reste difficile
Pouvoir des fournisseurs	Eleve	Les institutions sources controlent l'accès et les formats de donnees
Pouvoir des clients	Moyen	Les utilisateurs ont beaucoup d'alternatives gratuites, donc l'usage doit etre tres clair
Menace des substituts	Elevee	Sites officiels, articles de presse, analystes et outils perso peuvent remplacer partiellement l'usage

34.4 4. Positionnement et strategie marketing

34.4.1 4.1 Positionnement

POPolitics se positionne comme une **plateforme civic-tech de reference pour lire, comparer et suivre l'activite parlementaire**.

Le positionnement souhaite peut se formuler ainsi :

Pour les citoyens et analystes qui veulent comprendre la politique a partir de faits, POPolitics est la plateforme qui transforme la donnee parlementaire brute en lecture claire, personnalisee et exploitable.

34.4.2 4.2 Proposition de valeur

La proposition de valeur repose sur quatre promesses :

- **voir clair** dans des donnees habituellement opaques,
- **gagner du temps** dans la recherche et l'analyse,
- **suivre ce qui compte** grace a la personnalisation,
- **faire confiance** a une approche documentee, traçable et ouverte.

34.4.3 4.3 Avantage concurrentiel

Les avantages concurrentiels les plus solides sont :

- croisement AN / Senat / UE dans une meme experience,

- niveau de lisibilité pensée pour le grand public,
- profondeur de lecture suffisante pour les journalistes,
- trajectoire produit crédible après les études,
- dimension open source et civic-tech rassurante pour la crédibilité.

34.4.4 4.4 Ciblage

Le ciblage recommandé est progressif.

Horizon	Cible prioritaire	Objectif
Court terme	Citoyens engagés et jury Epitech	Prouver la clarté du produit et sa valeur immédiate
Moyen terme	Journalistes, chercheurs, ONG	Installer la crédibilité analytique du projet
Long terme	Partenaires médias, institutions, associations	Structurer un modèle durable et un rayonnement large

34.5 5. Marketing mix (4P)

34.5.1 5.1 Produit

Le produit est une plateforme web structurée autour de trois blocs :

- **portail collectif** pour explorer votes, textes et tendances,
- **page personnelle** pour suivre élus et thèmes,
- **couche analytique** pour fournir indicateurs, exports et, à terme, synthèses IA.

Les composantes du produit les plus marketables sont :

- tableau de votes lisible,
- fiches élus,
- indicateurs de cohérence,
- comparateurs,
- visualisations,
- alertes personnalisées.

34.5.2 5.2 Prix

La logique tarifaire doit soutenir la mission du projet.

Offre	Politique tarifaire recommandée
Grand public	Gratuit

Offre	Politique tarifaire recommandee
Pro / media / ONG	Abonnement mensuel ou annuel a tester apres preuve d'usage
API / partenariats	Tarification sur devis selon volume et besoins

Hypotheses de prix a explorer post-lancement :

- offre Pro entre **15 et 29 EUR / mois**,
- offre equipe entre **99 et 299 EUR / mois**,
- prestations data ou partenariats sur mesure.

34.5.3 5.3 Distribution

La distribution principale est **digitale directe** :

- site web / application web,
- documentation GitHub,
- relais sociaux,
- demonstration live en soutenance et evenements,
- reseau de partenaires civic-tech et medias.

Le produit doit rester accessible sans friction : pas d'installation locale pour l'utilisateur final, pas de courbe d'apprentissage inutile, acces rapide au coeur de la valeur.

34.5.4 5.4 Communication

La communication marketing s'appuie sur :

- des contenus pedagogiques et data-driven,
- des captures d'ecran et visualisations parlantes,
- des cas d'usage concrets,
- un discours de marque centre sur la transparence, pas sur le commentaire partisan.

Le detail du plan de communication est documente dans [02-plan-de-communication.md](#).

34.6 6. Approche centree client (4C)

34.6.1 6.1 Client

Le client principal n'est pas defini seulement par son profil socio-demographique, mais par son besoin :

- mieux comprendre,

- mieux verifier,
- mieux suivre,
- mieux reutiliser.

34.6.2 6.2 Cout

Le cout perçu ne se limite pas au prix. Pour POPolitics, il inclut :

- le temps passe a comprendre l'interface,
- l'effort necessaire pour trouver une information,
- la confiance accordee aux donnees,
- le risque de surcharge informationnelle.

La strategie doit donc viser a reduire ce cout perçu avant meme de parler d'argent.

34.6.3 6.3 Convenience

La commodite d'usage est un facteur cle :

- experience fluide sur mobile et desktop,
- acces rapide aux recherches principales,
- filtres comprehensibles,
- notifications utiles plutot qu'invasives,
- consultation simple meme pour un non-expert.

34.6.4 6.4 Communication

La communication avec le client doit rester :

- claire,
- factuelle,
- non partisane,
- utile,
- reguliere.

Chaque prise de parole doit repondre a une question implicite de l'utilisateur :

- pourquoi cela me concerne,
 - qu'est-ce que je comprends grace a vous,
 - pourquoi puis-je vous faire confiance.
-

34.7 7. Strategie de communication

Cette section synthétise la logique marketing. Le detail operationnel est documente dans [02-plan-de-communication.md](#).

34.7.1 7.1 Objectifs

- faire connaitre la marque POPolitics,
- installer une image de fiabilite,
- recruter des beta-utilisateurs,
- favoriser le bouche-a-oreille et la reprise par des tiers,
- soutenir la continuite du projet apres les etudes.

34.7.2 7.2 Cibles

- citoyens engages,
- journalistes et data journalistes,
- chercheurs et experts,
- ONG et associations civic-tech,
- ecole, jury, partenaires potentiels.

34.7.3 7.3 Messages

- **Vous pouvez enfin voir ce que votent reellement vos representants.**
- **Les donnees politiques deviennent lisibles, comparables et verifiables.**
- **POPolitics fait gagner du temps sans sacrifier la rigueur.**
- **Le projet continue apres les etudes : ce n'est pas une simple demo.**

34.7.4 7.4 Canaux

- GitHub,
- LinkedIn,
- X / Mastodon,
- relations presse cibles,
- soutenance et evenements,
- reseau communautaire civic-tech.

34.7.5 7.5 Plan media

Le plan media recommande reste majoritairement **organique** au lancement.

Canal	Priorite	Logique
GitHub	Haute	Credibilite technique et open source
LinkedIn	Haute	Notoriete pro, storytelling

Canal	Priorite	Logique
X / Mastodon	Moyenne	de progression Diffusion rapide de faits et visualisations
Email cible	Moyenne	Contact direct avec medias, ONG, partenaires
Paid media	Faible	A tester seulement si traction organique deja validee

34.8 8. Plan d'actions marketing

34.8.1 8.1 Actions digitales

Action	Objectif	Horizon
Optimiser le README et la landing narrative	Clarifier la promesse produit	Court terme
Publier des captures et cas d'usage	Rendre la valeur visible	Court terme
Produire des posts LinkedIn de sprint	Installer la marque et la progression	Court terme
Ouvrir une roadmap publique	Montrer la suite du projet	Moyen terme
Lancer une newsletter ou changelog public	Fideliser une audience recurrente	Moyen terme

34.8.2 8.2 Actions evenementielles

Action	Objectif
Soutenance avec demo live	Prouver la valeur et la maturite du produit
Presentation dans un meetup etudiant ou civic-tech	Gagner en visibilite et en retours externes
Participation a un salon open source ou data	Rencontrer partenaires et utilisateurs avances

34.8.3 8.3 Partenariats

Les partenariats les plus pertinents sont :

- associations civic-tech,
- medias independants,
- ecoles et laboratoires de recherche,

- ONG specialisees en democratie et transparence,
- collectifs de journalistes data.

Objectif partenariat :

- gagner en credibilite,
- acceder a de nouveaux usages,
- obtenir du feedback qualifie,
- envisager des relais de diffusion et de financement.

34.8.4 8.4 Calendrier editorial

Le calendrier editorial detaille est deja defini dans [02-plan-de-communication.md](#). Du point de vue marketing, il doit suivre trois temps :

Phase	Objectif editorial
Avant MVP	Raconter la construction et installer le probleme traite
Apres MVP	Montrer des cas d'usage concrets et des preuves produit
Post-etudes	Faire rayonner POPolitics comme produit durable avec une suite claire

34.9 9. Budget previsionnel

34.9.1 9.1 Repartition des couts

Le budget marketing recommande doit rester realiste et sobre au lancement.

34.9.1.1 Scenario 1 - lancement academique

Poste	Estimation
Production de contenus (en interne)	0 EUR
Captures / GIF / assets	0 a 50 EUR
Nom de domaine / outils de base	10 a 30 EUR / mois
Emailing / formulaire / analytics	0 a 20 EUR / mois
Deplacements evenements	0 a 150 EUR
Total indicatif	10 a 250 EUR

34.9.1.2 Scenario 2 - lancement post-etudes

Poste	Estimation
Identite visuelle et kit media	100 a 300 EUR
Outils de diffusion / CRM / newsletter	20 a 60 EUR / mois

Poste	Estimation
Hebergement de demo publique	40 a 100 EUR / mois
Production video courte	0 a 200 EUR
Tests d'acquisition payants	100 a 300 EUR
Total indicatif	260 a 960 EUR au lancement

34.9.2 9.2 ROI attendu

Le ROI doit etre lu en deux temps.

34.9.2.1 ROI non financier au lancement

- notoriété de la marque,
- crédibilité du projet,
- acquisition de beta-testeurs,
- opportunités de partenariat,
- légitimité pour la suite post-etudes.

34.9.2.2 ROI financier a moyen terme

Hypotheses de traction a surveiller :

- **50 utilisateurs Pro** a 19 EUR / mois = **950 EUR / mois**,
- **10 organisations** sur une offre equipe a 149 EUR / mois = **1 490 EUR / mois**,
- **1 a 2 partenariats data** annuels pouvant financer la maintenance ou la croissance.

Ces chiffres sont des hypotheses de cadrage marketing, pas des engagements commerciaux.

34.10 10. Indicateurs de performance (KPI)

34.10.1 10.1 Notoriété

- trafic sur la landing page,
- vues du README GitHub,
- impressions LinkedIn,
- mentions externes du projet,
- nombre de stars GitHub.

34.10.2 10.2 Acquisition

- nombre d'inscriptions,
- taux de conversion visiteur vers compte,
- provenance des utilisateurs,

- nombre de demandes de demo ou contacts entrants.

34.10.3 10.3 Engagement

- nombre de recherches effectuees,
- taux d'utilisation des filtres,
- frequence de consultation des pages elus,
- taux d'ouverture des notifications ou emails,
- nombre d'exports ou comparaisons lances.

34.10.4 10.4 Fidelisation

- retention a 7, 30 et 90 jours,
- nombre d'utilisateurs recurrentes,
- nombre d'elus suivis par utilisateur,
- taux de desactivation des notifications,
- feedback positif / demandes de fonctionnalites.

34.11 Conclusion

POPPolitics a un positionnement marketing pertinent parce qu'il repond a un manque reel : la difficulte a comprendre rapidement et sereinement l'activite parlementaire. Le projet dispose d'un angle clair, d'une proposition de valeur lisible et d'une capacite a toucher plusieurs publics sans diluer sa mission.

La strategie recommandee est la suivante :

- lancer avec une promesse simple et forte pour le grand public,
- prouver la profondeur d'analyse aupres des journalistes et experts,
- construire une marque credible par la transparence et l'open source,
- transformer le projet d'etudes en produit durable avec une feuille de route publique.

Le succes marketing de POPPolitics ne dependra pas uniquement de la communication, mais de l'alignement entre trois elements : **la fiabilite des donnees, la clarte de l'experience utilisateur et la regularite du rayonnement public.**

35 POPPolitics – Plan de Communication Externe

35.1 1. Introduction

Ce document décrit la **stratégie de communication externe** du projet POPPolitics : comment le projet se présente, se diffuse et interagit avec le monde en dehors de l'équipe (grand public, journalistes, école, jury, communauté civic-tech).

La communication **interne** (équipe, Scrum, GitHub) est traitée dans [05-communication-interne.md](#).

35.2 2. Objectifs de communication

Objectif	Description	Horizon
Notoriété	Faire connaître POPolitics auprès des cibles pertinentes (citoyens, journalistes, civic-tech)	Moyen terme
Crédibilité	Démontrer la fiabilité des données et la rigueur du projet	Court terme
Engagement	Attirer de premiers utilisateurs testeurs et recueillir des retours	Court terme
Légitimité académique	Valoriser le projet auprès du jury et de l'école Epitech	Immédiat
Rayonnement	Positionner POPolitics comme une référence civic-tech open source	Long terme

35.3 3. Cibles de communication

35.3.1 3.1 Cibles primaires

Cible	Profil	Canal privilégié	Message clé
Citoyens engagés	25–45 ans, actifs politiquement, à l'aise avec le numérique	LinkedIn, X (Twitter), Instagram	« Enfin un outil simple pour comprendre qui vote quoi »
Journalistes / Data journalistes	Presse indépendante, médias en ligne, blogueurs data	LinkedIn, X (Twitter), Mastodon	« Des données propres et croisées, prêtes à l'emploi »
Jury & école Epitech	Encadrants pédagogiques, correcteurs	Présentation orale, rapport, démo live	« Projet rigoureux, technique maîtrisée, valeur civic-tech démontrée »
Autorités &	Institutions et	Email ciblé,	« Un outil open

Cible	Profil	Canal privilégié	Message clé
associations pour la transparence publique	organisations œuvrant pour la transparence de la vie publique et le débat démocratique	LinkedIn, événements civic-tech	source qui rend l'activité parlementaire lisible et vérifiable par tous »

Exemples d'acteurs ciblés :

Organisation	Type	Lien
Haute Autorité pour la Transparence de la Vie Publique (HATVP)	Autorité administrative indépendante	Transparence des mandats et patrimoines des élus
Regards Citoyens	Association civic-tech	Données ouvertes sur le Parlement français
Démocratie Ouverte	Association	Innovation démocratique et participation citoyenne
La Quadrature du Net	Association	Libertés numériques et transparence publique
Open Data	Écosystème open data	Réutilisation des données publiques
Commission d'accès aux documents administratifs (CADA)	Autorité administrative indépendante	Accès aux documents publics

35.3.2 3.2 Cibles secondaires

Cible	Profil	Canal privilégié	Message clé
Communauté civic-tech	Associations, développeurs open source, think tanks	GitHub, forums civic-tech, événements	« Projet open source contribuant à la transparence démocratique »
Experts politiques	Chercheurs, consultants, analystes	LinkedIn, email direct	« Un outil d'analyse multi-institutionnel inédit »
Étudiants / développeurs	Étudiants en informatique ou sciences politiques	GitHub, forums, Discord public	« Un projet open source pour apprendre et contribuer »

35.4 4. Messages clés

35.4.1 Message principal

POPolitics rend l'activité parlementaire accessible à tous — en transformant des milliers de votes bruts en insights clairs, personnalisés et vérifiables.

35.4.2 Messages secondaires

- **Fiabilité** : « Des données officielles, mises à jour quotidiennement, issues de trois institutions publiques. »
 - **Simplicité** : « Comprendre comment vote votre élu en moins de 30 secondes. »
 - **Profondeur** : « Des analyses croisées AN / Sénat / UE introuvables ailleurs. »
 - **Transparence** : « Projet open source, données sous licence ouverte, méthodologie documentée. »
 - **IA au service du citoyen** : « Des résumés automatiques pour rester informé sans être submergé. »
-

35.5 5. Canaux et stratégie par canal

35.5.1 5.1 GitHub (dépôt public)

Objectif	Crédibilité technique, transparence, contribution communautaire
Contenu	README clair, documentation complète, badges CI/CD, CONTRIBUTING.md
Fréquence	En continu (commits réguliers visibles)
Responsable	Lazare (DevOps) + Jaures (Tech Lead)
KPI	Nombre de stars, forks, issues ouvertes par des tiers

Actions clés : - Rédiger un README attractif avec captures d'écran et GIF de démo dès le Sprint 4. - Ajouter un badge "build passing", couverture de tests, licence. - Publier un CONTRIBUTING.md pour inviter les contributions externes.

35.5.2 5.2 LinkedIn

Objectif	Notoriété professionnelle, crédibilité académique, networking civic-tech
Contenu	Posts de progression (sprints, fonctionnalités), articles de fond, démo finale

Fréquence	1 post toutes les 2–3 semaines (calé sur les fins de sprint)
Responsable	Samy (PO)
Ton	Professionnel, engagé, pédagogique

Calendrier éditorial LinkedIn :

Sprint	Sujet de publication
Sprint 0	Lancement du projet — présentation équipe & vision
Sprint 2	Les coulisses de l'ETL — comment on ingère les données parlementaires
Sprint 4	MVP en ligne — première démo du portail collectif
Sprint 7	Focus UX — comment on a conçu la page personnelle
Sprint 9	IA & politique — comment on résume automatiquement un débat parlementaire
Sprint 11	Article bilan — ce qu'on a appris en construisant POPolitics

35.5.3 5.3 X (Twitter) / Mastodon

Objectif	Visibilité rapide, engagement communauté civic-tech et journalistes data
Contenu	Résultats d'analyse insolites, captures de la plateforme, citations chocs
Fréquence	1–2 posts par semaine à partir du Sprint 4
Responsable	Samy + Evilavy
Ton	Direct, factuel, parfois provocateur (chiffres surprenants)

Exemples de posts : - « 73 % des amendements sur l'environnement rejetés depuis 2022 — visualisé avec POPolitics. [\[lien\]](#) » - « Votre député a voté contre ce texte 4 fois de suite. On a rendu ça visible. » - « Open source, données publiques, 0 trackers — voilà POPolitics. »

35.5.4 5.4 Présentation jury / soutenance

Objectif	Valider la qualité académique et technique du projet
-----------------	--

Format	Diaporama (15–20 min) + démo live (10 min) + Q&A (10 min)
Responsable	Samy (présentation) + Jaures (démo technique)
Date cible	Fin Sprint 11

Structure du pitch de soutenance :

1. **Problème** (2 min) : la démocratie représentative est opaque pour le citoyen moyen.
2. **Solution** (2 min) : POPolitics — données brutes → insights actionnables.
3. **Architecture technique** (4 min) : ETL, datamarts, Django, Next.js, couche IA.
4. **Démo live** (10 min) : parcours utilisateur complet + fonctionnalité phare.
5. **Impact & perspectives** (2 min) : open source, évolutivité, données futures.

35.5.5 5.5 Email / Relations presse

Objectif	Cibler les journalistes data et les associations civic-tech
Format	Email de présentation personnalisé + lien vers démo
Fréquence	Ponctuel — à partir du Sprint 11 (version stable)
Responsable	Samy
Cibles prioritaires	Médiapart data, Next INpact, Regards Citoyens, La Quadrature du Net

35.6 6. Ligne éditoriale & charte de communication

35.6.1 Ton général

- **Factuel et sobre** : on cite les données, on ne dramatise pas.
- **Accessible** : on évite le jargon technique en communication externe.
- **Engagé** : on croit à la transparence démocratique, on l'affirme.
- **Honnête** : projet académique open source, pas un produit commercial.

35.6.2 Ce qu'on ne fait PAS

- Pas de prise de position partisane — POPolitics est **apolitique**.
- Pas de communication sur des données non vérifiées.
- Pas de publication de données personnelles au-delà du mandat public des élus.
- Pas de promesses de fonctionnalités non livrées.

35.6.3 Identité visuelle (à définir en Sprint 4)

Élément	Statut
Logo POPolitics	À créer (responsable : Evilavy)
Palette de couleurs	À définir avec la charte UI
Typographie	À définir avec la charte UI
Captures d'écran officielles	Disponibles à partir du Sprint 4

35.7 7. Calendrier de communication global

35.7.1 7.1 Pendant le projet (calé sur les sprints)

Période	Événement	Action communication
Sprint 0	Lancement projet	Post LinkedIn “on démarre” + publication dépôt GitHub
Sprint 2	ETL fonctionnel	Post LinkedIn “coulisses data”
Sprint 4	MVP Portail	Post LinkedIn + captures d'écran + README mis à jour
Sprint 7	Page personnelle	Post LinkedIn focus UX + X/Twitter
Sprint 9	Couche IA	Post LinkedIn “IA et politique”
Sprint 11	Soutenance	Présentation jury + article bilan LinkedIn + communication école

35.7.2 7.2 Post-projet — Calendrier éditorial (après soutenance)

L'objectif post-projet est de **capitaliser sur le travail accompli** : valoriser le projet sur les CV, attirer des contributions open source et ancrer POPolitics comme référence civic-tech.

Échéance	Canal	Action	Responsable
J+1 semaine	GitHub	Nettoyage final du dépôt : README complet, captures, GIF de démo, licence claire, CONTRIBUTING.md	Lazare + Jaures
J+1 semaine	LinkedIn	Article bilan : « Ce qu'on a appris en construisant une	Samy

Échéance	Canal	Action	Responsable
		<i>plateforme civic-tech en M2</i> » — retour d'expérience technique et humain	
J+2 semaines	X / Mastodon	Thread de présentation : vision, architecture, stack, résultats — avec captures et lien GitHub	Samy + Evilavy
J+2 semaines	Email ciblé	Envoi du projet à 5–10 organisations civic-tech pertinentes (Regards Citoyens, La Quadrature du Net, Démocratie Ouverte...)	Samy
J+1 mois	LinkedIn	Post “données insolites” : une analyse concrète produite avec POPolitics (ex. taux d'abstention record, élu le plus atypique...)	Thomas Marchand — ou l'équipe
J+1 mois	GitHub	Ouverture des issues “good first issue” pour inviter des contributions externes	Jaures
J+2 mois	LinkedIn	Post de chaque membre sur son rôle dans le projet — valorisation individuelle (portfolio, recrutement)	Chaque membre
J+3 mois	Événements	Soumission d'une présentation à un événement civic-	Samy

Échéance	Canal	Action	Responsable
		tech ou tech étudiant (ex. Open Source Experience, DataDays...) si opportunité	
Continu	GitHub	Maintenance légère : réponse aux issues, merge des PR externes, mise à jour des dépendances critiques	Lazare

Principes du calendrier post-projet :

POPOLitics n'est pas un projet académique qui s'arrête à la soutenance — c'est un **produit qui continue de vivre et d'évoluer**. La communication post-études doit refléter cette ambition.

- **Maintenir le rythme** : continuer à publier régulièrement, même à fréquence réduite, pour ancrer POPOLitics dans la durée.
- **Faire grandir la communauté** : chaque publication est une opportunité d'attirer des contributeurs, des partenaires ou de futurs utilisateurs réguliers.
- **Documenter la suite** : publier une roadmap publique sur GitHub pour montrer que le projet avance (nouvelles données, nouvelles fonctionnalités, nouvelles institutions).
- **Construire une identité durable** : à terme, POPOLitics peut viser une reconnaissance institutionnelle (partenariats avec des médias, des associations, voire des institutions publiques).
- **Valoriser sans s'éparpiller** : la croissance post-études doit rester maîtrisée — mieux vaut un projet qui tient ses promesses sur le long terme qu'une communication ambitieuse non suivie d'effets.

35.8 8. Responsabilités

Tâche	Responsable	Support
Posts LinkedIn	Samy	Toute l'équipe (relecture)
Posts X / Mastodon	Samy, Evilavy	—
README GitHub	Lazare, Jaures	Samy (contenu)
Présentation soutenance	Samy	Jaures (partie technique)
Identité visuelle	Evilavy	Raphaël
Relations presse / email	Samy	—

35.9 9. Indicateurs de succès (KPIs)

KPI	Cible	Mesure
Stars GitHub	> 20 avant soutenance	GitHub Insights
Vues README GitHub	> 200	GitHub Traffic
Vues posts LinkedIn	> 500 par post (en moyenne)	LinkedIn Analytics
Engagement communauté	≥ 1 retour externe (issue, message) avant Sprint 11	GitHub Issues / DM
Note jury soutenance	≥ 15/20	Feedback école